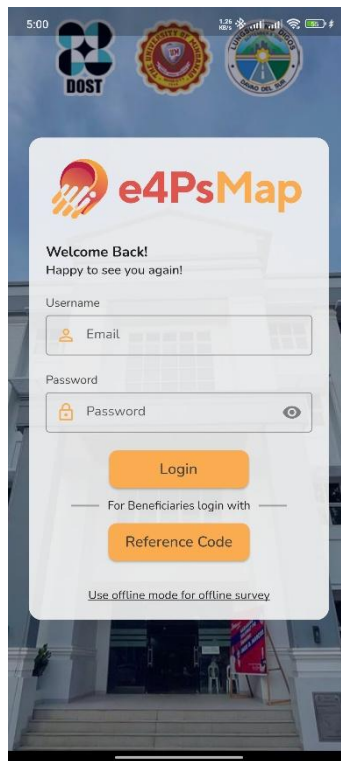




**E4PSMAP: A MOBILE APPLICATION FOR 4PS
BENEFICIARIES FOR THE CITY OF DIGOS WITH GEO
TAGGING TECHNOLOGY**

RALPH IAN O. NOYNAY
RHONA MAE MAGSAYO
JONALYN E. CALVO
JOHN RAVEN V. MANULAT

SOURCE CODE



```
import 'dart:convert';
import 'dart:io';
```

```
import
'package:e4psmapmig/funtions/toast_files.dart';
import
'package:e4psmapmig/pages/views/dashboard.dar
t';
import
'package:e4psmapmig/pages/views/dashboard_on
line.dart';
import 'package:flutter/material.dart';
import 'package:fluttertoast/fluttertoast.dart';
import
'package:permission_handler/permission_handler.
dart';
```

```
import '../Database/get.dart';
import '../Database/online_api_accessor.dart';
import '../Database/sqlitehelpernew.dart';
import '../config/variables.dart' as globals;
import '../funtions/alert_widget.dart';
import 'beneficiary_dashboard.dart';
```

```
class Login extends StatefulWidget {
  const Login({Key? key}) : super(key: key);
```

```
@override
State<Login> createState() => _LoginState();
}
```

```
class _LoginState extends State<Login> {
  bool isChecked = false;
  bool showcode = true;
  Color? yellow = const Color.fromRGBO(251, 171,
80, 1);
  bool internet_connected = false;
  String offlinecode = "";
  final TextEditingController _email_controller =
TextEditingController();
  final TextEditingController _passw_controller =
TextEditingController();
  final TextEditingController _offline_code =
TextEditingController();
  TextEditingController refcode =
TextEditingController();
  bool loggingin = false;
```

```
bool isloading = false;
bool showpass = true;
```

```
Future Checkinternet() async{
  try{
```

```

    final result = await
    InternetAddress.lookup('google.com');
    if (result.isNotEmpty &&
result[0].rawAddress.isNotEmpty){
      print('Connected');
      setState(() {
        internet_connected = true;
      });
    }else{
      print('No internet');
    }

}

}catch (e){
  print('No internet$e');
  internet_connected = false;
}
}

fluttertoast(){
  Fluttertoast.showToast(msg: 'Logging in please
wait ...',
    toastLength: Toast.LENGTH_SHORT,
    gravity: ToastGravity.CENTER,
    timeInSecForIosWeb: 2,
    backgroundColor: Colors.grey,
    textColor: Colors.white,
    fontSize: 18
  );
}

```

```

void checkpermstorage()async{
  var stat = await Permission.storage.status;

  if(stat.isDenied){
    await Permission.storage.request();
  }
}

int haveoffcode = 0;

countoffpas()async{
  var dbhelper = DBhelper_qualification_new();
  var num = await dbhelper.count_code();
  setState(() {
    haveoffcode = num!;
  });
  if(haveoffcode > 0){
    setter();
  }
}

```

```

}

getcode()async{
  var dbhelper = DBhelper_qualification_new();
  var offcodes = await dbhelper.getcode();
  return offcodes;
}

setter()async{
  var jsoneCodes = await getcode();
  var decodeCodes = await
jsonDecode(jsoneCodes);

  setState() {
    offlinecode = decodeCodes[0]['offline_code'];
  });
}

@override
void initState() {

  countoffpas();
  Checkinternet();
  checkpermstorage();
  setter();
  super.initState();
}

@override
Widget build(BuildContext context) {
  return Container(
    decoration: const BoxDecoration(image:
DecorationImage(image:
AssetImage('assets/mobilebg.png'), fit:
BoxFit.cover)),
    child: Scaffold(
      backgroundColor: const
Color(0x00070d30).withOpacity(0.4),
      body: Stack(
        children: [
          Center(
            child: SingleChildScrollView(
              child: Container(
                alignment: Alignment.topCenter,
                decoration: BoxDecoration(color:
Colors.grey[100]?.withOpacity(0.9), borderRadius:
const BorderRadius.all(Radius.circular(15))),

```



```

Center(child: ElevatedButton(
  onPressed: () async{
    Checkinternet();

    try{
      if (internet_connected){
        // fluttertoast();
        isloading = true;

        var authMethod = await
checkpass(_email_controller.text,
_passw_controller.text);
        setState((){isloading = false;});
        if (authMethod[0]['granted'] ==
'true'){

Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
Homepage_online()));
          globals.user_perm =
authMethod[0]['position'];
          print(globals.user_perm);
          globals.online_access = true;
          globals.user_name =
authMethod[0]['bhw_name'];
          globals.user_brgy =
authMethod[0]['barangay'];
          globals.user_id =
authMethod[0]['e4ps_user_id'];
        }else if (authMethod[0]['granted']
== 'false'){
          print(isloading);
          showDialog <String>(context:
context,
            builder: (BuildContext context)
=> AlertDialog(
              icon: const Icon(Icons.error,
color: Colors.red,size: 50,),
              title: const Text("Incorrect
Login"),
              content: const Text("Email
and password does not exist"),
              actions: [
                TextButton(onPressed: () =>
Navigator.pop(context), child: const Text("OK"))
              ],
            ));
        }else{

```

```

          showDialog <String>(context:
context,
            builder: (BuildContext context)
=> AlertDialog(
              icon: const Icon(Icons.error,
color: Colors.red,size: 50,),
              title: const Text("Something
went wrong"),
              content: const Text("Error"),
              actions: [
                TextButton(onPressed: () =>
Navigator.pop(context), child: const Text("OK"))
              ],
            ));
        }
      }else{
          showDialog <String>(context:
context,
            builder: (BuildContext context)
=> AlertDialog(
              icon: const
Icon(Icons.signal_wifi_statusbar_connected_no_in
ternet_4, color: Colors.red,size: 50,),
              title: const Text("No Internet
Connection"),
              content: const Text("Please
check your internet connection"),
              actions: [
                TextButton(onPressed: () =>
Navigator.pop(context), child: const Text("OK"))
              ],
            ));
        }
      }catch(e){
          showDialog <String>(context:
context,
            builder: (BuildContext context)
=> AlertDialog(
              icon: const Icon(Icons.error,
color: Colors.red,size: 50,),
              title: const Text("Something
went wrong"),
              content: const Text("An error
has ocured while logging in to the application"),
              actions: [
                TextButton(onPressed: () =>
Navigator.pop(context), child: const Text("OK"))
              ],
            ));
        }
      }
    }
  }
}

```

```

    },
    style: ElevatedButton.styleFrom(
      backgroundColor: yellow,
      padding: const
EdgeInsets.symmetric(horizontal: 60, vertical: 10),
      shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))
    ),
    child: Text('Login', style:
TextStyle(color: Colors.grey[800], fontSize: 18),),),),
    const SizedBox(height: 10,),

    Container(
      padding: const
EdgeInsets.symmetric(horizontal: 30),
      child: const Row(
        mainAxisAlignment:
MainAxisAlignment.center,
        crossAxisAlignment:
CrossAxisAlignment.center,
        children: [
          Expanded(child: Divider(thickness:
2, height: 2, color: Colors.grey,endIndent: 10,)),
          Text("For Beneficiaries login with"),
          Expanded(child: Divider(thickness:
2, height: 2, color: Colors.grey,indent: 10,)),
        ],
      ),
    ),
    const SizedBox(height: 10,),

    Center(
      child: ElevatedButton(onPressed: (){

        showDialog <String>(context:
context,

        builder: (BuildContext context)
=> AlertDialog(
          scrollable: true,
          title: const Text("Enter
Reference Code", style: TextStyle(fontSize: 25)),
          content: Padding(padding:
const EdgeInsets.all(8),
            child: Form(

```

```

        child: Column(
          crossAxisAlignment:
CrossAxisAlignment.start,
          children: [
            const Text('Code', style:
TextStyle(fontSize: 18),),
            TextFormField(controller:
refcode, decoration: const InputDecoration(border:
OutlineInputBorder(),

            contentPadding:EdgeInsets.only(top: 10, bottom:
10, left: 20),),),
          ],
        ),
      ),
    ),
    actions: [
      TextButton(onPressed: () =>
Navigator.pop(context), child: const
Text("Cancel")),
      TextButton(onPressed: ()async{
        globals.user_perm = 'User';

        globals.user_data = await
e4ps_ref_code(refcode.text);
        if(globals.user_data.isEmpty){
          String refcodee =
refcode.text;

          alert_global(context, 'No
Data', 'No data corresponding reference code:
$refcodee', const Icon(Icons.error, color:
Colors.blue, size: 60,));
        }else{

          Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
benifeciary_dashboard()));
        }

      }, child: const Text("Submit"))
    ],
  ));
//
Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=>
Homepage()));
    },style: ElevatedButton.styleFrom(
      backgroundColor: yellow,
      padding: const
EdgeInsets.symmetric(horizontal: 20, vertical: 10),

```

```

        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))
        ), child: Text('Reference Code', style:
TextStyle(color: Colors.grey[800], fontSize: 18),),),
    ),
    // Center(
    // child: ElevatedButton(onPressed:
() {
    // showDialog <String>(context:
context,
    // builder: (BuildContext context)
=> AlertDialog(
    // scrollable: true,
    // title: const Text("Enter Offline
Code", style: TextStyle(fontSize: 25)),
    // content: Padding(padding:
EdgeInsets.all(8),
    // child: Form(
    // child: Column(
    // crossAxisAlignment:
CrossAxisAlignment.start,
    // children: [
    // Text('Code', style:
TextStyle(fontSize: 18),),
    //
    TextFormField(obscureText: true ,controller:
_offline_code, decoration:
InputDecoration(border: OutlineInputBorder(),
    //
    contentPadding:EdgeInsets.only(top: 10, bottom:
10, left: 20),),),
    // ],
    // ),
    // ),),
    // actions: [
    // TextButton(onPressed: () =>
Navigator.pop(context), child: Text("Cancel")),
    // TextButton(onPressed: (){
    // if(_offline_code.text ==
globals.offlinepass){
    // Navigator.pop(context);
    // globals.user_perm = 'Field
Worker';
    //
    // globals.online_access =
false;

```

```

        // globals.profile_image =
AssetImage('assets/offline_mode.png');
        //
Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=>
Homepage()));
        // }else{
        // fluttertoast_func('Incorrect
Code');
        // }
        // }, child: Text("Submit"))
        // ],
        // ));
        // //
Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=>
Homepage()));
        // }, child: Text('Go offline Mode',
style: TextStyle(color: Colors.grey[800], fontSize:
18),),style: ElevatedButton.styleFrom(
        // primary: yellow,
        // padding:
EdgeInsets.symmetric(horizontal: 20, vertical: 10),
        // shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))
        // ),),
        // ),
        // ElevatedButton(onPressed: (){
        // globals.user_perm = 'User';
        //
Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=>
beneficiary_dashboard()));
        // }, child: Text('User'))
        const SizedBox(height: 30,),
        Center(
        child: GestureDetector(
        child: const Text('Use offline mode
for offline survey',
        textAlign: TextAlign.end,
        style: TextStyle(decoration:
TextDecoration.underline),),
        onTap: (){
        if(haveoffcode >0){

```

```

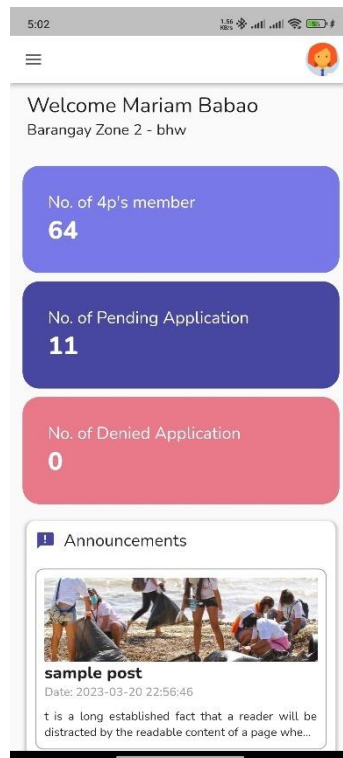
        showDialog <String>(context:
context,
        builder: (BuildContext
context) => AlertDialog(
        scrollable: true,
        title: const Text("Enter
Offline Code", style: TextStyle(fontSize: 25)),
        content: StatefulBuilder(
        builder:(BuildContext
context, StateSetter setState){
        return Padding(padding:
const EdgeInsets.all(8),
        child: Form(
        child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
        const Text('Code',
style: TextStyle(fontSize: 18)),
TextFormField(obscureText: showcode ,controller:
_offline_code,
        decoration:
InputDecoration(border: const
OutlineInputBorder(),
        suffixIcon:
IconButton(onPressed: (){
        setState(){
        showcode =
!showcode;
        });
        }, icon:
Icon(showcode ? Icons.visibility :
Icons.visibility_off),
        ),
        contentPadding:const EdgeInsets.only(top: 10,
bottom: 10, left: 20)),),
        ],
        ),
        ),);
        }
        ),
        actions: [
        TextButton(onPressed: ()
=> Navigator.pop(context), child: const
Text("Cancel")),
        TextButton(onPressed: (){

```

```

        if(_offline_code.text ==
offlinecode){
        Navigator.pop(context);
        globals.user_perm =
'Field Worker';
        globals.online_access =
false;
        globals.profile_image =
const AssetImage('assets/offline_mode.png');
        Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
Homepage()));
        }else{
        flutterstoast_func('Incorrect Code');
        }, child: const
Text("Submit"))
        ],
        ));
        }else{
        flutterstoast_func('Login to online
first to setup device offline code');
        },),
        ),
        ),
        ),
        ),
        Visibility(visible: isloading,child:
Container(padding: const
EdgeInsets.all(20),decoration: const
BoxDecoration(shape: BoxShape.rectangle, color:
Colors.black38),child: Center(child:isloading?const
CircularProgressIndicator(strokeWidth: 10,
valueColor: AlwaysStoppedAnimation<Color>(
Color.fromRGBO(75, 79, 177, 1)
),):const Text("Loading"))),),
        ],
        ),
        ),
        );
        }
        }

```



```
import
'package:e4psmapmig/funtions/Header.dart';
import 'package:flutter/material.dart';

import '../Database/dashboard.dart';
import
'../Database/online_announcements.dart';
import '../Database/online_api_accessor.dart';
import '../funtions/anouncements_pages.dart';
import '../funtions/sidemenu.dart';
import '../config/variables.dart' as globals;
import '../config/variables.dart';
import '../funtions/willscope_router.dart';

class Homepage_online extends StatefulWidget {
  const Homepage_online({Key? key}) : super(key:
key);

  @override
  State<Homepage_online> createState() =>
_Homepage_onlineState();
}
class _Homepage_onlineState extends
State<Homepage_online> {

// Color textcol = Colors.black54;
var name = 'Devmaster';
var numerooflog = '0';
var numeroofpedning = 'Loading Data...';
var numeroofdenied = 'Loading Data...';
var barangay_quant = '26';
var numeroof4ps = '4,296';
var dateupdated = 'Month year';
var dateupdated2 = 'Month year';
var dateupdated3 = 'Month year';
var num_applicants = '1,000';
var num_tobeapproved = '780';
var num_of_applicants;
var num_of_pending_applicants;
var num_of_denied_applicants;
var announcementlist;
bool waiting = true;
String barangay = "";
String position = "";

count_appl()async{
  var applCounter = await
count_applicant(globals.user_brgy);
  setState(() {
    num_of_applicants = applCounter;

```

```

        waiting = false;
    });
}

count_pending()async{
    setState(() {
        numberofpedning = "Loading Data...";
    });
    var numpennding = await
count_pending_appl("0");
    setState(() {
        numberofpedning = numpennding.toString();
    });
}

count_denied()async{
    setState(() {
        numberofdenied = "Loading Data...";
    });
    var numden = await count_pending_appl("2");
    setState(() {
        numberofdenied = numden.toString();
    });
}

Future loadannouncement()async{
    setState(() {
        announcmentlist = [];
    });
    var announcements = await
getannouncements();
    setState(() {
        announcmentlist = announcements;
    });
}

@override
void initState() {
    count_appl();
    count_pending();
    count_denied();
    loadannouncement();
    // countProf();
    setState(() {
        barangay = globals.user_brgy;
        position = globals.user_perm;
    });
}

```

```

    super.initState();
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: top_header(context),
        drawer: const Navigationmenu(),
        body: WillPopScope(
            onWillPop: () async{
                return await scope_toout(context);
            },
            child: SingleChildScrollView(
                child: Container(
                    child: Column(crossAxisAlignment:
CrossAxisAlignment.start,
                    children: [
                        Padding(
                            padding: const EdgeInsets.only(top: 10,
left: 20),
                            child: Text('Welcome
${globals.user_name}',style: const
TextStyle(fontFamily: 'nunito', fontSize: 25,)),
                        ),
                        Padding(
                            padding: const EdgeInsets.only(left: 20),
                            child: Row(children: [
                                // Text(numberoflog , style:
TextStyle(fontWeight: FontWeight.w800, color:
Colors.blue, fontSize: 20),),
                                Text('Barangay $barangay - $position',
style: const TextStyle(fontSize: 18),)
                            ]),
                        ),
                        const SizedBox(height: 30,),

                        // Container(
                        //   height: 170,
                        //   width: 500,
                        //   child: Dash_Pages(),
                        // ),
                        // Container(
                        //   margin: EdgeInsets.fromLTRB(15, 0,
15,0),
                        //   padding:
EdgeInsets.symmetric(horizontal: 30, vertical: 30),
                        //   decoration: BoxDecoration(color:

```

```

Color.fromRGBO(125, 160, 250, 1), borderRadius:
BorderRadius.all(Radius.circular(20))),
    // child: Column(
    //   crossAxisAlignment:
CrossAxisAlignment.start,
    //   children: [
    //     Text('Digos City\'s Barangay', style:
TextStyle(color: Colors.white, fontSize: 20)),
    //     Row(
    //       children: [
    //         Text(barangay_quant, style:
TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30)),
    //         Text('Barangays', style:
TextStyle(color: Colors.white, fontSize: 20)),
    //       ],
    //     )
    //   ],
    // ),
    // SizedBox(height: 10),
Container(
  margin: const EdgeInsets.fromLTRB(15,
0, 15, 0),
  padding: const
EdgeInsets.symmetric(horizontal: 30, vertical: 30),
  decoration: const BoxDecoration(color:
Color.fromRGBO(121, 120, 233, 1), borderRadius:
BorderRadius.all(Radius.circular(20))),
  child: Column(
    crossAxisAlignment:
CrossAxisAlignment.start,
    children: [
      const Text('No. of 4p\'s member',
style: TextStyle(color: Colors.white, fontSize: 20)),
      Row(
        children: [
          Visibility(visible: waiting, child:
const Text("Loading Data...", style:
TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30))),
          Visibility(visible: !waiting, child:
Text(num_of_applicants.toString(), style: const
TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30))),
        ],
      ),
    ],
  ),
)

```

```

    ],
  ),
),
const SizedBox(height: 10),
Container(
  margin: const EdgeInsets.fromLTRB(15,
0, 15, 0),
  padding: const
EdgeInsets.symmetric(horizontal: 30, vertical: 30),
  decoration: const BoxDecoration(color:
Color.fromRGBO(71, 71, 161, 1), borderRadius:
BorderRadius.all(Radius.circular(20))),
  child: Column(
    crossAxisAlignment:
CrossAxisAlignment.start,
    children: [
      const Text('No. of Pending
Application', style: TextStyle(color: Colors.white,
fontSize: 20)),
      Row(
        children: [
          Text(numberofpending, style: const
TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30)),
          // Text(' ' + dateupdated, style:
TextStyle(color: Colors.white, fontSize: 20)),
        ],
      )
    ],
  ),
const SizedBox(height: 10),
Container(
  margin: const EdgeInsets.fromLTRB(15,
0, 15, 0),
  padding: const
EdgeInsets.symmetric(horizontal: 30, vertical: 30),
  decoration: const BoxDecoration(color:
Color.fromRGBO(233, 120, 137, 1.0),
borderRadius:
BorderRadius.all(Radius.circular(20))),
  child: Column(
    crossAxisAlignment:
CrossAxisAlignment.start,
    children: [
      const Text('No. of Denied
Application', style: TextStyle(color: Colors.white,
fontSize: 20)),
      Row(

```



```
//
Text(currentitem['e4ps_ann_title'], style:
TextStyle(fontWeight: FontWeight.w800, fontSize:
20),),
//          Text('Date: ' +
currentitem['e4ps_ann_created'],style:
TextStyle(fontWeight: FontWeight.w500, fontSize:
14, color: Colors.grey), ),
//          SizedBox(height: 10,),
//          InkWell(child:
Text(currentitem['e4ps_ann_content'], style:
TextStyle(), textAlign: TextAlign.justify, maxLines:
2, overflow: TextOverflow.ellipsis,), onTap: (){
//          print("hello");
//
Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=>
Announcement()));
//          },),
//          ],
//          ),
//          );
//          }
//          ),
//          ],
//          ),
//          );
//          }
//          }
//          }
}
```

```
Widget widget_barangays(String brgy, String
data){
return Column(
children: [
Row(
children: [
Expanded(child: Text(brgy, style: const
TextStyle(fontSize: 18, fontWeight:
FontWeight.w400)),),
Expanded(child: Text(data, style: const
TextStyle(fontSize: 18, fontWeight:
FontWeight.w400)),),
],
),
const SizedBox(height: 10,),
],
);
```

```
}

class Dash_Pages extends StatefulWidget {
  const Dash_Pages({Key? key}) : super(key: key);

  @override
  State<Dash_Pages> createState() =>
    _Dash_PagesState();
}

class _Dash_PagesState extends
State<Dash_Pages> {
  var name = 'Devmaster';
  var numberOflog = '0';
  var barangay_quant = '26';
  var numberOf4ps = '4,296';
  var dateupdated = 'Month year';
  var dateupdated2 = 'Month year';
  var dateupdated3 = 'Month year';
  var num_applicants = '1,000';
  var num_tobeapproved = '780';

  int pagechange = 0;
  final List <Color?> _color =
[Colors.blue[200],Colors.grey[400],Colors.grey[40
0],Colors.grey[400],];

  final PageController _pageController =
PageController(initialPage: 0);

  void isshown(){

    for (var i = 0; i<2; i++){
      if (i == pagechange){
        setState(() {
          _color[i] = Colors.blue[200];
        });
      }else{
        _color [i] = Colors.grey[400];
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        SizedBox(
```

```

width: 500,
height: 140,
child: PageView(
  controller: _pageController,
  onPageChanged: (index){
    setState(() {
      pagechange = index;
      isshown();
    });
  },
  children: <Widget> [
    // Text('data1'),
    // Text('data2'),
    // Text('data3')
    Container(
      margin: const EdgeInsets.fromLTRB(15, 0,
15,0),
      padding: const
EdgeInsets.symmetric(horizontal: 30, vertical: 30),
      decoration: const BoxDecoration(color:
Color.fromRGBO(125, 160, 250, 1), borderRadius:
BorderRadius.all(Radius.circular(20))),
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          const Text('Digos City\'s Barangay',
style: TextStyle(color: Colors.white, fontSize: 20)),
          Row(
            children: [
              Text(barangay_quant, style: const
TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30)),
              const Text('Barangays', style:
TextStyle(color: Colors.white, fontSize: 20),)
            ],
          )
        ],
      ),
      Container(
        margin: const EdgeInsets.fromLTRB(15, 0,
15,0),
        padding: const
EdgeInsets.symmetric(horizontal: 30, vertical: 30),
        decoration: const BoxDecoration(color:
Color.fromRGBO(71, 71, 161, 1), borderRadius:
BorderRadius.all(Radius.circular(20))),
        child: Column(

```

```

        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          const Text('Number of Data Collected',
style: TextStyle(color: Colors.white, fontSize: 20)),
          Row(
            children: [
              Text(databasecount.toString(), style:
const TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30)),
              // Text(' ' + dateupdated, style:
TextStyle(color: Colors.white, fontSize: 20),)
            ],
          )
        ],
      ),
      // Container(
      //   margin: EdgeInsets.fromLTRB(15, 0,
15,0),
      //   padding:
EdgeInsets.symmetric(horizontal: 30, vertical: 20),
      //   decoration: BoxDecoration(color:
Color.fromRGBO(121, 120, 233, 1), borderRadius:
BorderRadius.all(Radius.circular(20))),
      //   child: Column(
      //     crossAxisAlignment:
CrossAxisAlignment.start,
      //     children: [
      //       Text('4Ps Applicants', style:
TextStyle(color: Colors.white, fontSize: 20)),
      //       Row(
      //         children: [
      //           Text(num_applicants, style:
TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30)),
      //           Text(' ' + dateupdated2, style:
TextStyle(color: Colors.white, fontSize: 20),)
      //         ],
      //       )
      //     ],
      //   ),
      //   Container(
      //     margin: EdgeInsets.fromLTRB(15, 0,
15,0),
      //     padding:
EdgeInsets.symmetric(horizontal: 30, vertical: 20),
      //     decoration: BoxDecoration(color:

```

```

Color.fromRGBO(243, 121, 126, 1), borderRadius:
BorderRadius.all(Radius.circular(20))),
    // child: Column(
    //   crossAxisAlignment:
CrossAxisAlignment.start,
    //   children: [
    //     Text('For 4Ps Approval', style:
TextStyle(color: Colors.white, fontSize: 20)),
    //     Row(
    //       children: [
    //         Text(num_tobeapproved, style:
TextStyle(color: Colors.white, fontWeight:
FontWeight.w900, fontSize: 30)),
    //         Text(' ' + dateupdated3, style:
TextStyle(color: Colors.white, fontSize: 20),)
    //       ],
    //     ),
    //   ],
    ),
    const SizedBox(height: 5,),
    Container(
      child: Center(
        child: Row(
          crossAxisAlignment:
CrossAxisAlignment.center,
          mainAxisAlignment:
MainAxisAlignment.center,
          children: [
            Container(
              decoration: BoxDecoration(
                color: _color[0],
                shape: BoxShape.circle
              ),
              height: 8,
              width: 8,
            ),

```

```

const SizedBox(width: 5,),
Container(
  decoration: BoxDecoration(
    color: _color[1],
    shape: BoxShape.circle
  ),
  height: 8,
  width: 8,
),
// SizedBox(width: 5,),
// Container(
//   // decoration: BoxDecoration(
//   //   color: _color[2],
//   //   shape: BoxShape.circle
//   // ),
//   height: 8,
//   width: 8,
// ),
// SizedBox(width: 5,),
// Container(
//   // decoration: BoxDecoration(
//   //   color:_color[3],
//   //   shape: BoxShape.circle
//   // ),
//   height: 8,
//   width: 8,
// ),
// SizedBox(width: 5,),
],
),
),
),
],
);
}
}

```

The screenshot displays a mobile application interface with three main sections:

- Primary Questions:**
 - Household Head Applicant * (Dropdown menu)
 - Combine family monthly income * (Dropdown menu)
 - Number of family members (including applicant) * (Dropdown menu)
 - Is the applicant currently pregnant? * (Radio buttons for Yes/No)
 - If yes, please enter the pregnancy starting period * (Text input)
 - Does applicant have children aging 0-18 * (Radio buttons for Yes/No)
 - Year of Residence * (Text input)
- Household Head Profile:**
 - Note: Required Fields are indicated with *
 - Household Head Profile Image (Placeholder icon)
 - Last Name * (Text input)
 - First Name * (Text input)
 - Middle Name (Text input)
 - Suffix (Dropdown menu)
 - Gender * (Dropdown menu)
 - Civil Status * (Dropdown menu)
- Address:**
 - House Number/Building Name/Block (Text input)
 - Street Name * (Text input)
 - Barangay * (Dropdown menu)
 - Purok * (Text input)
 - City * (Text input)
 - Province * (Text input)
 - Zip Code * (Text input)
 - Latitude (Text input)
 - Longitude (Text input)

```
import 'dart:async';
import 'dart:io';
```

```
import
'package:e4psmapmig/Database/sqlitehelper.dart';
import
'package:e4psmapmig/Models/temp_models/hh_
members_temp.dart';
import
'package:e4psmapmig/funtions/Header.dart';
import 'package:exif/exif.dart';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:fluttertoast/fluttertoast.dart';
import 'package:geolocator/geolocator.dart';
import 'package:image_picker/image_picker.dart';
import
'package:path_provider/path_provider.dart';
import 'package:sqflite/sqflite.dart';
import '../Database/synchronize.dart';
import '../funtions/sidemenu.dart';
import 'package:intl/intl.dart';
import 'package:flutter_picker/flutter_picker.dart';
import 'package:path/path.dart' as paths;
import '../config/variables.dart' as globals;
```

```
import '../funtions/textwidget.dart';
import '../views/dashboard.dart';
import '../views/dashboard_online.dart';
```

```
// StreamController <bool> streamController =
StreamController<bool>();
```

```
class add_applicant_online extends StatefulWidget
{
  const add_applicant_online({Key? key}) :
  super(key: key);
```

```
  @override
  State<add_applicant_online> createState() =>
  add_applicant_onlineState();
}
```

```
class add_applicant_onlineState extends
State<add_applicant_online> {
  final ImagePicker _imagepicker = ImagePicker();
  final ImagePicker _ha_imagepicker =
ImagePicker();
  var dateunformatted;
  PickedFile? _imgfile;
  PickedFile? _ha_imgfile;
  String prof_serial = "";
  final GlobalKey <ScaffoldState> _scaffoldkey =
```

```

GlobalKey<ScaffoldState>();
int count = 0;
bool max = true;
int preg = 0;
int child = 0;
String ispreg = "";
String ischild = "";

List <bool> immun = [false,false];
List <String> immun_det = ['Influenza', 'Tetanus,
diphtheria, and whooping cough(Tdap; Td)'];
String selectedimmun = "";

final childrens = <String>[];
final preghist_arr = <String>[];
final preginfo_arr = <String>[];
int quant_child = 0;
int pregindex = 0;
int preghistindex = 0;
int updateindexes = 0;

void numkind_loop(){
  quant_child =
int.parse(_qual_num_children.text);
  for (var i = childrens.length; i < quant_child; i++){
    int ind = i+1;
    childrens.add(ind.toString());
    preginfo_arr.add(ind.toString());
  }
  // widget_numkids();
}

// void numpreginf_loop(){
//   pregindex =
int.parse(_txt_preg_numchild.text);
  //   for (var i = preginfo_arr.length; i < pregindex;
i++){
    //     int ind = i+1;
    //     preginfo_arr.add(ind.toString());
    //   }
  //   widget_preghist();
  // }

// void numpreghist_loop(){
//   preghistindex =
int.parse(_txt_preg_numchild.text) +
preghist_arr.length;
  //   for (var i = preghist_arr.length; i <

```

```

preghistindex; i++){
  //   int ind = i+1;
  //   preghist_arr.add(ind.toString() + '-' +
list_pregnaninf.length.toString());
  // }
  //   widget_preghist();
  // }

//end kids -----

//top navi enabler -----

bool enable_profile = false;
bool enable_cont = false;
bool enable_preg_inf= false;
bool enable_child= false;
bool enable_preg_hist = false;
bool enable_submit = false;
bool age_018 = false;
bool age_218 = true;
//end of top navi enabler -----

//radio buttons options -----
HavePregnant? _pregchoice;
underage? _agechoice;

bool yes_preg = true;
bool yes_age = false;
bool yes_preghist = false;
bool yes_preg_txt = false;
bool isloading = false;
bool getlatlongimg = false;

// end of radio buttons options -----
-----
// TEmporary lists -----
----
var temp_list_memb = [];
var obj_memb;

//combo boxes -----
---
var incomes = [
  'Select',
  'Below 5000',
  '5000 - 7499',
  '7500 - 9999',

```

```

    '10000 - 12000',
    '12000 Above'
];

var items = [
    'Select',
    'Father',
    'Mother',
    'Child'
];

var gender = [
    'Select',
    'Male',
    'Female'
];

var civilstat = [
    'Select',
    'Single',
    'Married',
    'Widowed',
    'Separated',
    'Annulled'
];

var typeofmember = [
    'Select',
    'Student',
    'Out of School youth',
    'Working',
    'Toddler'
];

var members = [
    'Select',
    'Mother',
    'Father',
    'Grandmother',
    'Grandfather',
    'Others'
];

var typeofdelivery = [
    'Select',
    'Normal',
    'Cesarian'
];

var outcomeofdelivery = [

```

```

    'Success',
    'Fail'
];

var school = [
    'Select',
    'Elementary',
    'High School',
    'College Undergraduate',
    'College Graduate'
];

var suffix = [
    'None',
    'Jr.',
    'Sr.',
    'II',
    'III',
    'IX'
];

var typeresidence = [
    'Select',
    'Rent',
    'Owned',
    'Living With Parents'
];

var housedesc = [
    'Select',
    'Concrete',
    'Semi-Concrete',
    'Wood',
    'Makeshift'
];

var bloodtype = [
    'Select',
    'A+',
    'A-',
    'O+',
    'O-',
    'B+',
    'B-',
    'AB+',
    'AB-',
    'Unknown'
];

```

```

var numbermemembers = [

'Select','1','2','3','4','5','6','7','8','9','10','11','12','13',
'14','15','16','17','18','19','20'
];

var vita = [
'Select',
' Folic Acid Tablets',
'None'
];

var post = [
'Select',
'Yes',
'No'
];

String selectedvita = 'Select';
String selectedpost = 'Select';
String selectedvitapost = 'Select';
String selectednumbermem = 'Select';
String selectedbloodtype = 'Select';
String selecttyperes = 'Select';
String selecthousedesc = 'Select';
String selectedbrgy = 'Select';
String selectedsuffix = 'None';
String selectedschool = 'Select';
String m_selectedschool = 'Select';
String selecttypedelivery = 'Select';
String selectoutcomedelivery = 'Select';
String selectedmemb = 'Select';
String incomeselect = 'Select';
String dropvalue = 'Select';
String dropvalue_member = 'Select';
String genderdrop = 'Select';
String ch_gender = 'Select';
String chp_gender = 'Select';
String mem_genderdrop = 'Select';
String civildrop = 'Select';
String member_civildrop = 'Select';
String m_civildrop = 'Select';
String mem_civildrop = 'Select';
String typedrop = 'Select';
//end -----
-----

int page = 0;

```

```

final PageController _pagecontroller =
PageController(initialPage: 0);

//Text input controllers -----
-----

//For Qualification -----
-----

final TextEditingController _qual_num_children =
TextEditingController();
final TextEditingController _qual_preg_date =
TextEditingController();
final TextEditingController _qual_years =
TextEditingController();

//For Profile -----
-----

final TextEditingController _profile_lname =
TextEditingController();
final TextEditingController _profile_fname =
TextEditingController();
final TextEditingController _profile_mname =
TextEditingController();
final TextEditingController _profile_bday =
TextEditingController();
final TextEditingController _profile_occupation =
TextEditingController();
final TextEditingController _profile_contnumber =
TextEditingController();
final TextEditingController _profile_email =
TextEditingController();

//for contact/address -----
-----

final TextEditingController _contact_housenum =
TextEditingController();
final TextEditingController _contact_street =
TextEditingController();
final TextEditingController _contact_lat =
TextEditingController();
final TextEditingController _contact_long =
TextEditingController();
final TextEditingController _contact_purok =
TextEditingController();
final TextEditingController _contact_city =
TextEditingController();
final TextEditingController _contact_province =
TextEditingController();
final TextEditingController _contact_zipcode =

```

```

TextEditingController();

String hold_member = 'Select';
final TextEditingController _hold_fullname =
TextEditingController();
String hold_gender = 'Select';
String hold_cs = 'Select';
final TextEditingController _hold_bday =
TextEditingController();
final TextEditingController _hold_occupation =
TextEditingController();
final TextEditingController _hold_contact =
TextEditingController();
final TextEditingController _hold_email =
TextEditingController();

// other variables -----
-----

String istosubmit = 'Add';
String btn_preginf = 'Next';
String btn_preghist = 'Add';
var profile_code;
String ref_code = '';
//pages for page view -----
-----

// void isshown(){
//
// if (page == 0){
//   setState() {
//     _height = 870;
//   };
// }else if (page == 1){
//   setState() {
//     _height = 1300;
//   };
// }else if (page == 2){
//   setState() {
//     _height = 750;
//   };
// }else if (page == 3){
//   setState() {
//     _height = 1230;
//   };
// }
// }

//scroll to top -----
-----

```

```

final ScrollController _scrollController =
ScrollController();

//database controllers
// adddata(){
//   var dbhelper;
//
//   dbhelper = DBhelper_qualification();
//   var quali =
model_qualif_model(_txt_quant_housenum.text,
dropvalue, selectedmemb, incomeselect,
int.parse(selectednumbermem),
//   int.parse(_qual_num_pregnant.text),
int.parse(_qual_num_children.text),
_qual_num_years.text, 'unsynced');
//   dbhelper.createdata(quali);
//   print('Saved Qualification');
//
//
//   dbhelper = DBhelper_preginfo();
//   var preginfo =
model_pregdetails_temp(_txt_quant_housenum.t
ext, 'PRG-'+_txt_quant_housenum.text,
selectedbloodtype,
double.parse(_txt_preg_height.text),
double.parse(_txt_preg_weight.text),
//   _txt_preg_allergies.text,
_txt_preg_medication.text,
_txt_preg_lastmens.text,int.parse(_txt_preg_numc
hild.text), 'unsynced' );
//   dbhelper.createdata(preginfo);
//
//   for (int i =0; i< list_pregnaninf.length; i++){
//     dbhelper = DBhelper_preghistory();
//     var preghist =
model_preghist_temp(list_pregnaninf[i]['hh_id'],
list_pregnaninf[i]['preghist_id'],
list_pregnaninf[i]['preghist_birthorder'],
list_pregnaninf[i]['preghist_weight1'],
list_pregnaninf[i]['preghist_weight2'],
//
list_pregnaninf[i]['preghist_weight3'],list_pregnani
nf[i]['preghist_immunization'],
list_pregnaninf[i]['preghist_vitamins'],list_pregnani
nf[i]['preghist_typedelivery'],list_pregnaninf[i]['pre
ghist_outcome'],list_pregnaninf[i]['preghist_postn
atal'],

```

```

//
list_pregnaninf[i]['preghist_weightpost'],list_pregn
aninf[i]['preghist_vitaminspost'],
list_pregnaninf[i]['status']);
// dbhelper.createdata(preghist);
// // print('Saved Preghist');
// }
//
// if(yes_age){
// // print('Saved Preginfo');
// for (int i = 0; i<list_children.length; i++){
// dbhelper = DBhelper_childdata();
// var ch_profile =
model_children_temp(list_children[i]['ch_id'],
list_children[i]['ch_ch_id'],
list_children[i]['ch_birth_order'],
list_children[i]['ch_name'],list_children[i]['ch_age']
,
list_children[i]['ch_gender'],list_children[i]['ch_hei
ght'], list_children[i]['ch_weight'],
// list_children[i]['ch_bday'],
list_children[i]['ch_birthplace'],
list_children[i]['ch_bakuna'],
list_children[i]['status']);
// dbhelper.createprofile(ch_profile);
// print('Saved Child');
// }
// }
//
//
// //to be looped
// }

addprofile()async{
profile_code = await
upload_profile(
copied,
_profile_lname.text,
_profile_fname.text,
_profile_mname.text,
selectedsuffix,
genderdrop,
civildrop,
dateconvert(_profile_bday.text),
selectedschool,
_profile_occupation.text,
_profile_contnumber.text,
_profile_email.text,
dropvalue,

```

```

incomeselect,
selectednumbermem,
ispreg,
_qual_preg_date.text,
'0',
ischild,
'0',
_qual_years.text,
selecttypeses,
selecthousedesc,
_contact_housenum.text,
_contact_street.text,
_contact_purok.text,
selectedbrgy,
_contact_city.text,
_contact_province.text,
_contact_zipcode.text,
_contact_lat.text,
_contact_long.text,
copied2,
globals.user_id
);

ref_code = profile_code[0]['refcode'];
}

addmembers()async{
for(int i = 1; i<temp_list_memb.length; i++){
await
upload_household(profile_code[0]['profileserial'],
temp_list_memb[i]['relation'],
temp_list_memb[i]['fullname'],dateconvert(temp_l
ist_memb[i]['bday']), temp_list_memb[i]['gender'],
temp_list_memb[i]['civil_stat'],
temp_list_memb[i]['occupation'],
temp_list_memb[i]['contact'],
temp_list_memb[i]['email']);
}
}

createbeneficiary()async{
await addprofile();
await addmembers();

print("done");
}

int index = 0;

```

```

int maxitems = 0;
String? converted;
String? converted2;
String copied = "";
String copied2 = "";

topermanentstorage()async{

  if(converted != null){
    Directory appDir = await
getApplicationDocumentsDirectory();
    File imgfile = File(converted!);
    String filename =
paths.basename(imgfile.path);
    String path = '${appDir.path}/${filename}';
    await imgfile.copy(path);
    copied = path;
    print(path);
  }else{
    copied = "";
  }
}

topermanentstorageh()async{

  if(converted2 != null){
    Directory appDir = await
getApplicationDocumentsDirectory();
    File imgfile = File(converted2!);
    String filename =
paths.basename(imgfile.path);
    String path = '${appDir.path}/${filename}';
    await imgfile.copy(path);
    copied2 = path;
    print(path);
  }else{
    copied2 = "";
  }
}

dateconvert(String inptdate){

  String dateStr = inptdate;
  DateTime date =
DateFormat('MM/dd/yyyy').parse(dateStr);
  String formattedDate = DateFormat('yyyy-MM-

```

```

dd').format(date);

  return formattedDate;
}

fluttermtoast(String message){
  Fluttertoast.showToast(msg: message,
    toastLength: Toast.LENGTH_SHORT,
    gravity: ToastGravity.CENTER,
    timeInSecForIosWeb: 2,
    backgroundColor: Colors.grey,
    textColor: Colors.white,
    fontSize: 18
  );
}

//-----exif data
exifLatitudeLongitudePoint(File img) async{
  final data = await readExifFromBytes(await
img.readAsBytes());
  if (data.containsKey('GPS GPSLongitude')) {
    final gpsLatitude = data['GPS GPSLatitude'];
    final latitudeSignal = data['GPS
GPSLatitudeRef']!.printable;
    List latitudeRation = gpsLatitude!.values.toList();
    List latitudeValue = latitudeRation.map((item) {
      return (item.numerator.toDouble() /
item.denominator.toDouble());
    }).toList();
    double latitude = latitudeValue[0] +
(latitudeValue[1] / 60) + (latitudeValue[2] / 3600);
    if (latitudeSignal == 'S') latitude = -latitude;
    _contact_lat.text = latitude.toString();

    final gpsLongitude = data['GPS GPSLongitude'];
    final longitudeSignal = data['GPS
GPSLongitude']!.printable;
    List longitudeRation =
gpsLongitude!.values.toList();
    List longitudeValue =
longitudeRation.map((item) {
      return (item.numerator.toDouble() /
item.denominator.toDouble());
    }).toList();
    double longitude =longitudeValue[0] +
(longitudeValue[1] / 60) + (longitudeValue[2] /
3600);
    if (longitudeSignal == 'W') longitude = -
longitude;

```

```

        _contact_long.text = longitude.toString();
    }
}

//get lat long
Future<Position> _getGeoLocationPosition()
async {
    bool serviceEnabled;
    LocationPermission permission;

    serviceEnabled = await
Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) {
        await Geolocator.openLocationSettings();
        return Future.error('Location services are
disabled.');
```

```

    }
    permission = await
Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
        permission = await
Geolocator.requestPermission();
        if (permission == LocationPermission.denied) {
            return Future.error('Location permissions are
denied');
```

```

        }
    }
    if (permission ==
LocationPermission.deniedForever) {
        return Future.error(
            'Location permissions are permanently
denied, we cannot request permissions.');
```

```

    }
    return await
Geolocator.getCurrentPosition(desiredAccuracy:
LocationAccuracy.high);
}

reload_table()async{
    setState(() {
        count = temp_list_memb.length;
        table_temp();
    });
}

checkmaxmem(){
    if(temp_list_memb.length >=
int.parse(selectednumbermem)){
```

```

        setState((){
            max = false;
        });
    }else{
        setState((){
            max = true;
        });
    }
}

@override
void initState() {
    // TODO: implement initState

    _contact_city.text = 'Digos City';
    _contact_province.text = 'Davao del Sur';
    super.initState();
}

@override
void dispose(){
    _scrollController.dispose();
    super.dispose();
}

void _scrollToTop() {
    _scrollController.animateTo(0,
        duration: const Duration(milliseconds: 150),
        curve: Curves.linear);
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        key: _scaffoldkey,
        resizeToAvoidBottomInset: true,
        appBar: top_header(context),
        drawer: const Navigationmenu(),
        body: WillPopScope(
            onWillPop: ()async{
                final result = await showDialog <bool> (
                    context: context,
                    builder: (context){
                        return AlertDialog(
                            title: const Text('Exiting the form?'),
                            content: const Text('You are about to
EXIT the from, work will not be save'),
                            actions:[
                                TextButton(onPressed: () =>
Navigator.of(context).pop(false), child: const
```

```

Text('Cancel')),
      TextButton(onPressed: (){
        if(!globals.online_access){
          Navigator.of(context).pop(true);
          Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) => const
Homepage()));
        }else{
          Navigator.of(context).pop(true);
          Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) => const
Homepage_online()));
        }
      }, child: const Text('Close'))
    ],
  );

  }
);
return Future.value(result);
},
child: Stack(
  children: [
    Column(
      children: [
        Flexible(
          flex: 1,
          child: Container(
            margin: const EdgeInsets.symmetric(
vertical: 10),
            child: SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: Row(
                children: [
                  const SizedBox(width: 10,),
                  //-----qualification
                  ElevatedButton(onPressed: (){
                    DBhelper_qualification path =
DBhelper_qualification();
                    getDatabasesPath().then((value)
=> print(value));
                    _pagecontroller.jumpToPage(0);
                  },
                    style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
                      shape:
RoundedRectangleBorder(borderRadius:

```

```

BorderRadius.circular(10))),
                      child: const Text('Qualification'),
                    ),
                    const SizedBox(width: 10,),
                    //-----profile
                    ElevatedButton(onPressed:
enable_profile? (){
                      _pagecontroller.jumpToPage(1);
                    }:null,
                      style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
                        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
                        disabledForegroundColor: const
Color.fromRGBO(75, 79, 177, 1).withOpacity(0.38),
                        disabledBackgroundColor: const
Color.fromRGBO(75, 79, 177, 1).withOpacity(0.12)),
                        child: const Text('Profile'),
                      ),
                    const SizedBox(width: 10,),
                    //-----contact
                    ElevatedButton(onPressed:
enable_cont? (){
                      _pagecontroller.jumpToPage(2);
                    }:null,
                      style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
                        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
                        disabledForegroundColor: const
Color.fromRGBO(75, 79, 177, 1).withOpacity(0.38),
                        disabledBackgroundColor: const
Color.fromRGBO(75, 79, 177, 1).withOpacity(0.12)),
                        child: const Text('Address'),
                      ),
                    const SizedBox(width: 10,),
                    //-----children
                    Visibility(
                      visible: true,
                      child: ElevatedButton(onPressed:
enable_child? (){
                        _pagecontroller.jumpToPage(3);
                      }:null,
                        style:
ElevatedButton.styleFrom(backgroundColor: const

```

```

Color.fromRGBO(75, 79, 177, 1),
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
disabledForegroundColor: const
Color.fromRGBO(75, 79, 177, 1).withOpacity(0.38),
disabledBackgroundColor: const
Color.fromRGBO(75, 79, 177, 1).withOpacity(0.12)),
    child: const Text('Members'),
  ),
),
],
),
),
),
),
),

// ----- for forms -----
-----
Flexible(
  flex: 10,
  child: Container(
    margin: const EdgeInsets.only(right:
10, left: 10, top: 5, bottom: 5),
    child: SizedBox(
      width: double.infinity,
      child: PageView(
        physics: const
NeverScrollableScrollPhysics(),
        controller: _pagecontroller,
        onPageChanged: (index){
          setState(() {
            page = index;
          });
        },
        children: [

          //0. Qualification
          SingleChildScrollView(
            controller: _scrollController,
            child: Form(
              child: Column(
                crossAxisAlignment:
CrossAxisAlignment.start,
                children: <Widget>[
                  const Text('Primary Questions',
style: TextStyle(fontSize: 30, fontWeight:
FontWeight.w500)),
                  const SizedBox(height: 10,),

```

```

Container(
  padding: const
EdgeInsets.all(10),
  decoration:
BoxDecoration(color: Colors.white, borderRadius:
BorderRadius.circular(10), border: Border.all(color:
Colors.black12)),
  child: Column(
    mainAxisAlignment:
MainAxisAlignment.start,
    crossAxisAlignment:
CrossAxisAlignment.start,
    children: [
      txtbox('Household Head
Applicant', " *"),
      DecoratedBox(
        decoration:
BoxDecoration(
          border: Border.all(color:
Colors.grey, width: 1),
          borderRadius:
BorderRadius.circular(5)
        ),
        child: Padding(
          padding: const
EdgeInsets.symmetric(horizontal: 10),
          child: DropdownButton(
            isExpanded: true,
            underline: Container(),
            value: dropvalue,
            alignment:
Alignment.center,
            items:
items.map((String items){
              return
DropdownMenuItem(
                value: items,
                child: Text(items,
textAlign: TextAlign.center,),
              ).toList(), onChanged:
(String? newval){
                setState(() {
                  dropvalue = newval!;
                });
              },),
            ),

```

```

const SizedBox(height: 20,),
txtbox('Combine family
monthly income', " *"),
DecoratedBox(
  decoration:
BoxDecoration(
  border: Border.all(color:
Colors.grey, width: 1),
  borderRadius:
BorderRadius.circular(5)
),
  child: Padding(
    padding: const
EdgeInsets.symmetric(horizontal: 10),
    child: DropdownButton(
      isExpanded: true,
      underline: Container(),
      value: incomeselect,
      alignment:
Alignment.center,
      items:
incomes.map((String incomes){
  return
DropdownMenuItem(
    value: incomes,
    child: Text(incomes,
textAlign: TextAlign.center,),
  );
}).toList(), onChanged:
(String? newval){
  setState() {
    incomeselect =
newval!;
  });
},),
),
const SizedBox(height: 20,),
txtbox('Number of family
members (including applicant)', " *"),
DecoratedBox(
  decoration:
BoxDecoration(
  border: Border.all(color:
Colors.grey, width: 1),
  borderRadius:
BorderRadius.circular(5)
),
  child: Padding(
    padding: const
EdgeInsets.symmetric(horizontal: 10),
    child: DropdownButton(
      isExpanded: true,
      underline: Container(),
      value: incomeselect,
      alignment:
Alignment.center,
      items:
incomes.map((String incomes){
  return
DropdownMenuItem(
    value: incomes,
    child: Text(incomes,
textAlign: TextAlign.center,),
  );
}).toList(), onChanged:
(String? newval){
  setState() {
    incomeselect =
newval!;
  });
},),
),
const SizedBox(height: 20,),
txtbox('Is the applicant
currently pregnant?', " *"),
Row(
  children: [
    Expanded(
      child: ListTile(
        title: const Text('Yes'),
        leading: Radio(
          value:
HavePregnant.yes,
          groupValue:
_pregchoice,
          onChanged:
(HavePregnant? value){
            setState() {
              _pregchoice =

```

```

value!;
        setState(() {
          yes_preg_txt =
true;
          ispreg = 'yes';
          preg = 1;
        });
      },
    ),
  ),
Expanded(
  child: ListTile(
    title: const Text('No'),
    leading: Radio(
      value:
HavePregnant.no,
      groupValue:
_pregchoice,
      onChanged:
(HavePregnant? value){
value!;
        setState(() {
          yes_preg_txt =
false;
          ispreg = 'no';
          preg = 0;
        });
      },
    ),
  ),
),
),
),
const SizedBox(height: 20,),
txtbox('If yes, please enter
the pregnancy starting period', " *"),
SizedBox( width:
double.infinity, child: TextField(
  enabled: yes_preg_txt,
  controller:
_qual_preg_date,
  readOnly: true ,
  decoration: const

```

```

InputDecoration(
  hintText: 'Type
Something',
  border:
OutlineInputBorder(),
  contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal: 10)),
  onTap: () async{
    showpregdate(context,
_qual_preg_date);
  },
)),
const SizedBox(height: 20,),
txtbox('Does applicant have
children aging 0-18', " *"),
Row(
  children: [
    Expanded(
      child: ListTile(
        title: const Text('Yes'),
        leading: Radio(
          value: underage.yes,
          groupValue:
_agechoice,
          onChanged:
(underage? value){
value!;
        setState(() {
          _agechoice =
        setState(() {
          yes_age = true;
          ischild = 'yes';
        });
      },
    ),
  ),
),
Expanded(
  child: ListTile(
    title: const Text('No'),
    leading: Radio(
      value: underage.no,
      groupValue:
_agechoice,
      onChanged:
(underage? value){
        setState(() {

```

```

        _agechoice =
value!;

        setState(() {
          yes_age = false;
          ischild = 'no';
        });
      },
    ),
  ),
],
),

    txtbox('Year of Residence', '
*),

    SizedBox( width:
double.infinity, child: TextField(keyboardType:
TextInputType.number,
        inputFormatters: [
FilteringTextInputFormatter.allow(RegExp('[0-
9]')),],

        controller: _qual_years,
        decoration: const
InputDecoration(
        hintText: 'Type
Something',

        border:
OutlineInputBorder(),

        contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),),

    const SizedBox(height: 20,),
    txtbox('Type of Residence',
' *'),

    DecoratedBox(
      decoration:
BoxDecoration(
        border: Border.all(color:
Colors.grey, width: 1),

        borderRadius:
BorderRadius.circular(5)
      ),
      child: Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 10),

        child: DropdownButton(
          isExpanded: true,
          underline: Container(),
          value: selecthousedesc,
          alignment:
Alignment.center,

          items:
housedesc.map((String housedesc){
            return
DropdownMenuItem(
              value: housedesc,
              child: Text(housedesc,
                textAlign: TextAlign.center,

```

```

          underline: Container(),
          value: selecttypeses,
          alignment:
Alignment.center,

          items:
typeses.map((String typeses){
            return
DropdownMenuItem(
              value: typeses,
              child:
Text(typeses, textAlign: TextAlign.center,),
            );
          }).toList(), onChanged:
(String? newval){
            setState(() {
              selecttypeses =
newval;

            });
          },),
        ),

const SizedBox(height: 20,),
txtbox('House Description',
' *'),

    DecoratedBox(
      decoration:
BoxDecoration(
        border: Border.all(color:
Colors.grey, width: 1),

        borderRadius:
BorderRadius.circular(5)
      ),
      child: Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 10),

        child: DropdownButton(
          isExpanded: true,
          underline: Container(),
          value: selecthousedesc,
          alignment:
Alignment.center,

          items:
housedesc.map((String housedesc){
            return
DropdownMenuItem(
              value: housedesc,
              child: Text(housedesc,
                textAlign: TextAlign.center,

```

```

        );
      }).toList(), onChanged:
    (String? newval){
      setState() {
        selecthousedesc =
      newval!;
    });
  },
),
),
const SizedBox(height: 20,),
Container(alignment:
AlignmentDirectional.centerEnd,
  child:
ElevatedButton(onPressed: (){
  if(dropvalue == 'Select'
  || incomeselect == 'Select' ||
  selectednumbermem == 'Select' || _qual_years ==
  " || selecttyperes == 'Select' || selecthousedesc ==
  'Select'){
    fluttoast('Please fill
up all required fields');
  }else
  if(_qual_years.text.length < 4){
    fluttoast('Invalid
Year of Residence');
  } else{
    checkmaxmem();
  }
  _pagecontroller.jumpToPage(1);
  setState() {
    enable_profile = true;
    _scrollToTop();
  });
  // scrollup();
}, style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
  shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Next'),),
),
),

```

```

    ],
  ),
),
//1. Profile
SingleChildScrollView(
  controller: _scrollController,
  child: Form(
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: <Widget>[
        const Text('Household Head
Profile', style: TextStyle(fontSize: 30, fontWeight:
FontWeight.w500)),
        const SizedBox(height: 10,),
        Text('Note: Required Fields are
indicated with *', style: TextStyle(fontSize: 18,
fontWeight: FontWeight.w500, color:
Colors.grey[400])),
        const SizedBox(height: 20,),
        imagepicker(),
        const SizedBox(height: 10,),
        Center(child:
txtbox('Household Head Profile Image', '')),
        const SizedBox(height: 20,),
        Container(
          padding: const
EdgeInsets.all(10),
          decoration:
BoxDecoration(color: Colors.white, borderRadius:
BorderRadius.circular(10), border: Border.all(color:
Colors.black12)),
          child: Column(
            crossAxisAlignment:
CrossAxisAlignment.start,
            children: [
              txtbox('Last Name', ' *'),
              widget_textfield(_profile_lname),
              const SizedBox(height: 20,),
              txtbox('First Name', ' *'),
              widget_textfield(_profile_fname),
              const SizedBox(height: 20,),

```

```

        txtbox('Middle Name', ''),

widget_textfield(_profile_mname),
        const SizedBox(height: 20,),

        txtbox('Suffix', ''),
        DecoratedBox(
            decoration:
BoxDecoration(
                border: Border.all(color:
Colors.grey, width: 1),
                borderRadius:
BorderRadius.circular(5)
            ),
            child: Padding(
                padding: const
EdgeInsets.symmetric(horizontal: 10),
                child: DropdownButton(
                    isExpanded: true,
                    underline: Container(),
                    value: selectedsuffix,
                    alignment:
Alignment.center,
                    items: suffix.map((String
suffix){
                        return
DropdownMenuItem(
                            value: suffix,
                            child: Text(suffix,
textAlign: TextAlign.center,),
                        );
                    }).toList(), onChanged:
(String? newval){
                        setState() {
                            selectedsuffix =
newval!;
                        }
                    },
                ),
            ),
            const SizedBox(height: 20,),

            txtbox('Gender', ' *'),
            DecoratedBox(
                decoration:
BoxDecoration(
                    border: Border.all(color:
Colors.grey, width: 1),
                    borderRadius:

```

```

BorderRadius.circular(5)
                ),
                child: Padding(
                    padding: const
EdgeInsets.symmetric(horizontal: 10),
                child: DropdownButton(
                    isExpanded: true,
                    underline: Container(),
                    value: genderdrop,
                    alignment:
Alignment.center,
                    items:
gender.map((String gender){
                        return
DropdownMenuItem(
                            value: gender,
                            child: Text(gender,
textAlign: TextAlign.center,),
                        );
                    }).toList(), onChanged:
(String? newval){
                        setState() {
                            genderdrop = newval!;
                        }
                    },
                ),
            ),
            const SizedBox(height: 20,),

            txtbox('Civil Status', ' *'),
            DecoratedBox(
                decoration:
BoxDecoration(
                    border: Border.all(color:
Colors.grey, width: 1),
                    borderRadius:
BorderRadius.circular(5)
                ),
                child: Padding(
                    padding: const
EdgeInsets.symmetric(horizontal: 10),
                child: DropdownButton(
                    isExpanded: true,
                    underline: Container(),
                    value: civildrop,
                    alignment:
Alignment.center,
                    items:
civilstat.map((String civilstat){

```

```

return
DropdownMenuItem(
  value: civilstat,
  child: Text(civilstat,
textAlign: TextAlign.center,),
  );
}).toList(), onChanged:
(String? newval){
  setState() {
    civildrop = newval!;
  });
  },,
),
const SizedBox(height: 20,),

  txtbox('Birthday', ' *'),
  SizedBox( width:
double.infinity, child: TextField(
    controller: _profile_bday,
    readOnly: true ,
    decoration: const
InputDecoration(
  hintText: 'Type
Something',
  border:
OutlineInputBorder(),
  contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal: 10)),
  onTap: () async{
    showPickerDate(context,
_profile_bday, 1);
  },
  )),
const SizedBox(height: 20,),

  txtbox('Educational
Attainment', ' *'),
  DecoratedBox(
    decoration:
BoxDecoration(
  border: Border.all(color:
Colors.grey, width: 1),
  borderRadius:
BorderRadius.circular(5)
),
    child: Padding(
      padding: const
EdgeInsets.symmetric(horizontal: 10),

```

```

child: DropdownButton(
  isExpanded: true,
  underline: Container(),
  value: selectedschool,
  alignment:
Alignment.center,
  items:
school.map((String school){
    return
DropdownMenuItem(
  value: school,
  child: Text(school,
textAlign: TextAlign.center,),
  );
}).toList(), onChanged:
(String? newval){
  setState() {
    selectedschool =
newval!;
  });
  },,
),
//
widget_textfield(_profile_educational_attainment)
,
const SizedBox(height: 20,),

  txtbox('Occupation', ' *'),
  widget_textfield(_profile_occupation),
  const SizedBox(height: 20,),

  txtbox('Contact Number', '
'),
  SizedBox( width:
double.infinity, child: TextField(keyboardType:
TextInputType.number,
  inputFormatters: [
FilteringTextInputFormatter.allow(RegExp('[0-
9+]'))],,
  controller:
_profile_contnumber,
  maxLength: 10,
  onTap: (){
    if(_profile_contnumber.text.isEmpty){
      _profile_contnumber.text = "9";

```

```

    }
  },
  decoration: const
InputDecoration(
    prefixText: "+63",
    hintText: 'Type
Something',
    border:
OutlineInputBorder(),
    contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),),
    const SizedBox(height: 20,),
    ElevatedButton(onPressed:
(){
print(_profile_contnumber.text);
    }, child: const Text('press')),
    txtbox('Email Address', ''),
widget_textfield(_profile_email),
    const SizedBox(height: 20,),
    ],
  ),
),
),
    Row(
      mainAxisAlignment:
MainAxisAlignment.end,
      children: [
        ElevatedButton(onPressed:
(){
_pagecontroller.jumpToPage(0);
        _scrollToTop();
      }, style:
ElevatedButton.styleFrom(backgroundColor:
Colors.grey[500],
      shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Previous')),),
    const SizedBox(width: 10,),
    ElevatedButton(onPressed:
() async{
      if( _profile_lname == '' ||
_profile_fname == '' || genderdrop == 'Select' ||

```

```

civildrop == 'Select' || _profile_bday == '' ||
selectedschool == 'Selected' ||
_profile_occupation == ''){
      fluttermtoast('Please fill up
all the required fields');
    }else if
(_profile_contnumber.text.length < 10 &&
_profile_contnumber.text.isNotEmpty) {
      fluttermtoast('Invalid Mobile
Number');
    } else{
      String age = '';
      String middlename = '';
      if
(_profile_mname.text.isNotEmpty){
        middlename =
_profile_mname.text[0];
      }
      obj_memb =
temp_memb_profile("", temp_list_memb.length,
'Leader', "${_profile_fname.text} $middlename.
${_profile_lname.text}", _profile_bday.text, age,
genderdrop, civildrop, _profile_occupation.text,
_profile_contnumber.text, _profile_email.text);
      if
(temp_list_memb.isEmpty){
temp_list_memb.insert(0,
obj_memb.member_map());
      }else{
        temp_list_memb[0] =
obj_memb.member_map();
      }
      reload_table();
      print(temp_list_memb);
_pagecontroller.jumpToPage(2);
      _scrollToTop();
      setState(() {
        // print(_imgfile?.path);
        enable_cont = true;
      });
      // Position pos = await
_getGeoLocationPosition();

```

```

        // setState() {
        //   _contact_lat.text =
pos.latitude.toString();
        //   _contact_long.text =
pos.longitude.toString();
        // });

    }, style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Next'),)
    ],
  ),
  ),
  ),
  //2. Address
SingleChildScrollView(
  controller: _scrollController,
  child: Form(
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: <Widget>[
        const Text('Address', style:
TextStyle(fontSize: 30, fontWeight:
FontWeight.w500)),
        const SizedBox(height: 10,),
        Container(
          padding: const
EdgeInsets.all(10),
          decoration:
BoxDecoration(color: Colors.white, borderRadius:
BorderRadius.circular(10), border: Border.all(color:
Colors.black12)),
          child: Column(
            crossAxisAlignment:
CrossAxisAlignment.start,
            children: [

```

```

        txtbox('House
Number/Building Name/Block', ''),
        widget_textfield(_contact_housenum),
        const SizedBox(height: 20,),
        txtbox('Street Name', ' '),
        widget_textfield(_contact_street),
        const SizedBox(height: 20,),
        txtbox('Barangay', ' '),
        DecoratedBox(
          decoration:
BoxDecoration(
            border: Border.all(color:
Colors.grey, width: 1),
            borderRadius:
BorderRadius.circular(5)
          ),
          child: Padding(
            padding: const
EdgeInsets.symmetric(horizontal: 10),
            child: DropdownButton(
              isExpanded: true,
              underline: Container(),
              value: selectedbrgy,
              alignment:
Alignment.center,
              items:
globals.barangay.map((String barangay){
                return
DropdownMenuItem(
                  value: barangay,
                  child: Text(barangay,
                    textAlign: TextAlign.center,
                  );
                }).toList(), onChanged:
(String? newval){
                  setState() {
                    selectedbrgy =
                    newval;
                  });
                },),
              ),
            ),
            const SizedBox(height: 20,),

```

<pre> txtbox('Purok', ' *'), widget_textfield(_contact_purok), const SizedBox(height: 20,), txtbox('City', ' *'), widget_textfield3(_contact_city, true), const SizedBox(height: 20,), txtbox('Province', ' *'), widget_textfield3(_contact_province, true), const SizedBox(height: 20,), txtbox('Zip Code', ' *'), widget_textfield(_contact_zipcode), const SizedBox(height: 20,), Row(mainAxisAlignment: MainAxisAlignment.center, crossAxisAlignment: CrossAxisAlignment.center, children: [Expanded(flex: 2, child: SizedBox(width: double.infinity, child: Column(children: [txtbox('Latitude', ''), TextField(controller: _contact_lat, readOnly: true, decoration: const InputDecoration(hintText: 'Type Something', border: OutlineInputBorder(), contentPadding: EdgeInsets.symmetric(vertical: 10, horizontal: 10))),),],)),), const SizedBox(width: 10,), Expanded(flex: 1, child: Container(margin: const EdgeInsets.only(top: 20), child: ElevatedButton(onPressed: () async{ showDialog <String>(context: context, builder: (BuildContext context) => AlertDialog(icon: const Icon(CupertinoIcons.location, color: Color.fromRGBO(75, 79, 177, 1),size: 50,), title: const Text("Location"), content: const Text("Do you want to set the beneficiary location here?"), actions: [TextButton(onPressed: () => </pre>	<pre> 10,), Expanded(flex: 2, child: SizedBox(width: double.infinity, child: Column(children: [txtbox('Longitude', TextField(controller: _contact_long, readOnly: true, decoration: const hintText: 'Type border: contentPadding: EdgeInsets.symmetric(vertical: 10, horizontal: 10))),),],)),), const SizedBox(width: 10,), Expanded(flex: 1, child: Container(margin: const EdgeInsets.only(top: 20), child: ElevatedButton(onPressed: () async{ showDialog <String>(context: context, builder: (BuildContext context) => AlertDialog(icon: const Icon(CupertinoIcons.location, color: Color.fromRGBO(75, 79, 177, 1),size: 50,), title: const Text("Location"), content: const Text("Do you want to set the beneficiary location here?"), actions: [TextButton(onPressed: () => </pre>
--	--

```

Navigator.pop(context), child: const Text("No")),

IconButton(onPressed: () async{
  Position pos =
  await _getGeoLocationPosition();
  setState(() {

    _contact_lat.text = pos.latitude.toString();

    _contact_long.text = pos.longitude.toString();
  });

  Navigator.pop(context);
}, child: const
Text("Yes")),

),
));
), style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Icon(CupertinoIcons.location_solid, color:
Colors.white,,),
),
),
),
Row(
  mainAxisAlignment:
MainAxisAlignment.start,
  children: [
    const SizedBox(width:
5,),
    Transform.scale(
      scale: 1,
      child: SizedBox(
        height: 46,
        width: 15,
        child: Checkbox(value:
getlatlongimg, onChanged: (bool? value) {
          setState(() {
            getlatlongimg =
value!;
          });
        }, ),
      ),
    ),
  ],
),

```

```

),
const Padding(
  padding:
EdgeInsets.only(left: 8.0),
  child: Text('Get Location
by Image', style: TextStyle(color: Colors.black,,),
),
// SizedBox(width: 60,,),
],
),
const SizedBox(height: 20,,),

ha_imagepicker(),
const SizedBox(height: 20,,),

Row(
  mainAxisAlignment:
MainAxisAlignment.end,
  children: [

    ElevatedButton(onPressed: (){
      _pagecontroller.jumpToPage(1);
      _scrollToTop();
    }, style:
ElevatedButton.styleFrom(backgroundColor:
Colors.grey[500],
shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Previous'),),
const SizedBox(width:
10,,),

    ElevatedButton(onPressed: (){
      if(_contact_street.text
== " || selectedbrgy == 'Select' ||
_contact_purok.text == " || _contact_city.text == "
|| _contact_province.text == " ||
_contact_zipcode.text == "){
        fluttermtoast('Please fill
up all the required fields or input correct value');
      }else{
        _pagecontroller.jumpToPage(3);
        _scrollToTop();
        setState(() {
          enable_child = true;

```

```

    });
  }
}, style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Next'),)
    ],
  ),
    ],),),
    ],
  ),
  ),
),
//3. Members
SingleChildScrollView(
  controller: _scrollController,
  child: Form(
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: <Widget>[
        const Text('Add Member
Profile', style: TextStyle(fontSize: 30, fontWeight:
FontWeight.w500)),
        const SizedBox(height: 10,),
        Text('Note: Required Fields are
indicated with *', style: TextStyle(fontSize: 18,
fontWeight: FontWeight.w500, color:
Colors.grey[400])),),
        const SizedBox(height: 20,),
        Container(
          padding: const
EdgeInsets.all(10),
          decoration:
BoxDecoration(color: Colors.white, borderRadius:
BorderRadius.circular(10), border: Border.all(color:
Colors.black12)),
          child: Column(
            crossAxisAlignment:
CrossAxisAlignment.start,
            children: [

```

```

Container(
  decoration:
BoxDecoration(color: Colors.white, borderRadius:
BorderRadius.circular(10), border: Border.all(color:
const Color.fromRGBO(75, 79, 177, 1))),
    constraints: const
BoxConstraints(
      minWidth: 20,
      maxWidth: 60
    ),
    child: Row(
      children: [
        const
Icon(Icons.person_outline, color:
Color.fromRGBO(75, 79, 177, 1)),
        Text(count.toString()),
        const Text("/"),
        Text(selectednumbermem),
      ],
    ),),
    const SizedBox(height: 10,),
    txtbox('Type of Member', "
*"),
    DecoratedBox(
      decoration:
BoxDecoration(
        border: Border.all(color:
Colors.grey, width: 1),
        borderRadius:
BorderRadius.circular(5)
      ),
      child: Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 10),
        child: DropdownButton(
          isExpanded: true,
          underline: Container(),
          value: hold_member,
          alignment:
Alignment.center,
          items:
items.map((String items){
            return
DropdownMenuItem(
              value: items,
              child: Text(items,
textAlign: TextAlign.center),

```

```

        );
    }).toList(), onChanged:
(String? newval){
        setState() {
            hold_member =
newval!;
        });
    },
),
const SizedBox(height: 20,),

    txtbox('Full Name', ' *'),

    widget_textfield(_hold_fullname),
    const SizedBox(height: 20,),

    txtbox('Gender', ' *'),
    DecoratedBox(
        decoration:
BoxDecoration(
            border: Border.all(color:
Colors.grey, width: 1),
            borderRadius:
BorderRadius.circular(5)
        ),
        child: Padding(
            padding: const
EdgeInsets.symmetric(horizontal: 10),
            child: DropdownButton(
                isExpanded: true,
                underline: Container(),
                value: hold_cs,
                alignment:
Alignment.center,
                items:
civilstat.map((String civilstat){
                    return
DropdownMenuItem(
                        value: civilstat,
                        child: Text(civilstat,
                            textAlign: TextAlign.center,)),
                );
            }).toList(), onChanged:
(String? newval){
                setState() {
                    hold_cs = newval!;
                });
            },),
        ),
const SizedBox(height: 20,),

    txtbox('Birthday', ' *'),
    SizedBox( width:
double.infinity, child: TextField(
        controller: _hold_bday,
        readOnly: true ,
        decoration: const
InputDecoration(
            hintText: 'Type

```

```

Something',
border:
OutlineInputBorder(),
contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal: 10)),
onTap: () async{
showPickerDate(context,
_hold_bday, 1);
},
)),
const SizedBox(height: 20,),

txtbox('Occupation', ' '),

widget_textfield(_hold_occupation),
const SizedBox(height: 20,),

txtbox('Contact Number', ' '),

SizedBox( width:
double.infinity, child: TextField(keyboardType:
TextInputType.number,
inputFormatters: [
FilteringTextInputFormatter.allow(RegExp('[0-9+]')),],
controller: _hold_contact,
maxLength: 10,
onTap: (){

if(_hold_contact.text.isEmpty){
_hold_contact.text =
"9";
}
},
decoration: const
InputDecoration(
prefixText: "+63",
hintText: 'Type
Something',
border:
OutlineInputBorder(),
contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),),
const SizedBox(height: 20,),

txtbox('Email Address', ''),

widget_textfield(_hold_email),

const SizedBox(height: 20,),

Center(
child:
ElevatedButton(onPressed: max? () async{
if(hold_member ==
'Select' || _hold_fullname.text == " ||
hold_gender == 'Select' || hold_cs == 'Select' ||
_hold_bday.text == "){
fluttertoast('Please fill
up all the required fields');
}else{
String age = "";

obj_memb =
temp_memb_profile("",temp_list_memb.length,
hold_member, _hold_fullname.text,
_hold_bday.text, age, hold_gender, hold_cs,
_hold_occupation.text, _hold_contact.text,
_hold_email.text);

temp_list_memb.add(obj_memb.member_map());

print(temp_list_memb);

reload_table();
checkmaxmem();
clear_memb();
_scrollToTop();
}

}):null, style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const Text('Add
Household Member'),),
),
),
const SizedBox(height: 20,),
Container(
padding: const
EdgeInsets.all(10),
decoration:
BoxDecoration(color: Colors.white, borderRadius:

```

```

BorderRadius.circular(10), border: Border.all(color:
Colors.black12)),
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          SingleChildScrollView(
            scrollDirection:
Axis.horizontal,
            child: table_temp())
        ],
      ),
    ),
    Row(
      mainAxisAlignment:
MainAxisAlignment.end,
      children: [
        ElevatedButton(onPressed:
() {
          _pagecontroller.jumpToPage(0);
          _scrollToTop();
        }, style:
ElevatedButton.styleFrom(backgroundColor:
Colors.grey[500],
          shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Previous')),
        const SizedBox(width: 10,),
        ElevatedButton(onPressed:
() async {
          if( selectedschool ==
'Select' || _profile_bday.text == '' ||
_profile_lname.text == '' || _profile_fname.text ==
''){
            fluttermtoast('Please fill up
all the required fields');
          }else{
            await
topermanentstorage();
            await
topermanentstorageh();

            setState(() {
              isloading = true;
            });
            await createbeneficiary();

```

```

        setState(() {
          isloading = false;
        });

        _pagecontroller.jumpToPage(4);
        _scrollToTop();
      }
    }, style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
          shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Submit')),
      ],
    ),
    ],
  ),
  //4. Done
  SingleChildScrollView(
    controller: _scrollController,
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.center,
      mainAxisAlignment:
MainAxisAlignment.center,
      children: <Widget>[
        const Center(child:
Text('Beneficiary Successfully Registered', style:
TextStyle(fontSize: 30, fontWeight:
FontWeight.w500), textAlign: TextAlign.center,)),
        const SizedBox(height: 25,),
        const Center(child:
Text('Beneficiary Serial Code', style:
TextStyle(fontSize: 18, fontWeight:
FontWeight.w500, color: Colors.black))),
        const SizedBox(height: 10,),
        Center(child: Text(ref_code,
style: const TextStyle(fontSize: 30, fontWeight:
FontWeight.w500, color: Colors.blueAccent))),
        const SizedBox(height: 10,),
        ElevatedButton(onPressed: ()
async {

```

```

Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
add_applicant_online()));

        }, style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Done', style: TextStyle(fontSize: 20)),)

    ],
    ),
    ),
    ],
    ),
    ),),
    ),
    ],
    ),
    Visibility(visible: isloading,child:
Container(padding: const
EdgeInsets.all(20),decoration: const
BoxDecoration(shape: BoxShape.rectangle, color:
Colors.black87),child: Column(
        mainAxisAlignment:
MainAxisAlignment.center,
        crossAxisAlignment:
CrossAxisAlignment.center,
        children: [
            Center(child: SizedBox(height: 70, width:
70,child: isloading?const CircularProgressIndicator(
                backgroundColor: Color.fromRGBO(237,
237, 238, 1.0), strokeWidth: 10, valueColor:
AlwaysStoppedAnimation<Color>(
                    Color.fromRGBO(75, 79, 177, 1)
                ),
                ):const Text("Loading")),),
            const SizedBox(height: 20,),
            text('Submitting Data'),
        ],
    )),
    ],
    ),
    )
);
}

```

```

Future<void> alertmenu_delete(String name, int
index){
    if(index == 0){
        return showDialog(context: context, builder:
(BuildContext context)
        {
            return AlertDialog(
                title: const Text('Delete Member'),
                content: const Text('Household LEADER
cannot be deleted'),
                actions: [
                    TextButton(onPressed: () {
                        Navigator.of(context).pop();
                    }, child: const Text('Close'))
                ],
            );
        });
    }else{
        return showDialog(context: context, builder:
(BuildContext context)
        {
            return AlertDialog(
                title: const Text('Delete Member'),
                content: Text('Do you want to delete $name
?'),
                actions: [
                    TextButton(onPressed: () {

                        temp_list_memb.removeAt(index);
                        reload_table();
                        checkmaxmem();
                        Navigator.of(context).pop();
                    }, child: const Text('Yes')),
                    TextButton(onPressed: () {
                        Navigator.of(context).pop();
                    }, child: const Text('Close')),
                ],
            );
        });
    }
}

```

```

Future<void> alertmenu_view(Map<String,
dynamic> rowdata){

    return showDialog(
        context: context,
        builder: (BuildContext context) {

```

```

return AlertDialog(
  title: const Text('Household Information'),
  content: ConstrainedBox(
    constraints: BoxConstraints(maxHeight:
MediaQuery.of(context).size.height * 0.7),
    child: SingleChildScrollView(
      child: Column(
        mainAxisAlignment:
MainAxisAlignment.start,
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          text_string('Relation - ',
rowdata['relation']),
          text_string('Fullname - ',
rowdata['fullname']),
          text_string('Birthday - ', rowdata['bday']),
          text_string('Gender - ',
rowdata['gender']),
          text_string('Civil Status - ',
rowdata['civil_stat']),
          text_string('Occupation - ',
rowdata['occupation']),
          text_string('Contact # - ',
rowdata['contact']),
          text_string('Email - ', rowdata['email']),
        ],
      ),
    ),
  ),
  actions: [
    TextButton(
      onPressed: () {
        Navigator.of(context).pop();
      },
      child: const Text('Close'),
    ),
  ],
);
}

```

```

Future<void> alertmenu_edit(Map<String,
dynamic> rowdata){

  if(rowdata['index'] == 0){
    return showDialog(context: context, builder:

```

```

(BuildContext context)
  {
    return AlertDialog(
      title: const Text('Edit Household Member'),
      content: const Text('Household LEADER
cannot be edited'),
      actions: [
        TextButton(onPressed: () {
          Navigator.of(context).pop();
        }, child: const Text('Close'))
      ],
    );
  }
}

String holdMemberEdit = rowdata['relation'];
TextEditingController holdFullnameEdit =
TextEditingController();
String holdGenderEdit = rowdata['gender'];
String holdCsEdit = rowdata['civil_stat'];
TextEditingController holdBdayEdit =
TextEditingController();
TextEditingController holdOccupationEdit =
TextEditingController();
TextEditingController holdContactEdit =
TextEditingController();
TextEditingController holdEmailEdit =
TextEditingController();

holdFullnameEdit.text = rowdata['fullname'];
holdBdayEdit.text = rowdata['bday'];
holdOccupationEdit.text =
rowdata['occupation'];
holdContactEdit.text = rowdata['contact'];
holdEmailEdit.text = rowdata['email'];

return showDialog(
  context: context,
  builder: (BuildContext context) {
    return AlertDialog(
      title: const Text('Edit Household Member'),
      content: ConstrainedBox(
        constraints: BoxConstraints(maxHeight:
MediaQuery.of(context).size.height * 0.7),
        child: SingleChildScrollView(
          child: Column(
            mainAxisAlignment:
MainAxisAlignment.start,
            crossAxisAlignment:

```

```

CrossAxisAlignment.start,
  children: [
    txtbox('Type of Member', ' *'),
    DecoratedBox(
      decoration: BoxDecoration(
        border: Border.all(color: Colors.grey,
width: 1),
        borderRadius:
BorderRadius.circular(5)
      ),
      child: Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 10),
        child: DropdownButton(
          isExpanded: true,
          underline: Container(),
          value: holdMemberEdit,
          alignment: Alignment.center,
          items: items.map((String items){
            return DropdownMenuItem(
              value: items,
              child: Text(items, textAlign:
TextAlign.center,)),
          );
        }).toList(), onChanged: (String?
newval){
          setState(() {
            holdMemberEdit = newval!;
          });
        },),
      ),
      const SizedBox(height: 20,),

      txtbox('Full Name', ' *'),
      widget_textfield(holdFullnameEdit),
      const SizedBox(height: 20,),

      txtbox('Gender', ' *'),
      DecoratedBox(
        decoration: BoxDecoration(
          border: Border.all(color: Colors.grey,
width: 1),
          borderRadius:
BorderRadius.circular(5)
        ),
        child: Padding(
          padding: const
EdgeInsets.symmetric(horizontal: 10),

```

```

child: DropdownButton(
  isExpanded: true,
  underline: Container(),
  value: holdGenderEdit,
  alignment: Alignment.center,
  items: gender.map((String gender){
    return DropdownMenuItem(
      value: gender,
      child: Text(gender, textAlign:
TextAlign.center,)),
  );
}).toList(), onChanged: (String?
newval){
  setState(() {
    holdGenderEdit = newval!;
  });
},),
),
const SizedBox(height: 20,),

txtbox('Civil Status', ' *'),
DecoratedBox(
  decoration: BoxDecoration(
    border: Border.all(color: Colors.grey,
width: 1),
    borderRadius:
BorderRadius.circular(5)
  ),
  child: Padding(
    padding: const
EdgeInsets.symmetric(horizontal: 10),
    child: DropdownButton(
      isExpanded: true,
      underline: Container(),
      value: holdCsEdit,
      alignment: Alignment.center,
      items: civilstat.map((String civilstat){
        return DropdownMenuItem(
          value: civilstat,
          child: Text(civilstat, textAlign:
TextAlign.center,)),
      );
    }).toList(), onChanged: (String?
newval){
      setState(() {
        holdCsEdit = newval!;
      });
    },),

```

```

    ),
  ),
  const SizedBox(height: 20,),

  txtbox('Birthday', ' *'),
  SizedBox( width: double.infinity, child:
TextField(
  controller: holdBdayEdit,
  readOnly: true ,
  decoration: const InputDecoration(
    hintText: 'Type Something',
    border: OutlineInputBorder(),
    contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal: 10)),
  onTap: () async{
    showPickerDate(context,
holdBdayEdit, 1);
  },
)),
  const SizedBox(height: 20,),

  txtbox('Occupation', ' '),
  widget_textfield(holdOccupationEdit),
  const SizedBox(height: 20,),

  txtbox('Contact Number', ' '),
  SizedBox( width: double.infinity, child:
TextField(keyboardType: TextInputType.number,
  inputFormatters: [
FilteringTextInputFormatter.allow(RegExp('[0-9+]'))],
  controller: holdContactEdit,
  maxLength: 13,
  onTap: (){
    if(holdContactEdit.text.isEmpty){
      holdContactEdit.text = "+639";
    }
  },
  decoration: const InputDecoration(
    hintText: 'Type Something',
    border: OutlineInputBorder(),
    contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),),
  const SizedBox(height: 20,),

  txtbox('Email Address', ''),
  widget_textfield(holdEmailEdit),
  const SizedBox(height: 20,),

```

```

    Center(
      child: ElevatedButton(onPressed: ()
async{
      obj_memb = temp_memb_profile("",
rowdata['index'], holdMemberEdit,
holdFullNameEdit.text, holdBdayEdit.text, '1',
holdGenderEdit, holdCsEdit,
holdOccupationEdit.text, holdContactEdit.text,
holdEmailEdit.text);
      temp_list_memb[rowdata['index']] =
obj_memb.member_map();

      reload_table();

      Navigator.of(context).pop();

    }, style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1),
  shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))), child: const
Text('Update Household Member'),),
    ),
  ],
),
),
actions: [
  TextButton(
    onPressed: () {
      Navigator.of(context).pop();
    },
    child: const Text('Close'),
  ),
],
);
}
}

Widget table_temp(){
  return DataTable(columnSpacing: 25,columns: [
    DataColumn(label: label('Info', 20)),
    DataColumn(label: label('Edit', 20)),
    // DataColumn(label: label('index', 20)),
    DataColumn(label: label('Delete', 20)),

```

```

        DataColumn(label: label('Relation', 20)),
        DataColumn(label: label('Fullname', 20)),
        DataColumn(label: label('Birthday', 20))
    ],
    rows: temp_list_memb.map((row) =>
DataRow(cells: [
    DataColumn(ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(255, 202, 38, 1), padding: const
EdgeInsets.all(10)), onPressed: () async{
    alertmenu_view(row);
    }, child: const Icon(CupertinoIcons.person,
size: 20)),),
    DataColumn(ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(69, 156, 253, 1), padding: const
EdgeInsets.all(10)), onPressed: () async{
    alertmenu_edit(row);
    }, child: const Icon(Icons.edit_outlined, size:
20)),),
    DataColumn(ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(255, 99, 99, 1), padding: const
EdgeInsets.all(10)), onPressed: () async{
    alertmenu_delete(row['fullname'],
row['index']));
    }, child: const Icon(CupertinoIcons.delete,
size: 20)),),
    // DataColumn(Text((index).toString())),
    DataColumn(Text(row['relation'])),
    DataColumn(Text(row['fullname'])),
    DataColumn(Text(row['bday'])),
    ])).toList()
    );
}

showPickerDate(BuildContext context,
TextEditingController controller, int useOn) {
    DateTime currentselected;

    if (controller.text.isNotEmpty){
        //yyyy-MM-dd
        //currentselected =
DateTime.parse(controller.text);

        Picker(
            hideHeader: true,
            adapter: DateTimePickerAdapter(value:
DateTime.parse(dateunformatted)),

```

```

        title: const Text("Select Date"),
        selectedTextStyle: const TextStyle(color:
Colors.blue),
        onConfirm: (Picker picker, List value) {

            String selctedbday = DateFormat('yyyy-MM-
dd').format((picker.adapter as
DateTimePickerAdapter).value!.toLocal());
            dateunformatted = selctedbday;
            controller.text =
DateFormat('MM/dd/yyyy').format(DateTime.parse(
selctedbday));

            if(useOn == 2) {
                compute_age(selctedbday);
            }
        }).showDialog(context);
    }else{
        Picker(
            hideHeader: true,
            adapter: DateTimePickerAdapter(),
            title: const Text("Select Date"),
            selectedTextStyle: const TextStyle(color:
Colors.blue),
            onConfirm: (Picker picker, List value) {
                String selctedbday = DateFormat('yyyy-MM-
dd').format((picker.adapter as
DateTimePickerAdapter).value!.toLocal());
                // controller.text = selctedbday;
                dateunformatted = selctedbday;
                controller.text =
DateFormat('MM/dd/yyyy').format(DateTime.parse(
selctedbday));

                if(useOn == 2){
                    compute_age(selctedbday);
                }
            }).showDialog(context);
    }
}

showpregdate(BuildContext context,
TextEditingController controller) {

    Picker(
        hideHeader: true,
        adapter: DateTimePickerAdapter(type:
PickerDateTimeType.kYM),
        title: const Text("Select Date"),

```

```

        selectedTextStyle: const TextStyle(color:
Colors.blue),
        onConfirm: (Picker picker, List value) {
            DateTime selectedDate = (picker.adapter as
DateTimePickerAdapter).value!;
            String formattedDate = DateFormat('MMM
yyyy').format(selectedDate);
            controller.text = formattedDate;
        },
    ).showDialog(context);
}

compute_age(String bday){
    DateTime forageselected;
    int inpyear;
    int currentyear;
    int currentmonth;
    int currentday;
    int age;
    int slectedmonth;
    int slectedday;
    int vaxage = 0;

    forageselected = DateTime.parse(bday);

    inpyear =
int.parse(forageselected.year.toString());
    currentyear = DateTime.now().year;
    currentmonth = DateTime.now().month;
    currentday = DateTime.now().day;
    age = currentyear - inpyear;
    slectedmonth =
int.parse(forageselected.month.toString());
    slectedday =
int.parse(forageselected.day.toString());

    // _text_ch_age.text = age.toString();

    if((int.parse(currentmonth.toString())-
slectedmonth) < 0 ||
(int.parse(currentday.toString())-slectedday < 0) ){
        vaxage = age - 1;
        // _text_ch_age.text = vaxage.toString();
    }else{
        vaxage = age;
        // _text_ch_age.text = vaxage.toString();
    }

    if(vaxage < 2){

```

```

        age_018 = true;
        age_218 = false;
    }else if(vaxage >= 2){
        age_218 = true;
        age_018 = false;
    }else{
        age_218 = false;
        age_018 = false;
    }
}

clear_memb(){
    hold_member = 'Select';
    _hold_fullname.clear();
    hold_gender = 'Select';
    hold_cs = 'Select';
    _hold_bday.clear();
    _hold_occupation.clear();
    _hold_contact.clear();
    _hold_email.clear();
}

Widget snackbar(){
    return const SnackBar(content:
Text('Snackbarni', style: TextStyle(color:
Colors.white)),,
        backgroundColor: Colors.green,
    );
}

Widget snackbarerror(String text){
    return SnackBar(content: Text(text, style: const
TextStyle(color: Colors.white)),,
        backgroundColor: Colors.green,
    );
}

Widget text(String data){
    return Text(data, style: const TextStyle(fontSize:
25, fontWeight: FontWeight.w600,
        color: Color.fromRGBO(255, 255, 255,
1.0)),textAlign: TextAlign.center,);
}

Widget widget_textfield2(TextEditingController
conroller, bool enableds, int length){
    return SizedBox( width: double.infinity, child:
TextField(

```

```

        controller: conroller,
        readOnly: enableds,
        maxLength: length,
        decoration: const InputDecoration(
          hintText: 'Type Something',
          border: OutlineInputBorder(),
          contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),));
    }

```

```

Widget
widget_textfield_onchange(TextEditingController
conroller){
    return SizedBox( width: double.infinity, child:
TextField(
    controller: conroller,
    readOnly: true,
    onChanged: (String value){
    if(int.parse(value) < 2){
        age_018 = true;
        age_218 = false;
    }else if(int.parse(value) >= 2){
        age_218 = true;
        age_018 = false;
    }else if(value.isEmpty){
        age_218 = false;
        age_018 = false;
    }
    },
    decoration: const InputDecoration(
        hintText: 'Type Something',
        border: OutlineInputBorder(),
        contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),));
    }

```

```

Widget widget_textfield3(TextEditingController
conroller, bool enableds){
    return SizedBox( width: double.infinity, child:
TextField(
    controller: conroller,
    readOnly: enableds,
    decoration: const InputDecoration(
        hintText: 'Type Something',
        border: OutlineInputBorder(),
        contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:

```

```

10))),));
    }

```

```

Widget widget_textfield(TextEditingController
conroller){
    return SizedBox( width: double.infinity, child:
TextField(
    controller: conroller,
    decoration: const InputDecoration(
        hintText: 'Type Something',
        border: OutlineInputBorder(),
        contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),));
    }

```

```

Widget widget_textarea(TextEditingController
conroller){
    return SizedBox( width: double.infinity, child:
TextField(
    controller: conroller,
    maxLines: 4,
    decoration: const InputDecoration(
        hintText: 'Type Something',
        border: OutlineInputBorder(),
        contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal:
10))),));
    }

```

```

Widget txtbox(String label, String astrsk){
    return RichText(text: TextSpan(text: label,style:
const TextStyle(fontSize: 18, color: Colors.black87),
children: [
        TextSpan(text: astrsk, style: const
TextStyle(color: Colors.deepOrange))
    ] );
    }

```

```

Widget txtbox_submit(String label, String astrsk){
    return RichText(text: TextSpan(text: label,style:
const TextStyle(fontSize: 18, color: Colors.black87,
fontWeight: FontWeight.w600), children: [
        TextSpan(text: astrsk, style: const
TextStyle(fontSize: 18, color:
Colors.black87,fontWeight: FontWeight.w400))
    ] );
    }

```

```

Widget widget_bday(TextEditingController
controller){
  return SizedBox( width: double.infinity, child:
TextField(
  controller: controller,
  readOnly: true ,
  decoration: const InputDecoration(
    hintText: 'Type Something',
    border: OutlineInputBorder(),
    contentPadding:
EdgeInsets.symmetric(vertical: 10, horizontal: 10)),
  onTap: () async{
    DateTime? selectdate = await showDatePicker(
      context: context,
      initialDate: DateTime.now(),
      firstDate: DateTime(1900),
      lastDate: DateTime(2100));
    if (selectdate != null){
      String dateformat = DateFormat('yyyy-MM-
dd').format(selectdate);
      setState(() {
        controller.text = dateformat;
      });
    }
  },
));
}

```

//image pickers -----

```

Widget imagepicker(){
  return Center(
    child: Stack(
      children: [
        CircleAvatar(
          radius: 50.0,
          backgroundImage: _imgfile != null?
FileImage(File(_imgfile!.path)): const
AssetImage("assets/profile.png") as ImageProvider,
          //AssetImage("assets/kids.png")

          // _imgfile == null? AssetImage(""):
FileImage(File(_imgfile!.path)),
        ),
        Positioned(
          bottom: 5,
          right: 8,
          child: InkWell(
            onTap: (){

```

```

      showModalBottomSheet(context:
context, builder: ((builder) => bottomMenu()));

      },
      child: const Icon(Icons.camera_alt, color:
Color.fromRGBO(239, 143, 3, 1), size: 30,)),
    ],
  ),
);
}

void getpic(ImageSource src) async{
  String path = "";
  final picfile = await _imagepicker.pickImage(
    source: src, imageQuality: 25
  );
  //
  // if(picfile != null){
  //   File imgfile = File(picfile.path);
  //   Directory appDir = await
getApplicationDocumentsDirectory();
  //   path =
'${appDir.path}/${DateTime.now().millisecondsSinc
eEpoch.toString()}.jpg';
  //   await imgfile.copy(path);
  //   print(path);
  // }

  // if(picfile != null){
  //
  //   Directory? getdocument = await
getExternalStorageDirectory();
  //   String docspath = getdocument!.path;
  //
  //   final file = File(picfile.path);
  //   final newfile = await file.copy(docspath);
  //   print(newfile.path);
  //
  // }

  converted = picfile!.path;
  setState(() {
    _imgfile = picfile as PickedFile?;
  });
  Navigator.pop(context);
}

Widget bottomMenu(){
  return Container(
    height: 100.0,

```

```

width: MediaQuery.of(context).size.width,
margin: const EdgeInsets.symmetric(horizontal:
20, vertical: 20),
child: Column(
  children: [
    const Text('Select Image', style: TextStyle
(fontSize: 25, fontWeight: FontWeight.w600, color:
Colors.black45)),
    const SizedBox(height: 10,),
    Row(
      mainAxisAlignment:
MainAxisAlignment.center,
      children: [
        ElevatedButton.icon(style:
ElevatedButton.styleFrom(backgroundColor:
Colors.white), onPressed: (){
          getpic(ImageSource.camera);
        }, icon: const
Icon(Icons.camera_alt_outlined, color:
Colors.black45,), label: const Text('Camera', style:
TextStyle(color: Colors.black45, fontWeight:
FontWeight.w600))),),
        const SizedBox(width: 20,),
        ElevatedButton.icon(style:
ElevatedButton.styleFrom(backgroundColor:
Colors.white), onPressed: (){
          getpic(ImageSource.gallery);
        }, icon: const Icon(Icons.image_outlined,
color: Colors.black45,), label: const Text('Gallery',
style: TextStyle(color: Colors.black45, fontWeight:
FontWeight.w600))),),
      ],
    ),
  ],
);
}

Widget ha_imagepicker(){
  return Center(
    child: Stack(
      children: [
        Image(image: _ha_imgfile != null?
FileImage(File(_ha_imgfile!.path)): const
AssetImage("assets/house.png") as
ImageProvider,
        // CircleAvatar(
        //   radius: 50.0,
        //   backgroundImage: _ha_imgfile != null?

```

```

FileImage(File(_ha_imgfile!.path)):
AssetImage("assets/kids.png") as ImageProvider,
        // ),
        Positioned(
          bottom: 5,
          right: 8,
          child: InkWell(
            onTap: (){
              showModalBottomSheet(context:
context, builder: ((builder) => ha_bottomMenu()));
            },
            child: const Icon(Icons.camera_alt, color:
Color.fromRGBO(239, 143, 3, 1), size: 30,)),
          ),
        );
      }

void ha_getpic(ImageSource src) async{
  Uint8List byte;
  var picfile = await
_ha_imagepicker.pickImage(source: src,
imageQuality: 25);

  // byte = await picfile!.readAsBytes();
  // print(byte.length);
  // converted2 = base64.encode(byte!);
  converted2 = picfile!.path;
  // print(converted2!.length);

  if(getlatlongimg){
    exifLatitudeLongitudePoint(File(picfile.path));
    // _getlatlong_img(File(picfile!.path));
  }

  setState(() {
    _ha_imgfile = picfile as PickedFile?;
  });
  Navigator.pop(context);
}

Widget ha_bottomMenu(){
  return Container(
    height: 100.0,
    width: MediaQuery.of(context).size.width,
    margin: const EdgeInsets.symmetric(horizontal:
20, vertical: 20),
    child: Column(
      children: [

```

```

        const Text('Select Image', style: TextStyle
(fontSize: 25, fontWeight: FontWeight.w600, color:
Colors.black45)),
        const SizedBox(height: 10,),
        Row(
            mainAxisAlignment:
MainAxisAlignment.center,
            children: [
                ElevatedButton.icon(style:
ElevatedButton.styleFrom(backgroundColor:
Colors.white),onPressed: (){
                    ha_getpic(ImageSource.camera);
                }, icon: const
Icon(Icons.camera_alt_outlined, color:
Colors.black45,), label: const Text('Camera', style:
TextStyle(color: Colors.black45, fontWeight:
FontWeight.w600))),),
                const SizedBox(width: 20,),
                ElevatedButton.icon(style:
ElevatedButton.styleFrom(backgroundColor:
Colors.white),onPressed: (){
                    ha_getpic(ImageSource.gallery);
                }, icon: const Icon(Icons.image_outlined,
color: Colors.black45,), label: const Text('Gallery',
style: TextStyle(color: Colors.black45, fontWeight:
FontWeight.w600))),),
            ],
        ),
    ],
),
);
}

// void textboxclearer_child() {
//   setState(() {
//     _text_ch_birthorder.clear();
//     _text_ch_name.clear();
//     _text_ch_age.clear();
//     ch_gender = 'Select';
//     _text_ch_height.clear();
//     _text_ch_weight.clear();
//     _text_ch_bday.clear();
//     _text_ch_bakuna.clear();
//     _text_ch_remarks.clear();
//     selecttypedelivery = 'Select';
//     selectoutcomedelivery = 'Select';
//     ch_vaccines_value = "";

```

```

//   _text_ch_birthplace.clear();
//   setState(() {
//     ch_vaccines_018 =
[false,false,false,false,false,false,false,false,fal
se,false];
//   });
//   setState(() {
//     ch_vaccines_218 =
[false,false,false,false,false,false,false,false];
//   });
//   });
// }
//
// void preginfo_clear() {
//   setState(() {
//     _txt_hpreg_weight1.clear();
//     _txt_hpreg_weight2.clear();
//     _txt_hpreg_weight3.clear();
//     selectedimmun = 'Select';
//     selectedvita = 'Select';
//     selecttypedelivery = 'Select';
//     selectoutcomedelivery = 'Select';
//     selectedpost = 'Select';
//     _txt_hpreg_weightpost.clear();
//     selectedvitapost = 'Select';
//   });
// }
//
// void preghist_clear(){
//   setState(() {
//     _txt_hpreg_chname.clear();
//     _txt_hpreg_age.clear();
//     chp_gender = 'Male';
//     _txt_hpreg_dateofbirth.clear();
//     _txt_hpreg_birthorder.clear();
//     selectoutcomedelivery = 'Select';
//     selecttypedelivery = 'Select';
//   });
// }

//End image picker -----
-----
}

enum HavePregnant {yes, no}
enum underage {yes, no}

```



```
import 'dart:io';
import '../Database/online_api_accessor.dart';
import '../config/variables.dart' as globals;
import
'package:e4psmapmig/funtions/Header.dart';
import
'package:e4psmapmig/funtions/sidemenu.dart';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:twilio_flutter/twilio_flutter.dart';

import '../funtions/textwidget.dart';
import '../funtions/toast_files.dart';
import '../funtions/willscope_router.dart';
```

```
class onlineData extends StatefulWidget {
  const onlineData({Key? key}) : super(key: key);

  @override
  State<onlineData> createState() =>
  onlineDataState();
}

class onlineDataState extends State<onlineData> {

  late TwilioFlutter twilioflutter;
```

```
final List<Map<String, dynamic>> _current_page
=[];
List<Map<String, dynamic>> json_data=[];
List<Map<String, dynamic>> searcheddata=[];
TextEditingController search_ctrl =
TextEditingController();
TextEditingController jumptopage =
TextEditingController();

var status_new = [
  'Select',
  'Active',
  'On Process',
  'Denied'
];

var _profilejson;
var _qualijson;
var _pregnantinfjson;
var _maternaljson;
var _childjson;
String _searchQuery = "";
bool next_state = true;
bool prev_state = true;
int _currentpage = 0;
final int _number_of_row = 10;
int data_length = 0;
```

```

int data_modu = 0;
int numpage = 0;
bool isloading = true;
String numdisplay = '';
// bool isloading_row = false;
bool childloadin = true;
bool mathistload = true;
bool issearching = false;
String selection = '';
List<Map<String, dynamic>> cor_child = [];
List<Map<String, dynamic>> cor_preg = [];
final TextEditingController _deletion_code =
TextEditingController();
bool isdropbox = false;
String selectedstatus = 'Select';

loaddata()async{
  List<dynamic> jsondata = _holdjoin;
  List<Map<String, dynamic>> datalist =
List<Map<String,
dynamic>>.from(jsondata.map((x) => Map<String,
dynamic>.from(x)));
  json_data.addAll(datalist);
  _total_page();
  print(json_data);

  setState(() {
    isloading = false;
  });
  _rowdata(0);
}

var search_criteria=[
  'Select',
  'Lastname',
  'Status',
];
String default_search = 'Select';

void _rowdata(int action){
  int start = 0;
  int end = 0;
  print('the orig $_currentpage');

  if(action == 0){
    start =(_currentpage) * _number_of_row;
    end = start + _number_of_row;
    setState(() {
      isloading = true;

```

```

});
    _current_page.clear();
    Future.delayed(const Duration(seconds: 1),(){
      _current_page.addAll(json_data.sublist(start,
end));
      print(_currentpage);
      _currentpage++;
      print('after next $_currentpage');
      setState(() {
        isloading = false;
        numdisplay = _currentpage.toString();
      });
    });
  }
  else if(action == 1){

    start =(_currentpage -2) * _number_of_row;
    end = start + _number_of_row;
    setState(() {
      prev_state = true;
    });
    setState(() {
      isloading = true;
    });
    _current_page.clear();
    Future.delayed(const Duration(seconds: 1),(){
      _current_page.addAll(json_data.sublist(start,
end));
      print(_currentpage);
      _currentpage --;
      print('after minus $_currentpage');
      setState(() {
        isloading = false;
        numdisplay = _currentpage.toString();
      });
    });
  }
  }else if(action == 2){

    _currentpage = int.parse(jumptopage.text)+1;
    start =(_currentpage -2) * _number_of_row;
    end = start + _number_of_row;
    setState(() {
      prev_state = true;
    });
    setState(() {
      isloading = true;
    });
  }

```

```

        _current_page.clear();
        Future.delayed(const Duration(seconds: 1),(){
            _current_page.addAll(json_data.sublist(start,
end));
            print(_currentpage);
            _currentpage --;

            setState(() {
                isloading = false;
                numdisplay = _currentpage.toString();
            });
        });

        if(end >= json_data.length){
            end = json_data.length;
            setState(() {
                next_state = false;
            });
        }else{
            setState(() {
                next_state = true;
            });
        }

        if(start <= 0){
            setState(() {
                prev_state = false;
            });
        }else{
            setState(() {
                prev_state = true;
            });
        }
    }

    var _holdjoin;
    var _holdhousehold;
    bool isLoading = false;

    alljoin()async{
        _holdjoin = await
e4ps_applicant_info(globals.user_brgy);
        // print(_holdjoin);
    }

    loadhousehold(String serial)async{
        _holdhousehold = await
e4ps_household_member(serial);

```

```

        print(_holdhousehold);
        setState(() {
            isLoading = false;
        });
    }

    void criteria_selection(){
        if(default_search == 'Lastname'){
            selection = 'e4ps_profile_lname';
        }else if(default_search == 'Status') {
            selection = 'e4ps_profile_status';
        }
    }

    void _query(){
        setState(() {

            if(isdropdown){
                if(selectedstatus == "On Process"){
                    _searchQuery = "0";
                }else if(selectedstatus == "Active") {
                    _searchQuery = "3";
                }else if(selectedstatus == "Denied") {
                    _searchQuery = "2";
                }
            }else{
                _searchQuery = search_ctrl.text;
            }

            searcheddata.clear();

            if(RegExp(r'\d').hasMatch(_searchQuery)){
                searcheddata.addAll(json_data.where((item)
=> item[selection] == _searchQuery.toString()));
            }else{
                print('diriko');
                searcheddata.addAll(json_data.where((item)
=> item[selection].toLowerCase() ==
_searchQuery.toString().toLowerCase()));
            }
        });
    }

    void _total_page(){
        if(json_data.length % 10 >0){
            setState(() {
                numpage = json_data.length ~/ 10+1;
            });
        }
    }

```

```

    }else{
      setState(() {
        numpage = json_data.length ~/ 10;
      });
    }
  }

  dataloader()async{
    await alljoin();
    loaddata();
  }

  @override
  void initState() {
    // TODO: implement initState
    dataloader();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: top_header(context),
      drawer: const Navigationmenu(),
      body: WillPopScope(
        onWillPop: ()async{
          scope_dashboard(context);
          return false;
        },
        child: SingleChildScrollView(
          child: Container(
            margin: const EdgeInsets.all(10),
            child: Column(
              children: [
                const SizedBox(height: 20,),
                label('Applicant Data', 25),
                const SizedBox(height: 20,),
                Row(
                  children: [
                    Expanded(flex: 4, child: DecoratedBox(
                      decoration: BoxDecoration(
                        color: Colors.white,
                        border: Border.all(color: Colors.grey,
width: 1),
                        borderRadius:
BorderRadius.circular(5)
                      ),
                      child: Padding(

```

```

padding: const
EdgeInsets.symmetric(horizontal: 10),
child: DropdownButton(
  dropdownColor: Colors.white,
  isExpanded: true,
  underline: Container(color:
Colors.white,)),
  value: default_search,
  alignment: Alignment.center,
  items: search_criteria.map((String
defaultSearch){
    return DropdownMenuItem(
      value: defaultSearch,
      child: Text(defaultSearch,
textAlign: TextAlign.center,))
    );
  }).toList(), onChanged: (String?
newval){
    setState(() {
      default_search = newval!;
      if(default_search == 'Status'){
        isdropbox = true;
      }else{
        isdropbox = false;
        selectedstatus = 'Select';
      }
    });
  },),),),
  const SizedBox(width: 5,),
  Visibility(
    visible: isdropbox,
    child: Expanded(flex: 4, child:
DecoratedBox(
  decoration: BoxDecoration(
    color: Colors.white,
    border: Border.all(color:
Colors.grey, width: 1),
    borderRadius:
BorderRadius.circular(5)
  ),
  child: Padding(
    padding: const
EdgeInsets.symmetric(horizontal: 10),
    child: DropdownButton(
      dropdownColor: Colors.white,
      isExpanded: true,
      underline: Container(color:

```

```

Colors.white,),
    value: selectedstatus,
    alignment: Alignment.center,
    items: status_new.map((String
selectedstatus){
    return DropdownMenuItem(
        value: selectedstatus,
        child: Text(selectedstatus,
textAlign: TextAlign.center,),
    );
    }).toList(), onChanged: (String?
newval){
    setState(() {
        selectedstatus = newval!;
    });
    },),
    ),),
    ),
    Visibility(visible: !isdropbox, child:
Expanded(flex: 5,child:
widget_textfield2(search_ctrl, 'Search Member'))),
    const SizedBox(width: 10,),
    Expanded(flex: 2, child:
ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1), padding: const
EdgeInsets.all(10)),onPressed: () async{
    if(search_ctrl.text == " &&
selectedstatus == 'Select'){
        setState((){
            issearching = false;
        });
    }else if(default_search == 'Select'){
        fluttermtoast_func('Please Select
Search Criteria');
    }else if (default_search != 'Select' ||
search_ctrl.text != " || selectedstatus != 'Select'){
        setState((){
            issearching = true;
        });
        criteria_selection();
        _query();
    }
    }, child: const
Icon(CupertinoIcons.search, size: 25)),),
    ],
    ),
    const SizedBox(height: 20,),

```

```

Container(
    padding: const EdgeInsets.all(5),
    decoration: BoxDecoration(color:
Colors.white, border: Border.all(color: Colors.grey),
borderRadius: BorderRadius.circular(10) ),
    child: Column(
        children: [
            Visibility(
                visible: !issearching,
                child: SingleChildScrollView(
                    scrollDirection: Axis.horizontal,
                    child: isloading ? const Center(child:
CircularProgressIndicator(),):datatable()
                ),
            ),
            Visibility(
                visible: issearching,
                child: SingleChildScrollView(
                    scrollDirection: Axis.horizontal,
                    child: isloading ? const
Center(child:
CircularProgressIndicator(),):datatablesr()
                ),
            ),
        ],
    ),
    const SizedBox(height: 10,),
    Visibility(
        visible: !issearching,
        child: Row(
            mainAxisAlignment:
MainAxisAlignment.center,
            children: [
                ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1), shape: const
CircleBorder()),onPressed: prev_state? (){
                _rowdata(1);
            }:null,child: const
Icon(Icons.arrow_left),),
            Container(
                padding: const
EdgeInsets.symmetric(horizontal: 15),
                decoration: BoxDecoration(
                    border: Border.all(
                        color: const
Color.fromRGBO(75, 79, 177, 1)),

```

```

        borderRadius:
BorderRadius.circular(10)),
        child:
Text("$numdisplay/$numpage", style: const
TextStyle(fontSize: 18)),),

ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1), shape: const
CircleBorder()),onPressed: next_state? (){
_rowdata(0);

}:null,child: const
Icon(Icons.arrow_right), ),

],
),
),
Container(
margin: const
EdgeInsets.symmetric(horizontal: 70),
child: Row(
mainAxisAlignment:
MainAxisAlignment.center,
children: [
label2('Jump page to ', 16),
Expanded(child:
text_area_sml(jumptopage, "Page")),
ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1), shape: const
CircleBorder()),onPressed:()){
if(int.parse(jumptopage.text) <=
numpage){
_rowdata(2);
}else{
flutters_toast_func('Maximum page
number is: $numpage');
}

},child: const
Icon(Icons.fast_forward),),

],
),
),

const SizedBox(height: 30,)

```

```

    ],
  ),
),
),
);
}

Widget datatable(){
return DataTable(columnSpacing: 25,columns: [
DataColumn(label: label('View', 20)),
DataColumn(label: label('Status', 20)),
// DataColumn(label: label('index', 20)),
DataColumn(label: label('Fullname', 20)),
DataColumn(label: label('Contact', 20)),
DataColumn(label: label('Barangay', 20)),
// DataColumn(label: label('Year Registered',
20))
],
rows:_current_page.map((row) =>
DataRow(cells: [

DataCell(ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1), padding: const
EdgeInsets.all(10)),onPressed: () async{
// // print(_profilejson[index]['hh_id'] + " " +
index.toString());
// newarr(index);
// viewprofile(index);
// print(row['e4ps_profile_serial']);
setState(() {
isLoading = true;
});
await
loadhousehold(row['e4ps_profile_serial'].toString()
);
viewprofile(row['e4ps_profile_serial']);
}, child: const Icon(CupertinoIcons.eye, size:
25),)),
// DataCell(Text((index).toString()),

DataCell(txt_status(row['e4ps_profile_status'].toSt
ring()))),
DataCell(Text("${row['e4ps_profile_lname']},
${row['e4ps_profile_fname']}
${row['e4ps_profile_mname']}")),

DataCell(Text(row['e4ps_profile_contact'].toString(

```

```

    )),

    DataCell(Text(row['e4ps_address_brgy'].toString()))
    ,
    // DataCell(Text(row['qual_year_registered']))

    ])).toList()
    );
}

Widget datatablesr(){
  return DataTable(columnSpacing: 25,columns: [
    DataColumn(label: label('View', 20)),
    DataColumn(label: label('Status', 20)),
    // DataColumn(label: label('index', 20)),
    DataColumn(label: label('Fullname', 20)),
    DataColumn(label: label('Contact', 20)),
    DataColumn(label: label('Barangay', 20)),
    // DataColumn(label: label('Year Registered',
20))
  ],
    rows:searcheddata.map((row) =>
DataRow(cells: [

    DataColumn(ElevatedButton(style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1), padding: const
EdgeInsets.all(10)),onPressed: () async{
    //    // print(_profilejson[index]['hh_id'] + "
" + index.toString());
    // newarr(index);
    // viewprofile(index);
    // print(row['e4ps_profile_serial']);
    setState(() {
      isLoading = true;
    });
    await
loadhousehold(row['e4ps_profile_serial'].toString()
));

viewprofile(row['e4ps_profile_serial'].toString());
    }, child: const Icon(CupertinoIcons.eye, size:
25)),),
    // DataCell(Text((index).toString()))),

DataRow(txt_status(row['e4ps_profile_status'].toSt
ring()))),
    DataColumn(Text("${row['e4ps_profile_lname']},
${row['e4ps_profile_fname']}

```

```

    ${row['e4ps_profile_mname']}"))),

DataRow(Text(row['e4ps_profile_contact'].toString(
))),

DataRow(Text(row['e4ps_address_brgy'].toString()))
    ,
    // DataCell(Text(row['qual_year_registered']))

    ])).toList()
    );
}

Future<void> deleteprofile(int index) async {
  print(_profilejson[index]['hh_image']);

  final dir = File(_profilejson[index]['cont_image']);
  if( await dir.exists()){
    await dir.delete();
    print("na delete na sya");
  }else{
    print('wala sya');
  }
}

Widget label(String text, double fsize){
  return Text(
    text, style: TextStyle(fontSize: fsize, fontWeight:
FontWeight.w600),
  );
}

Widget label2(String text, double fsize){
  return Text(
    text, style: TextStyle(fontSize: fsize, fontWeight:
FontWeight.w600, color: const
Color.fromRGBO(75, 79, 177, 1)),
  );
}

Future<void> viewprofile(String hhid) async{
  print(hhid);
  var refcode = await get_refcode(hhid);
  print(refcode);
  bool haveref = false;
  String refecode = "";

  if(refcode[0]['haveserial'] == 'true'){
    haveref = true;

```

```

        refecode = refcode[0]['e4ps_ref_code'];
    }else{
        haveref = false;
    }

    int index = 0;
    for(int i = 0; i<_holdjoin.length; i++){
        if (hhid == _holdjoin[i]['e4ps_profile_serial']){
            // print(i);
            index = i;
        }
    }

    return showDialog(context: context, builder:
(BuildContext context){
        return AlertDialog(
            title: const Center(child: Text('Beneficiary
Profile')),
            content: SingleChildScrollView(
                child: Column(
                    mainAxisAlignment:
MainAxisAlignment.start,
                    crossAxisAlignment:
CrossAxisAlignment.start,
                    children: [
                        Center(child:
CircleAvatar(backgroundImage:
_holdjoin[index]['e4ps_profile_picture'] != ""?
NetworkImage(_holdjoin[index]['e4ps_profile_pict
ure']): const AssetImage('assets/profile.png') as
ImageProvider, radius: 50,)),
                        const SizedBox(height: 20,),
                        Center(child:
label("${_holdjoin[index]['e4ps_profile_lname']},
${_holdjoin[index]['e4ps_profile_fname']}
${_holdjoin[index]['e4ps_profile_mname']}", 20)),
                        Center(child: richtxt("", 'Household Head - ' "
"+
_holdjoin[index]['e4ps_qualify_type'].toString()))),
                        Visibility(visible: haveref,child:
Center(child: txt_with_size_ref(18, 'Ref. Code:
$refecode'))),
                        const SizedBox(height: 10,),

                        label2("Profile", 18),
                        richtxt('Gender: ',
_holdjoin[index]['e4ps_profile_gender'].toString()),

```

```

                        richtxt('Civil Status: ',
_holdjoin[index]['e4ps_profile_cs'].toString()),
                        richtxt('Birthdate: ',
_holdjoin[index]['e4ps_profile_birthdate'].toString(
)),
                        richtxt('Education: ',
_holdjoin[index]['e4ps_profile_educ'].toString()),
                        richtxt('Occupation: ',
_holdjoin[index]['e4ps_profile_job'].toString()),
                        richtxt('Contact No.: ',
_holdjoin[index]['e4ps_profile_contact'].toString())
,
                        richtxt('Email: ',
_holdjoin[index]['e4ps_profile_email'].toString()),

                        const SizedBox(height: 5,),
                        const Divider(thickness: 2,),
                        const SizedBox(height: 5,),

                        label2("Qualification", 18),
                        richtxt('Monthly Income: ',
_holdjoin[index]['e4ps_qualify_income'].toString())
,
                        richtxt('No. of Household Members: ',
_holdjoin[index]['e4ps_qualify_population'].toStrin
g()),
                        richtxt('Pregnancy Date: ',
_holdjoin[index]['e4ps_qualify_pregnant_num'].to
String()),
                        richtxt('No. of 0-18 years old: ',
_holdjoin[index]['e4ps_qualify_minor_num'].toStri
ng()),
                        richtxt('Year of Residency: ',
_holdjoin[index]['e4ps_qualify_yor'].toString()),
                        richtxt('Type of Residency: ',
_holdjoin[index]['e4ps_qualify_restype'].toString())
,
                        richtxt('House Description: ',
_holdjoin[index]['e4ps_qualify_description'].toStri
ng()),

                        const SizedBox(height: 5,),
                        const Divider(thickness: 2,),
                        const SizedBox(height: 5,),

                        label2("Address", 18),
                        const SizedBox(height: 5,),

                        Image(image:_holdjoin[index]['e4ps_address_img'])

```

```

!= ""?
NetworkImage(_holdjoin[index]['e4ps_address_image']): const AssetImage("assets/house.png") as
ImageProvider),
    const SizedBox(height: 10,),
    richtxt('Address: ',
"$[_holdjoin[index]['e4ps_address_purok']],
${_holdjoin[index]['e4ps_address_brgy']],
${_holdjoin[index]['e4ps_address_city']],
${_holdjoin[index]['e4ps_address_city']],
${_holdjoin[index]['e4ps_address_province']}" ),

    const SizedBox(height: 5,),
    const Divider(thickness: 2,),
    const SizedBox(height: 5,),

    Container(
        decoration: BoxDecoration(border:
Border.all(color: Colors.grey.shade400, width: 1)),
        child: SingleChildScrollView(
            scrollDirection: Axis.horizontal,
            child: DataTable(columnSpacing:
25,columns: [
                // DataColumn(label: label('Edit', 20)),
                DataColumn(label: label('Relation', 15)),
                DataColumn(label: label('Name', 15)),
                DataColumn(label: label('Birthdate',
15)),
                DataColumn(label: label('Gender', 15)),
                DataColumn(label: label('Civil Status',
15)),
                DataColumn(label: label('Occupation',
15)),
                DataColumn(label: label('Contact', 15)),
                DataColumn(label: label('Email', 15))

            ],rows: isLoading
            ? [const DataRow(cells:
[DataCell(CircularProgressIndicator())]):List.genera
te(_holdhousehold.length, (index){
                return DataRow(cells: [

DataCell(Text(_holdhousehold[index]['e4ps_hold_r
elation']).toString()),

DataCell(Text(_holdhousehold[index]['e4ps_hold_f

```

```

ullname']).toString()),

DataCell(Text(_holdhousehold[index]['e4ps_hold_
birthdate']).toString()),

DataCell(Text(_holdhousehold[index]['e4ps_hold_g
ender']).toString()),

DataCell(Text(_holdhousehold[index]['e4ps_hold_c
s']).toString()),

DataCell(Text(_holdhousehold[index]['e4ps_hold_
occupation']).toString()),

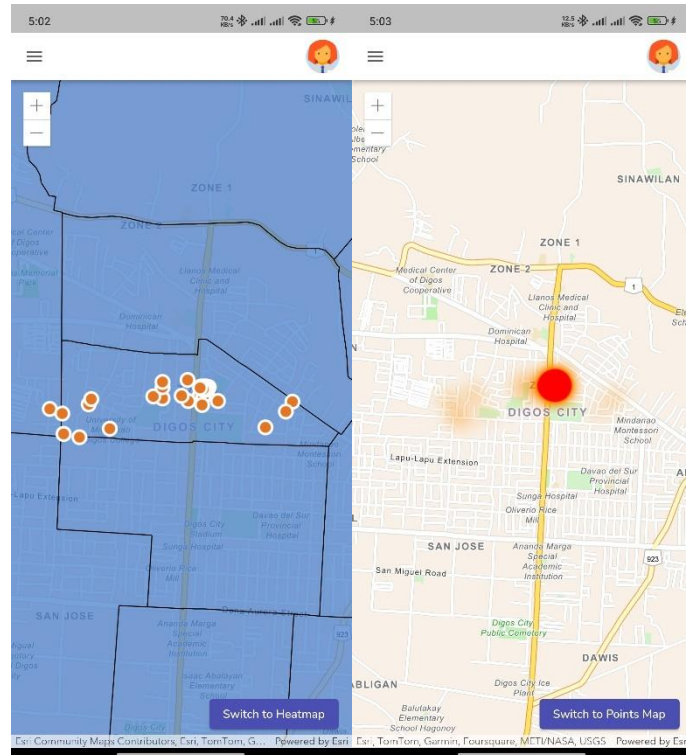
DataCell(Text(_holdhousehold[index]['e4ps_hold_c
ontact']).toString()),

DataCell(Text(_holdhousehold[index]['e4ps_hold_e
mail']).toString()),
                ]);
            }) ),
        ),
        ),
        const SizedBox(height: 20,),
    ],
    ),
    ),
    actions: [
        TextButton(onPressed: (){
            Navigator.of(context).pop();
        }, child: const Text('Close'))
    ],
    );
    });
    }

Widget richtxt(String title, data){
    return RichText(text: TextSpan(text: title, style:
const TextStyle(fontWeight: FontWeight.bold,
color: Colors.black), children:[
        TextSpan(text: data, style: const
TextStyle(fontWeight: FontWeight.w400, color:
Colors.black))
    ] ));
    }

}

```



```
import 'dart:convert';
import 'dart:typed_data';
```

```
import 'package:e4psmapmig/config/APIs.dart';
import
'package:e4psmapmig/funtions/textwidget.dart';
import 'package:flutter/material.dart';
import
'package:flutter_inappwebview/flutter_inappwebv
iew.dart';
import '../config/variables.dart' as globals;
```

```
import '../funtions/Header.dart';
import '../funtions/sidemenu.dart';
import '../funtions/willscope_router.dart';
```

```
class Mappage extends StatefulWidget {
  const Mappage({Key? key}) : super(key: key);

  @override
  State<Mappage> createState() =>
    _MappageState();
}
```

```
class _MappageState extends State<Mappage> {
  bool flutter_map = true;
  bool web_map = true;
  String user_brgy = globals.user_brgy;
```

```
bool isloadingmap = false;
```

```
String buttontxt = "Switch to Heatmap";
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    floatingActionButton: ElevatedButton(
      style:
        ElevatedButton.styleFrom(backgroundColor: const
          Color.fromRGBO(75, 79, 177, 1) ),
      onPressed: (){
        setState(() {
          web_map = !web_map;
          isloadingmap = true;
        });
        if(web_map){
          setState(() {
            buttontxt = "Switch to Heatmap";
          });
        }else if (!web_map){
          setState(() {
            buttontxt = "Switch to Points Map";
          });
        }
      },
      child: Text(buttontxt) ),
```

```

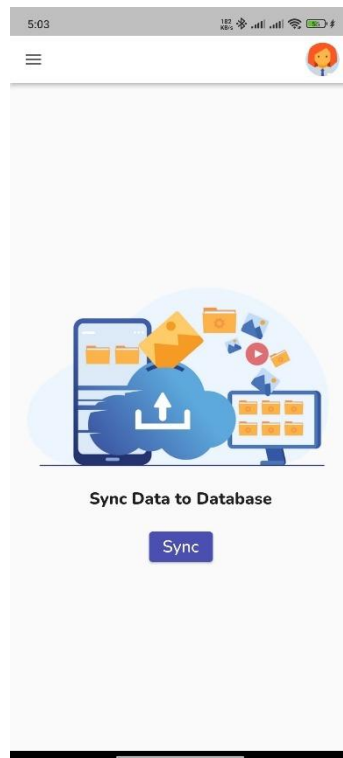
appBar: top_header(context),
drawer: const Navigationmenu(),
body: WillPopScope(
  onWillPop: ()async{
    scope_dashboard(context);
    return false;
  },
  child: Container(
    child: Center(
      child: Stack(
        children:[ Column(
          children: [
            Visibility(visible: isloadingmap,child:
Container(
  margin: const EdgeInsets.all(40),
  child: Center(child: Column(
    children: [
      const
CircularProgressIndicator(strokeWidth: 10,
valueColor: AlwaysStoppedAnimation<Color>(
  Color.fromRGBO(75, 79, 177, 1)
)),
  const SizedBox(height: 20,)),
  label('Loading Map', 20)
]),
)),
)),
Visibility(
  visible: web_map,
  child: Flexible(
    child: InAppWebView(
      initialUrlRequest: URLRequest(
        url:
WebUri(api_config.e4ps_map),
        method: 'POST',
        body:
Uint8List.fromList(utf8.encode("barangay=$user_b
rgy")),
        headers: {
          'Content-Type': 'application/x-
www-form-urlencoded'
        }
      ),
      onWebViewCreated: (controller) {},
      onLoadStart: (controller, url){
        setState(() {
          isloadingmap = true;
        });
      },
      onLoadStop: (controller, url){
        setState(() {
          isloadingmap = false;
        });
      },
    ),
  ),
),
onWebViewCreated: (controller) {},
onLoadStart: (controller, url){
  setState(() {
    isloadingmap = true;
  });
},
],
),
),
);
}
}

```

```

onLoadStop: (controller, url){
  setState(() {
    isloadingmap = false;
  });
},
),
),
),
Visibility(
  visible: !web_map,
  child: Flexible(
    child: InAppWebView(
      initialUrlRequest: URLRequest(
        url:
WebUri(api_config.heatmap_api),
        method: 'POST',
        body:
Uint8List.fromList(utf8.encode("barangay=$user_b
rgy")),
        headers: {
          'Content-Type': 'application/x-
www-form-urlencoded'
        }
      ),
      onWebViewCreated: (controller) {},
      onLoadStart: (controller, url){
        setState(() {
          isloadingmap = true;
        });
      },
      onLoadStop: (controller, url){
        setState(() {
          isloadingmap = false;
        });
      },
    ),
  ),
),
),
),
),
);
}
}

```



```
import 'dart:convert';
```

```
import
'package:e4psmapmig/Database/synchronize.dart';
import 'package:flutter/material.dart';
import 'package:fluttertoast/fluttertoast.dart';
import 'package:intl/intl.dart';
import '../config/variables.dart' as globals;
import '../Database/sqlitehelpernew.dart';
import '../funtions/Header.dart';
import '../funtions/sidemenu.dart';
import '../funtions/willscope_router.dart';
```

```
class Syncdata extends StatefulWidget {
  const Syncdata({super.key});
```

```
  @override
  State<Syncdata> createState() =>
  _SyncdataState();
}
```

```
class _SyncdataState extends State<Syncdata> {
```

```
  bool notosync = false;
```

```
  fluttertoast(String message){
```

```
Fluttertoast.showToast(msg: message,
  toastLength: Toast.LENGTH_SHORT,
  gravity: ToastGravity.CENTER,
  timeInSecForIosWeb: 2,
  backgroundColor: Colors.grey,
  textColor: Colors.white,
  fontSize: 18
);
}
```

```
int count = 0;
```

```
//-----jsn-----
-----
```

```
// allqualif() async{
//   var dbhelper = DBhelper_qualification();
//   var profiles = await
dbhelper.getjson_qualification_sync();
//   return profiles;
// }
//
```

```
allprofile()async{
  var dbhelper = DBhelper_qualification_new();
  var profiles = await dbhelper.getjson_join_sync();
  print(profiles);
  return profiles;
}
```

```

allhousehold()async{
    var dbhelper = DBhelper_qualification_new();
    var household = await
dbhelper.getjson_household();
    print(household);
    return household;
}

update_status(String serial, refcode)async{
    DBhelper_qualification_new dbhelper;
    dbhelper = DBhelper_qualification_new();
    var retval = await
dbhelper.update_synced(serial, refcode);
    return retval;
}

// allpreginfo()async{
//     var dbhelper = DBhelper_qualification();
//     var profiles = await
dbhelper.getjson_preginfo_sync();
//     return profiles;
// }
//
// allchild()async{
//     var dbhelper = DBhelper_qualification();
//     var profiles = await
dbhelper.getjson_child_sync();
//     return profiles;
// }
//
// allmaternal()async{
//     var dbhelper = DBhelper_qualification();
//     var profiles = await
dbhelper.getjson_maternal_sync();
//     return profiles;
// }

//-----counters-----
//

countProf() async {
    DBhelper_qualification_new dbhelper;
    dbhelper = DBhelper_qualification_new();

```

```

var numb = await dbhelper.count_unsync();

if (numb! <= 0){
    setState() {
        notosync = false;
    };
}else{
    setState() {
        notosync = true;
    };
}
}
//
// countPreginf() async {
//     var dbhelper;
//     dbhelper = DBhelper_qualification();
//     var numb = await
dbhelper.countpreginf_sync();
//     return numb;
// }
//
// countchild() async {
//     var dbhelper;
//     dbhelper = DBhelper_qualification();
//     var numb = await dbhelper.countchild_sync();
//     return numb;
// }
//
// countmaternal() async {
//     var dbhelper;
//     dbhelper = DBhelper_qualification();
//     var numb = await
dbhelper.countmaternal_sync();
//     return numb;
// }

//-----synced update -----
//

// update_profile(String id) async{
//     var dbhelper;
//     dbhelper = DBhelper_qualification();
//     await dbhelper.updateSynced_prof(id);
// }
//
// update_child(String id) async{
//     var dbhelper;
//     dbhelper = DBhelper_qualification();
//     await dbhelper.updateSynced_child(id);

```



```

Navigator.pop(context), child: const Text("OK"))
    ],
    ));
    setState((){isLoading = false;});
  } else{
    setState(() {syncingnow = "Syncing
Profiles";});

    for(int i = 0; i<profiles.length; i++){
      profileSerial = await
      upload_profile(profiles[i]['e4ps_profile_picture'].to
      String(), profiles[i]['e4ps_profile_lname'].toString(),
      profiles[i]['e4ps_profile_fname'].toString(),
      profiles[i]['e4ps_profile_mname'].toString(),
      profiles[i]['e4ps_profile_qualifier'].toString(),
      profiles[i]['e4ps_profile_gender'].toString(),

      profiles[i]['e4ps_profile_cs'].toString(),
      dateconvert(profiles[i]['e4ps_profile_birthdate']).t
      oString(), profiles[i]['e4ps_profile_educ'].toString(),
      profiles[i]['e4ps_profile_job'].toString(),
      profiles[i]['e4ps_profile_contact'].toString(),
      profiles[i]['e4ps_profile_email'].toString(),
      profiles[i]['e4ps_qualify_type'].toString(),

      profiles[i]['e4ps_qualify_income'].toString(),
      profiles[i]['e4ps_qualify_population'].toString(),
      profiles[i]['e4ps_qualify_pregnant_option'].toStrin
      g(),
      profiles[i]['e4ps_qualify_pregnant_num'].toString()
      , "0", profiles[i]['e4ps_qualify_minor_option'], "0",
      profiles[i]['e4ps_qualify_yor'].toString(),

      profiles[i]['e4ps_qualify_restype'].toString(),
      profiles[i]['e4ps_qualify_description'].toString(),
      profiles[i]['e4ps_address_house'].toString(),
      profiles[i]['e4ps_address_street'].toString(),
      profiles[i]['e4ps_address_purok'].toString(),
      profiles[i]['e4ps_address_brgy'].toString(),
      profiles[i]['e4ps_address_city'].toString(),
      profiles[i]['e4ps_address_province'].toString(),

      profiles[i]['e4ps_address_zipcode'].toString(),
      profiles[i]['e4ps_address_lat'].toString(),
      profiles[i]['e4ps_address_long'].toString(),
      profiles[i]['e4ps_address_img'].toString(),
      globals.user_id.toString());
      count++;
    }
  }

```

```

for(int x = 0; x< household.length;
x++){
  print(await
  upload_household(profileSerial[0]['profileserial'],
  household[x]['e4ps_hold_relation'],
  household[x]['e4ps_hold_fullname'],
  dateconvert(household[x]['e4ps_hold_birthday']),
  household[x]['e4ps_hold_gender'],
  household[x]['e4ps_hold_cs'],
  household[x]['e4ps_hold_occupation'],
  household[x]['e4ps_hold_contact'],
  household[x]['e4ps_hold_email']));
  }

  update_status(profiles[0]['e4ps_profile_serial'],
  profileSerial[0]['refcode']);

  setState(() {syncingnow = "Sync
Complete";});
  setState((){isLoading = false;});
  setState(() {syncingnow = "Starting to
sync";});

  showDialog <String>(context:
context,
    builder: (BuildContext context) =>
    AlertDialog(
      icon: const Icon(Icons.info, color:
Colors.blue,size: 50,),
      title: const Text("Data Successfully
Synced"),
      content: Text("$count data
successfully uploaded"),
      actions: [
        TextButton(onPressed: () =>
Navigator.pop(context), child: const Text("OK"))
        ],
      ));
  }
}catch (e){
  setState((){isLoading = false;});
  fluttermtoast(e.toString());
}
}:null,style:
ElevatedButton.styleFrom(backgroundColor: const
Color.fromRGBO(75, 79, 177, 1)),child: const
Text('Sync', style: TextStyle(fontSize: 20,,),),

```

```

// ElevatedButton(onPressed: ()async{
//   print(await update_profile("727"));
//
//
// }, child: Text("Test Update"))

// ElevatedButton(onPressed: ()async{
//   print( await countmaternal());
//   setState((){isLoading = true;});
//
// }, child: Text('Text'))
],
),
),
Visibility(visible: isLoading,child:
Container(padding: const
EdgeInsets.all(20),decoration: const
BoxDecoration(shape: BoxShape.rectangle, color:
Colors.black87),child: Column(
  mainAxisAlignment:
MainAxisAlignment.center,
  crossAxisAlignment:
CrossAxisAlignment.center,
  children: [

```

```

Center(child: SizedBox(height: 70, width:
70,child: isLoading?const CircularProgressIndicator(
  backgroundColor: Color.fromRGBO(237,
237, 238, 1.0), strokeWidth: 10, valueColor:
AlwaysStoppedAnimation<Color>(
  Color.fromRGBO(75, 79, 177, 1)
),
):const Text("Loading"))),
const SizedBox(height: 20,),
text('Please wait while we sync your
data'),
const SizedBox(height: 40,),
text(syncingnow),
],
))),
],
),
);
}
Widget text(String data){
  return Text(data, style: const TextStyle(fontSize:
25, fontWeight: FontWeight.w600,
  color: Color.fromRGBO(255, 255, 255,
1.0)),textAlign: TextAlign.center,);
}
}

```

```

import
'package:e4psmapmig/pages/forms/latlong_update.dart';
import
'package:e4psmapmig/pages/forms/new_form.dart';
import
'package:e4psmapmig/pages/tables/display_online_data.dart';
import
'package:e4psmapmig/pages/tables/displaydata.dart';
import
'package:e4psmapmig/pages/views/announcement.dart';
import
'package:e4psmapmig/pages/views/beneficiary_dashboard.dart';
import
'package:e4psmapmig/pages/views/dashboard.dart';
import
'package:e4psmapmig/pages/views/dashboard_online.dart';
import
'package:e4psmapmig/pages/views/map.dart';
import
'package:e4psmapmig/pages/views/syncpage.dart';
;
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import
'package:e4psmapmig/pages/forms/online_applicant.dart';

import '../pages/views/Support.dart';
import '../pages/views/about.dart';
import '../config/variables.dart' as globals;
import '../pages/views/beneficiary_profile.dart';

class Navigationmenu extends StatefulWidget {
  const Navigationmenu({super.key});

  @override
  State<Navigationmenu> createState() =>
    _NavigationmenuState();
}

```

```

class _NavigationmenuState extends
State<Navigationmenu> {
  // const Navigationmenu({Key? key}) : super(key:
key);
  List<Color> textcol =
[Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54];
  List<Color> icol =
[Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54,Colors.black54];
  List<Color> tilecol =
[Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent,Colors.transparent];

  bool viewmenu = false;
  bool v_dashboard = false;
  bool v_4ps_info = false;
  bool isopen = globals.ismenuopen;
  IconData _icon = Icons.keyboard_arrow_right;
  bool v_field = false;
  bool v_approved = true;
  bool v_announce = false;
  bool v_log = false;
  bool v_supp = false;
  bool v_about = false;
  bool v_map = false;
  // bool v_user = false;
  bool v_newappl = false;
  bool v_update = false;
  bool v_updatelatlong = false;
  bool v_userdashboard = false;
  bool v_user_profile = false;
  bool is_online = false;

  userconfig(){
    if (globals.user_perm == 'Field Worker'){
      v_dashboard = true;
      v_newappl = false;
      viewmenu = false;
    }
  }
}

```

```

v_field = true;
v_4ps_info = true;
v_approved = true;
v_announce = false;
v_log = false;
v_supp = false;
v_about = true;
v_map = false;
// v_user = false;
v_log = false;
v_supp = false;
v_about = true;
v_update = false;
v_updatelatlong = false;
is_online = false;
}else if (globals.user_perm == 'User'){
v_dashboard = false;
v_newappl = false;
viewmenu = false;
v_field = false;
v_approved = false;
v_4ps_info = false;
v_announce = false;
v_log = false;
v_supp = false;
v_about = false;
v_map = false;
// v_user = false ;
v_log = false;
v_supp = true;
v_about = true;
v_update = false;
v_userdashboard = true;
v_user_profile = true;
}else{
v_dashboard = true;
v_4ps_info = true;
is_online = true;
v_newappl = true;
viewmenu = false;
v_field = true;
v_approved = true;
v_announce = true;
v_log = true;
v_supp = true;
v_about = true;
v_map = true;
// v_user = true ;
v_log = true;

```

```

v_supp = true;
v_about = true;
v_update = true;
v_updatelatlong = true;
}
}

void colorchange(var ind){
for( var i = 0; i<13; i++){

if (i == ind){
setState() {
textcol[i] = Colors.white;
icol[i] = Colors.white;
tilecol[i] = const Color.fromRGBO(75, 73, 172,
1);
});
}else{
setState() {
textcol[i] = Colors.black54;
icol[i] = Colors.black54;
tilecol[i] = Colors.transparent;
});
}
}
}

void Iconchange(){

if (isopen){
setState() {
_icon = Icons.keyboard_arrow_down;
});
}else{
setState() {
_icon = Icons.keyboard_arrow_right;
});
}
}

@override
void initState() {
userconfig();
int ind = globals.side_selected;
for( var i = 0; i<13; i++){
if (i == ind){
setState() {
textcol[i] = Colors.white;

```

```

        icol[i] = Colors.white;
        tilecol[i] = const Color.fromRGBO(75, 73, 172,
1);
    });
    }else{
        setState(() {
            textcol[i] = Colors.black54;
            icol[i] = Colors.black54;
            tilecol[i] = Colors.transparent;
        });
    }
}
super.initState();
}

```

```

@override
Widget build(BuildContext context) {
    return Drawer(
        child: SingleChildScrollView(
            child: Column(
                crossAxisAlignment:
CrossAxisAlignment.stretch,
                children: [
                    drawerheader(context),
                    draweritems(context)
                ],
            ),
        ),
    );
}

```

```

Widget drawerheader (BuildContext context) =>
Container(
    padding: EdgeInsets.only(top:
MediaQuery.of(context).padding.top+30, bottom:
20),
    child: Image.asset('assets/logolong.png', height:
50,),
);

```

```

Widget draweritems (BuildContext context) =>
Wrap(
    runSpacing: 10,
    children: [
        Visibility(
            visible: v_dashboard,
            child: ListTile(
                leading: Icon(Icons.dashboard_outlined,
color: icol[0],),

```

```

        tileColor: tilecol[0],
        title: Text('Dashboard', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[0],),),
        onTap: (){
            colorchange(0);
            Navigator.pop(context);
            if(is_online){
                Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
Homepage_online()));
            }else{
                Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
Homepage()));
            }
        }
    }
}

```

```

        globals.side_selected = 0;
    },
),
),
Visibility(
    visible: v_map,
    child: ListTile(
        leading: Icon(Icons.map_outlined, color: icol
[1],),
        tileColor: tilecol[1],
        title: Text('Map', style: TextStyle(fontWeight:
FontWeight.w600,color: textcol[1])),
        onTap: (){
            colorchange(1);
            Navigator.pop(context);
            Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
Mappage()));
            globals.side_selected = 1;
        },
    ),
),
Visibility(
    visible: v_4ps_info,
    child: ListTile(
        leading: Icon(Icons.table_chart_outlined,
color: icol [2]),
        trailing: Icon(_icon, color: icol[2],),
        tileColor: tilecol[2],
        title: Text('4Ps Information', style:
TextStyle(fontWeight: FontWeight.w600,color:

```

```

textcol[2])),
  onTap: (){
    colorchange(2);
    globals.ismenuopen = !globals.ismenuopen;
    // viewmenu = !viewmenu;
    isopen = globals.ismenuopen;
    Iconchange();
  },
),
),

Visibility(
  visible: globals.ismenuopen,
  child: Padding(
    padding: const EdgeInsets.only(left: 70.0),
    child: Column(
      children: [
        // Visibility(
        //   visible: false,
        //   child: ListTile(
        //     tileColor: tilecol[9],
        //     title: Text('Add New Applicant', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[9])),
        //   onTap: (){
        //     colorchange(9);
        //     Navigator.pop(context);
        //     Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=>
Add_applicant2()));
        //     globals.side_selected = 9;
        //   },
        // ),
        // ),
        Visibility(
          visible: v_field,
          child: ListTile(
            tileColor: tilecol[9],
            title: Text('Add Applicant', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[9])),
            onTap: (){
              colorchange(9);

              if (is_online){
                Navigator.pop(context);
                Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
add_applicant_online()));

```

```

globals.side_selected = 9;
}else{
  Navigator.pop(context);
  Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
add_applicant()));
  globals.side_selected = 9;
}

},
),
),
  Visibility(
    visible: v_approved,
    child: ListTile(
      tileColor: tilecol[10],
      title: Text('4p\'s Local Data', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[10])),
      onTap: (){
        colorchange(10);
        Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) => const
offlineData()));
        globals.side_selected = 10;
        // Navigator.push(context,
MaterialPageRoute(builder: (context)=>
Add_applicant2()));
      },
    ),
    ),
    Visibility(
      visible: is_online,
      child: ListTile(
        tileColor: tilecol[3],
        title: Text('4p\'s Online Data', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[3])),
        onTap: (){
          colorchange(3);
          Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) => const
onlineData()));
          globals.side_selected = 3;
          // Navigator.push(context,
MaterialPageRoute(builder: (context)=>
Add_applicant2()));
        },
      ),
    ),

```

```

    ),
    Visibility(
      visible: v_updateatlong,
      child: ListTile(
        tileColor: tilecol[5],
        title: Text('Update Applicant Location',
style: TextStyle(fontWeight:
FontWeight.w600,color: textcol[5])),
        onTap: (){
          colorchange(5);
          Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) => const
latlongget()));
          globals.side_selected = 5;
          // Navigator.push(context,
MaterialPageRoute(builder: (context)=>
Add_applicant2()));
        },
      ),
    ),
  ],
),
)),
Visibility(
  visible: v_announce,
  child: ListTile(
    leading: Icon(Icons.announcement_outlined,
color: icol [4]),
    tileColor: tilecol[4],
    title: Text('Announcement', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[4])),
    onTap: (){
      colorchange(4);
      globals.side_selected = 4;
      Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
Announcement()));
    },
  ),
),
Visibility(
  visible: v_userdashboard,
  child: ListTile(
    leading: const
Icon(Icons.dashboard_outlined),
    tileColor: tilecol[11],
    title: Text('Dashboard', style:
TextStyle(fontWeight: FontWeight.w600,color:

```

```

textcol[11])),
    onTap: (){
      colorchange(11);
      Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) => const
benifeciary_dashboard()));
      globals.side_selected = 11;
      // Navigator.push(context,
MaterialPageRoute(builder: (context)=>
Add_applicant2()));
    },
  ),
  Visibility(
    visible: v_user_profile,
    child: ListTile(
      leading: const
Icon(CupertinoIcons.profile_circled),
      tileColor: tilecol[12],
      title: Text('My Profile', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[12])),
      onTap: (){
        colorchange(12);
        Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) => const
benifeciary_profile()));
        globals.side_selected = 12;
        // Navigator.push(context,
MaterialPageRoute(builder: (context)=>
Add_applicant2()));
      },
    ),
    ),
    Divider(color: Colors.grey[400],thickness: 2,
indent: 20, endIndent: 20,),

    // Visibility(
    //   visible: v_user,
    //   child: ListTile(
    //     leading: Icon(Icons.person_outline, color:
icol [5]),
    //     tileColor: tilecol[5],
    //     title: Text('System Users', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[5])),
    //     onTap: (){
    //       colorchange(5);
    //       globals.side_selected = 5;

```

```

// Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=>
Sys_user()));
// },
// ),
// ),
Visibility(
  visible: v_log,
  child: ListTile(
    leading: Icon(Icons.sync, color: icol [6]),
    tileColor: tilecol[6],
    title: Text('Sync Data', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[6])),
    onTap: (){
      colorchange(6);
      Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
Syncdata()));
      globals.side_selected = 6;
    },
  ),
),
Visibility(
  visible: v_supp,
  child: ListTile(
    leading:
Icon(Icons.medical_services_outlined, color: icol
[7]),
    tileColor: tilecol[7],
    title: Text('Support', style:
TextStyle(fontWeight: FontWeight.w600,color:
textcol[7])),
    onTap: (){
      colorchange(7);

```

```
import 'package:flutter/material.dart';
```

```

import
'../Database/online_announcements.dart';
import '../funtions/Header.dart';
import '../funtions/anouncements_pages.dart';
import '../funtions/sidemenu.dart';
import '../funtions/textwidget.dart';
import '../config/variables.dart' as globals;

```

```

Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
User_supp()));
      globals.side_selected = 7;
    },
  ),
),
Visibility(
  visible: v_about,
  child: ListTile(
    leading: Icon(Icons.info_outline, color: icol
[8]),
    tileColor: tilecol[8],
    title: Row(
      children: [
        Text('About', style: TextStyle(fontWeight:
FontWeight.w600,color: textcol[8])),
        const Text(' e4Ps Map', style:
TextStyle(color: Color.fromRGBO(251, 171, 80,
1),fontWeight: FontWeight.w600)),
      ],
    ),
    onTap: (){
      colorchange(8);
      Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context)=> const
app_about()));
      globals.side_selected = 8;
    },
  ),
),
],
);
}

```

```

class beneficiary_dashboard extends
StatefulWidget {
  const beneficiary_dashboard({Key? key}) :
super(key: key);

  @override
  State<beneficiary_dashboard> createState() =>
_beneficiary_dashboardState();

```

```

}

class _beneficiary_dashboardState extends
State<beneficiary_dashboard> {
  String name = 'Jane doe';
  String status = 'Active';
  var announcementlist;

  Future loadannouncement()async{
    setState(() {
      announcementlist = [];
    });
    var announcements = await
getannouncements();
    setState(() {
      announcementlist = announcements;
    });
  }

  loadalldata(){
    setState(() {
      name =
"${globals.user_data[0]['e4ps_profile_lname']},
${globals.user_data[0]['e4ps_profile_fname']}";
      status =
globals.user_data[0]['e4ps_profile_status'].toString
();
    });
  }

  @override
  void initState() {
    // TODO: implement initState
    loadalldata();
    loadannouncement();
    super.initState();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: top_header(context),
      drawer: const Navigationmenu(),
      body: Container(
        padding: const EdgeInsets.all(10),
        child: SingleChildScrollView(
          child: Column(

```

```

      mainAxisAlignment:
MainAxisAlignment.start,
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: [
        txt_title('Welcome $name'),
        const SizedBox(height: 5,),
        Row(
          children: [
            const Text('Member Status:'),
            const SizedBox(width: 10,),
            txt_status(status),
          ],
        ),
        const SizedBox(height: 20,),
        Container(
          width: double.infinity,
          margin: const
EdgeInsets.symmetric(horizontal: 10),
          padding: const EdgeInsets.all(10),
          decoration: BoxDecoration(color:
Colors.white,borderRadius:
BorderRadius.circular(10), boxShadow: [
            BoxShadow(
              color: Colors.grey.withOpacity(0.5),
              spreadRadius: 3,
              blurRadius: 2,
              offset: const Offset(0,3)
            )
          ]),
          child: Column(
            crossAxisAlignment:
CrossAxisAlignment.start,
            children: [
              const Row(
                children: [
                  Icon(Icons.announcement, color:
Color.fromRGBO(71, 71, 161, 1)),
                  SizedBox(width: 10,),
                  Text("Announcements", style:
TextStyle(fontFamily: 'nunito', fontSize: 20)),
                ],
              ),
              const SizedBox(height: 10,),
              widget_announce(announcementlist,
context)
            ],

```

),
,
l,
,
,

),
);
}
}

E4PSMAP: A MOBILE APPLICATION FOR 4PS BENEFICIARIES FOR THE CITY OF DIGOS WITH GEO TAGGING TECHNOLOGY

An Undergraduate Capstone Project
Presented to the Faculty of the Department of Technical Programs
UM Digos College
Digos City

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Information Technology

by

Ralph Ian O. Noynay
Rhona Mae Magsayo
Jonalyn E. Calvo

May 2023

APPROVAL SHEET

This capstone project entitled “**e4PsMap: A Mobile Application for 4Ps Beneficiaries for the City of Digos with Geo Tagging Technology**” prepared and submitted by **Ralph Ian O. Noynay, Rhona Mae Magsayo and Jonalyn E. Calvo**, in partial fulfillment of the requirement for the degree of **Bachelor of Science in Information Technology** has been examined and is hereby recommended for approval and acceptance.

JOHN RAVEN V. MANULAT, MIT
Research Adviser

PANEL OF EXAMINERS

APPROVED by the committee for Final Defense with the grade of _____.

EDUARD L. PULVERA, MSIS
Chair

SHANE RYAN A. DIAZ, MIT
Member

JOANE MAY B. DELIMA, MIT
Member

ACCEPTED in partial fulfillment for the degree of Bachelor of Science in Information Technology.

CONRADO B. PANERIO JR., MAT
Research Coordinator

CYVIL DAVE T. DASARGO, MIT
Program Chair, BSIT

EDUARD PULVERA, MSIS
Dean of College

TESSIE G. MIRALLES, Ph.D.
VP-Branch Operations

ACKNOWLEDGEMENT

We want to express our gratitude to those who have contributed to completing this capstone project. With their support and guidance, this project was possible.

First and foremost, to our family, friends, and loved ones who are there to support and motivate us in completing and doing this capstone project. Their constant encouragement and belief in our abilities have driven us to overcome the hardships we encountered during this research.

To Sir **John Raven V. Manulat, MIT**, our research adviser, for sharing their knowledge and expertise through the development of this research project. Also, for the advice and encouragement that helps us to fulfill and complete this project.

To the panel members, **Sir Cyvil Dave T. Dasargo, MIT, Sir Conrado B. Panerio Jr., MAT, Sir Shane Ryan A. Diaz, MIT, Sir Eduard L. Pulvera, MSIS, and Ms. Joane May B. Delima, MIT**, who have shared all their critics and valuable suggestions that will help us improve this project.

Above all, to God Almighty, the creator of knowledge and wisdom, who guides us and gives us strength to move forward. We have extensively recognized the divine blessings and providence that have sustained us.

Researchers

DEDICATION

This capstone project is dedicated to everyone who is there to support and inspire us in our academic endeavors. To our family for their unwavering love and sacrifices to meet our needs during the research. To our friends who give us motivation and encouragement. We also dedicate this to ourselves; for our hard work and unwavering dedication to finishing this capstone project. Above all, to God Almighty, for giving us the wisdom, knowledge, guidance, and strength to make this capstone project possible. God bless us all.

Ralph Ian O. Noynay

Rona Mae Magsayo

Jonalyn E. Calvo

TABLE OF CONTENTS

	Page
TITLE PAGE	i
APPROVAL SHEET	ii
ACKNOWLEDGMENT	iii
DEDICATION	iv
TABLE OF CONTENTS	v
LIST OF TABLES	vii
LIST OF FIGURES	viii
ABSTRACT	ix
 INTRODUCTION	
Project Context	1
Purpose and Description	2
Objectives of the Study	2
Scope and Limitation	2
 REVIEW OF RELATED LITERATURE/SYSTEM	
Related Literature	2
Related System	3
 TECHNICAL BACKGROUND	
Conceptual Framework	6
Algorithms	6
Project Technology	7
Software Tools in Application Development	8
 METHODOLOGY	
Requirement Analysis	8
Requirement Documentation	9
System Architecture	10
Development Approach	10

RESULTS AND DISCUSSIONS

Implementation Plan	14
Implementation Result	14

CONCLUSION AND RECOMMENDATIONS

Conclusion	16
Recommendation	16

REFERENCES	17
-------------------	-----------

APPENDICES

A	Test Cases	20
B	Source Code	42
C	Certificate of Permission	60
D	Routing Form	62
E	Assignment of Research Personnel Form	63
F	Grammarly Report	64
G	Plagiarism Result	65
H	Curriculum Vitae	68

LIST OF TABLES

Table		Page
1	Features comparison of different related system	5
2	Hardware Requirements	10
3	Software Requirements	10
4	Range of Mean and Interpretation	12
5	BHW Frequency Distribution	13
6	Implementation Plan	14

LIST OF FIGURES

Figure		Page
1	Conceptual Framework	6
2	Retrieve Data by Barangay	7
3	Receiving JSON Data	7
4	Heatmap Renderer	7
5	Quartic Kernel Function	7
6	Data Flow Diagram	8
7	System Architecture	10
8	Use Case Diagram	11
9	Location Obtained by the e4psMap App	12
10	Current Location base on Google Map	12
11	Results of the conduct of testing for users	13
12	Application Form	14
13	Saved Data	14
14	Offline Login	15
15	Offline Data to Online Sync	15
16	Table of Synced Data	15
17	Point Map	16
18	Heatmap Module	16

ABSTRACT

One of the programs created by the government to eliminate poverty is the 4ps (Pantawid Pamilyang Pilipino Program). The e4psMap application is a component system for the primary web system of e4psmap. This application will be a valuable tool for the BHW (Barangay Health Workers) to gather information about the 4ps beneficiaries around Digos City. The e4psmap application will allow BHWs to record 4ps data without carrying paper forms. The application can record the data from the 4ps beneficiaries, such as their family data, location, and other data, to determine their qualification as 4ps beneficiaries. It can also operate offline to ensure that even though beneficiaries are in far-flung areas where internet connectivity is scarce, BHWs can still use the application. As a result of integrating the application into their data-gathering task, the BHWs can perform the job without the hassle of carrying out several forms. The CSWDO will also quickly analyze the data collected to create reports about the 4ps beneficiaries. The application will be their aid in making their work more accessible and more efficient.

Keywords: *GIS, 4p's, E4psMap, Profiling, Geo Tagging.*

EXECUTIVE SUMMARY

The e4psMap application is an Android application developed to aid the BHWs in gathering 4p's beneficiary data per barangay within Digos. The health workers in Digos City still use paper forms for recording, which will manually check by the CSWDO to validate if the beneficiary is qualified for the program. With the help of the created application, health workers can record beneficiary data such as their location information, personal information, and other beneficiary data through their Android smartphones which will help them conduct their field research. Also, to combat the lack of internet connection in some areas, the application can operate offline, then data will be uploaded to the online database when the internet is available. All the data collected by the app will be available on the separate web version of the e4ps map developed by the GODDESS project, where the CSWDO can review all the applicants and validate their qualifications with the aid of a decision support system. The application would be a tool that would make the work of BHW easier and more efficient.

Categories and Subject Descriptors

CCS → Information Systems → Information Systems Applications → Enterprise Information Systems.

General Terms

Documentation, Algorithms, Management.

Keywords

GIS, 4ps, E4psMap, Profiling, Geo Tagging

1. INTRODUCTION

1.1 Project Context

During this era, society has become dependent on technology; every task can be done quickly with the help of technology [1]. With this advancement, data turns into digital data, producing vast data. Instead of using these technologies, some may still rely on empirical-based surveys and other primary data collection methods. With the mentioned methods, collecting the needed data may take much time and effort. The local government continuously works to improve the effectiveness and efficiency of its services by making data-driven decisions. The City of Digos has 26 barangays, and each barangay has a beneficiary of the 4ps (Pantawid Pamilyang Pilipino Program) livelihood program under the DSWD, which provides cash assistance to people experiencing poverty to alleviate their needs. The government's primary program for reducing

poverty (RA 11310 or 4Ps) gives impoverished households conditional cash transfers to improve Filipinos' health, nutrition, and educational conditions [2]. With a database that includes information on 26.51% of the population as of 2021, the program needs a complete tool to tackle poverty to pinpoint recipients of national and international aid for local government programs that are poor or almost poor [3]. In Digos City, 4,296 of the 47,598 households (9%) as of 2022 are beneficiaries of the 4p's program [4]. 4p's offers different programs that include the following beneficiaries, which may consist of cash assistance for education, health, and livelihood programs.

Despite having a list of beneficiaries, a system needs to help them document their lists, and the data collected may need to be complete enough to be used for writing the 4P's program outcomes. Also, when gathering data offline, there is no system to ensure that the information from that process will automatically sync online using the internet. In addition, the collected datasets, which can be a big help for the local government unit in planning and development, need to be updated, stored, analyzed, and visualized for the local government units to create meaningful insights. Every three years, the BHW must update its records. When updating the record, the BHW must validate whether these beneficiaries belong to marginalized individuals or live in Digos City. Those volumes of papers piled up in their cabinets occupy much space. Furthermore, because there are 26 barangays in remote locations, each BHW needed help locating these beneficiaries to update their report. They must conduct house-to-house surveys, and some beneficiaries may transfer from one place to another.

To fill the gap as to how the Barangay Health Workers could monitor the success or the compliance of the 4Ps program beneficiaries, a system that is capable of storing data about the 4Ps beneficiaries with the features that can help the BHW and CSWDO on monitoring the beneficiaries and also supporting them in their decision making. The e4psMap web system developed by the GODDESS (Good Governance through Data Science and Decision Support System), a DOST (Department of Science and Technology) funded project, is already a system that is capable of handling these features. The e4psMap application is also a component of the web system that can access both online and offline and can collect data and visualize the data by producing geospatial and

graphical presentations about the 4p's beneficiaries in every barangay. Since the application developed is a component system for the web system, it is purposed to enhance and extend the functionality of the web system. The application is not a stand-alone system but a mobile application intended to be integrated seamlessly into the existing web system. The e4psMap system will help support the local government in eliminating poverty.

1.2 Purpose and Description

1.2.1 Purpose

e4psMap is a mobile application that will serve as a monitoring tool by the BHW for the 4ps beneficiaries in Digos City. This system collects needed data to validate the 4Ps' application by capturing the image of their house and their respective locations. The system's mapping functionality will geotag the collected data. The collected data will generate reports to help the local government make decisions for its citizens' welfare.

1.2.2 Description

The mobile application entitled e4psMap: A Mobile Application for 4ps Beneficiaries for the City of Digos with Geo Mapping Technology, which means an online and offline platform that is capable of capturing data from the 4ps beneficiaries such as their information, family detail, and their geologic location. The mapping functionality of the application will use the collected data to plot the beneficiaries. The system is developed for mobile platforms, specifically Android operating systems, over the web since it is more accessible and portable than a website. The mobile application will only be available for smartphones running on the Android OS.

1.3 Objectives

The main objective of this research was to create a system or platform that captures, stores and presents the datasets of beneficiaries of the Pantawid Pamilyang Pilipino Program - 4ps within Digos City. The objective of this study is the following:

- 1.3.1 To create a mobile application system that collects the 4Ps data using the Flutter framework.
- 1.3.2 To integrate offline mode for data gathering and will be synced to the online database using the internet.
- 1.3.3 To geotag the beneficiary's address and generate an ArcGIS heatmap visualization.

1.4 Scope and Limitations

The e4ps map application needs an internet connection to submit local data online since the data required will be hosted online, and it also needs accounts that are verified to use it. The application will help aid BHW to monitor beneficiaries within their barangays.

1.4.1 Scope

This project's scope is that it was made for Android smartphone users. Therefore, it may be accessed using Android-powered devices. This system is also made as a component of a much bigger system, the e4psMap web system created by the GODDESS project, which is a web system that handles all the analytics and most data visualization of the data collected using this application. The system also has the capability of Geotagging, and the visualization of various data required to produce reports for the Digos City local government will all be included in this system. The application can also have the capability to use some of the features offline, and all of the data offline can be synced online and will be saved to the online database.

1.4.2 Limitations

Even though the application is for smartphone technology, it is limited to Android devices and would not support other operating systems. Also, this application will only be available for use in the locality of Digos City, which means that only Digos City LGU and 4Ps beneficiaries from 26 Barangays in Digos City can use this application. Also, since it is a component system of the e4psMap web system, it is highly dependent on the web system, which means that the data and its features will depend on what the web system has.

2. REVIEW RELATED LITERATURE/STUDIES

The state of technology is continuously evolving. Technology advancement has resulted in innovations and development for mobile applications that affect our daily lives, particularly in monitoring beneficiaries. Thus, this chapter presents the relevant literature that considers strengthening to serve as the foundation for this study.

2.1 Reviews of Related Literature/Studies

2.1.1 The Community-Based Monitoring System (CBMS) as a Local Planning Tool

The Community-Based Monitoring System is a data collection system designed to methodically process and integrate local government data to monitor both the microeconomic and macroeconomic effects of impacts [5]. This

study aims to fill existing data gaps in diagnosing the extent of poverty at the local level and help determine the causes of poverty, develop appropriate policies and programs, identify eligible beneficiaries, and assess the impact of policies and programs. It also promotes decentralization by enabling local governments to collect, analyze, and apply data for local planning and program implementation.

2.1.2 The Use of Geographic Information System Based Spatial Decision Support System in Developing the Urban Planning Process

This study demonstrates the importance of developing an appropriate mechanism to support decision-making based on a diverse set of data inputs to ensure the quality of the decision made [6]. However, it investigates how to meet the need for an urban and regional planning tool based on information technologies, which have posed a significant challenge to how things are done, accelerating decision-making and improving the planning process.

2.1.3 Use and Potential of Geographic Information System for Health Mapping in Eastern Mediterranean Region

This article's stated objectives are to raise awareness of the value and potential role of health mapping; to present a situation analysis on the use of health mapping in the region's countries; to identify constraints to the region's further development of health mapping; and to provide recommendations to countries on how to strengthen the development, maintenance, and utilization of health maps for planning and decision-making [7].

2.1.4 Strengthening Urban Community Governance through Geographical Information System and Participation

This paper assesses interactive technology's role in improving urban governance to meet social needs by strengthening local organizations' capacities, improving client access, service effectiveness and responsiveness, and community building [8]. According to their findings, residents find the interactive map helpful, especially in improving service access, whereas local organizations are more hesitant to adopt such technology due to high maintenance costs. To identify sustainable financing (via crowdsourcing) for GIS applications to improve urban community governance, the study's implications include the need to create incentives to encourage technology adoption, improve the capacity of local organizations, and establish better links

between community organizations and residents.

2.1.5 Geographic Information System for Mapping Public Service Location

This article aims to study, design, and create geographic information system-based public service location mapping systems (GIS) [9]. GIS technology is a helpful information system used to develop information systems for accurately mapping locations and areas, making it more straightforward for people to get information on how public services are distributed and making it easier for them to go to the spot. The data used comes from both third-party sources and field observations.

2.1.6 Using Spatial Analysis and GIS to Improve Planning and Resource Allocation in a Rural District of Bangladesh

Using geographic information systems (GIS) for public health is relatively common in Bangladesh. On the other hand, local-level managers' use of GIS for planning, monitoring, and decision-making needs to be better documented [10]. This study examined how well local government health administrators used maps with spatial data to plan, allocate resources, and monitor programs. Using GIS maps, funding was prioritized by clusters of disadvantaged communities and underserved unions (the lowest administrative units). The associations with the most deficient performance received additional funding from local government budgets to improve the readiness and supervision of health facility services.

2.1.7 Roles of Barangay Health Workers

Barangay health workers are the primary providers of basic healthcare in the barangay. They play an essential role in realizing the direct healthcare approach to health empowerment by providing affordable and appealing health services at the barangay level [11]. Barangay health workers assist health professionals by monitoring community members' health status, maintaining records of health activities, and ensuring the upkeep of barangay health centers and stations, including the safe custody of equipment, supplies, and health records.

2.2 Related System

These software/systems each have unique features that enable them to present all the data needed via an internet or Wi-Fi connection.

2.2.1 Pantawid Pamilyang Pilipino Program 4p's: Profiling System

The Pantawid Pamilyang Pilipino Program 4p's: Profiling System allows its users to help verify the status of each 4p beneficiary to see if they are meeting the conditions established by the DSWD to qualify for conditional cash awards [12]. Through this application, users can update beneficiaries' information and provide profiles, allowing users to track each beneficiary's progress through the program or receipt of financial grants. This study uses a website platform to improve the 4ps program by decreasing the number of "ghost beneficiaries" and letting the DSWD effectively keep track of its recipients.

Compared to the e4psMap application, it doesn't have a mobile version. Furthermore, the Pantawid Pamilyang Pilipino Program Profiling System is solely for profiling. It has no unique features that e4psMap offers, such as GIS visualization, that can generate insights. Since the system is web-based, it purely uses the internet to process transactions. It doesn't work offline; unlike e4psMap, offline transactions, such as gathering data, will sync online once it has internet access.

2.2.2 Web-Based Management System for Municipal Social Welfare and Development Office of Malalag with GIS Mapping and Decision Support using Decision Matrix Analysis

This study provides an accurate tool that will be used by social workers of the Municipal Social Welfare and Development Office to classify a large number of families; for planning, the possible outcome of using this tool along with the web-based management system will serve as the basis for planning of possible interventions by the said office or other concerned government units. For monitoring, social workers and enumerators can use the developed system to determine whether the identified intervention has improved the family concerned.

Similarly, this system is built on the same platform as e4psMap applications, such as mobile and web-based versions. The difference between the related system and the e4psMap application is that the mobile version of the system can't be used for offline transactions. Since the related system is a management system, it focuses only on classifying and monitoring many families to determine whether the identified action has helped the affected family. It also uses GIS technology, unlike the e4psMap application, which can generate a

heatmap showing beneficiaries' density in a particular area.

2.2.3 GIS Based Student Profiling System Basis for Marketing Predictive Analytics

This application platform was designed and developed as a GIS-based Student Profiling System to help the Cantilan Campus of Surigao del Sur State University visualize data representation using GIS mapping [13]. It visualizes the demographic distribution of the students' locations in specific municipalities and barangays. Also, the population and area of currently enrolled students in all programs offered can be viewed, and the student population from public and private school feeders can be observed by integrating data charts into the system. This information could be used in the university's intelligent recruitment scheme, marketing strategies, planning, proposed development programs, and monitoring.

Compared with the e4psMap application, which deals with the 4p's beneficiary profiling, the GIS-Based Student Profiling System, on the other hand, deals with the profiling of the students. Also, the system is only available on the web, while the e4psMap, on the other hand, is a system that is available both on the web and mobile. In the GIS-Based Student Profiling, they also have integrated the GIS technology, which lets them plot all the students in the map, which is presented with a point, the same with the e4psMap application; the difference is that aside from plotting beneficiaries to the map, the application is also able to produce a heatmap that visualizes the density of the beneficiaries in a particular area.

2.2.4 Barangay Profiling System with Analytics

Another related system is the Barangay Profiling System with Analytics for Barangay Pallua Sur in Tuguegarao City, which helps to automate the profiles of all barangay residents, stores relevant information, and can be updated as needed [14]. The platform also allows users to enter barangay complaints, which are saved in a database. One of the system's additional features is to generate barangay clearance required by residents for whatever purpose it may serve. This system can generate statistical records such as the barangay's total population, total registered voters, reported violations of the law, and total employed and unemployed citizens.

Like the e4psMap application, the Barangay Profiling System with Analytics can collect data

on the members under the barangay and use the data collected for analytics. The difference between the said system and the e4psMap application is the GIS functionality since the system cannot visualize the data collected on a map. Also, the difference between the related system and the e4psMap application is that the related system can release credentials such as barangay clearance derived from the profiles being saved in the system.

2.2.5 Rural Health Unit Decision Support System with Mapping

This study developed a web-based application for easy access to data and a Rural Health Unit Decision Support System (RHUOSS) with mapping to serve as an efficient record management system for the RHU of the Municipality of Solano, Nueva Vizcaya [15]. It also supports the unit's record management regarding patient profiling, record-keeping, and retrieval. This platform also generates reports in visualized forms that will aid the physician in his decision-making activities on better patient outcomes.

Unlike the e4psMap application, which deals with the 4P's beneficiary profiling, the Rural Health Unit Decision Support System with Mapping deals with patient profiling and is available only as a desktop application. The Rural Health Unit Decision Support System with Mapping mainly features disease occurrence mapping to assist physicians and other health professionals in the unit in their decision-making tasks, particularly in diagnosis, treatment, and recommendations. At the same time, the e4psMap application lets them plot all the beneficiaries on the map, which is presented with a point.

2.2.6 Development of an Information-Based Dashboard: Automation of Barangay Information Profiling System (BIPS) for Decision Support towards eGovernance

This platform aims to assist barangay officials in utilizing household profiling data by creating an aggregate form that can be visualized on the system dashboard [16]. Barangay officials can use the information as the foundation for budgeting and supporting documents to generate other ways to improve the quality of life and well-being of the governed community. This study uses the Barangay Information Profiling System to assist barangay officials in budget allocation and decision-making in their respective governed areas (BIPS).

As for the e4psMap application, the Development of an Information-Based Dashboard: Automation of Barangay Information Profiling System (BIPS) for Decision Support towards eGovernance can collect data on the barangay members. The difference between the related system and the e4psMap application is that it doesn't have a mobile version and doesn't use GIS technology. This system is only initiated with the idea that educational institutions around the barangay can help in the areas included and focuses on budget allocation and decision-making in their respective governed areas.

Legend:

- A. Pantawid Pamilyang Pilipino Program 4p's: Profiling System
- B. Web - Based Management System for MSWDO of Malalag with GIS Mapping
- C. GIS Based Student Profiling System Basis for Marketing Predictive Analytics
- D. Barangay Profiling System with Analytics
- E. Rural Health Unit Decision Support System with Mapping
- F. Development of an Information-Based Dashboard
- G. e4psMap: A mobile application for 4P's

Features	A	B	C	D	E	F	G
GIS Visualization		✓	✓				✓
Beneficiary Profiling	✓	✓	✓	✓	✓	✓	✓
Generate Insights		✓				✓	✓
Update beneficiary information	✓	✓		✓		✓	✓

Check for beneficiary status	✓	✓			✓	✓	✓
Reports generated by predictive statistics			✓				✓
Save and retrieve data offline when there is no internet and sync offline to online data when internet is available.							✓

Table 1. Features comparison of different related system

3. TECHNICAL BACKGROUND

Within this chapter, researchers have presented the different technology and technicalities used in the research. This will include the various APIs, Algorithms, and other technologies used to complete and achieve the desired research objectives.

3.1 Conceptual Framework

The BHW users will use the application.

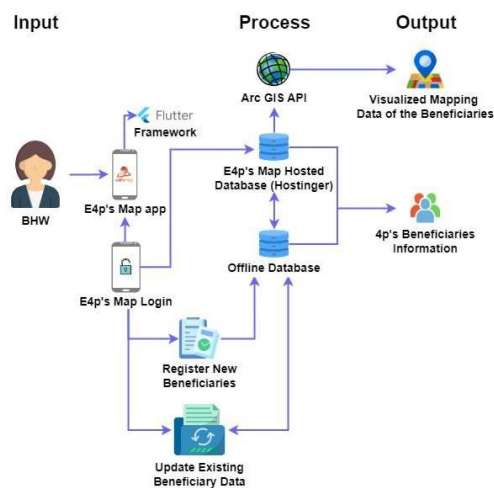


Figure 1: Conceptual Framework

Figure 1 shows the conceptual framework of the system, where the Barangay Health Worker

(BHW) can interact with the mobile system. The BHW can input different data in the system, including the information of the 4p's beneficiaries, such as their address, personal information, house members' information, and pregnant member information. The BHW can also update existing information for their yearly update on the 4p's beneficiaries. All the data collected will be stored in an offline database with no Internet, allowing the application to be used without any Internet connection. All data stored in the local database will be synced to the online hosted database when the internet is available. The online database will also be used in the web version of the system, which the CSW will also access. Data stored in the online database, such as the current location of the beneficiaries, will be used to implement the geotagging feature of the application using ArcGIS API. This lets the application visualize and plot beneficiaries' locations on the map.

3.2 Algorithms

Algorithms are mathematical functions and procedures that help solve problems. Several algorithms are used in the development of the system. These algorithms are used to generate meaningful insights from the data collected by the system.

3.2.1 Clustering Algorithm

The clustering algorithm is an algorithm that groups data into different clusters. The clustering algorithm is a process in which various data are grouped based on similarities. The algorithm lets the system group different data together and generates meaningful insights through graphical representation such as graphs and the GIS technology integrated with the system. One clustering type is density clustering; this clustering algorithm measures the density of a region with similar data. The density algorithm is used to measure the density of beneficiaries within a certain barangay; through it, the system can visualize what barangay has the most 4ps beneficiaries on the map.

$$\rho = \frac{1}{(r)^2} \sum_{i=1}^n \left[\frac{3}{\pi} * W_i \left(1 - \left(\frac{d_i}{r} \right)^2 \right)^2 \right]$$

The formula stated above is used to calculate and visualize the data on a heatmap using the ArcGIS map. The quartic kernel function has been used to visualize the data points in a heatmap, a kernel density estimation formula. The ρ represents the estimated density on the (x,y) location, r is the search radius or

bandwidth, i represents the number of points from 1 to n number of points within the search radius of x and y , W_i is the weighted field, but for the heatmap of the application, points have the same weight which is 1, and lastly is d_i which represents the distance between me and the (x,y) location [17].

```

$query = "SELECT
e4ps_profile_address.e4ps_address_lat,
e4ps_profile_address.e4ps_address_long,
e4ps_profile_address.e4ps_address_id,
FROM e4ps_profile_address WHERE e4ps_profile_address.e4ps_address_brgy = '$brgy'";

while ($results = $statement -> fetch()){
    array_push(
        $myarray, array(
            "e4ps_address_lat" => $results['e4ps_address_lat'],
            "e4ps_address_long" => $results['e4ps_address_long'],
            "e4ps_address_id" => $results['e4ps_address_id'],
        ));
}
echo json_encode($myarray);

```

Figure 2: Retrieve data by barangay

To supply the data for the formula, the system will retrieve all points with criteria; in Figure 2, it is only points based on a particular barangay. All data returned by the query will be put into an array of objects and returned as a JSON.

```

var features = data.map((point) => {
    return {
        geometry: {
            type: "point",
            longitude: point.e4ps_address_long,
            latitude: point.e4ps_address_lat,
        },
        attributes: {
            ObjectID: point.e4ps_address_id,
        },
    };
});

```

Figure 3: Receiving JSON data

Figure 3 shows the JavaScript code that will be a feature map collection that will hold all the location data from the JSON data returned from the query in Figure 2.

```

var heatmapRenderer = new HeatmapRenderer({
    blurRadius: 10,
    minPixelIntensity: 0,
    maxPixelIntensity: 20,
    field: null,
    rendererCreator: function (params, createRenderer) {
        var distances = features.map(function (feature) {
            return geometryEngine.distance(params.location, feature.geometry);
        });

        var weights = distances.map(function (distance) {
            return quarticKernelWeight(distance, searchRadius);
        });
        params.pixelData = weights;

        return createRenderer(params);
    },
});

```

Figure 4: Heatmap Renderer

Figure 4 shows the "heatmapRenderer" variable, which will handle all the rendering for the heatmap visualization of the data collected by the features variable in Figure 3. The code includes the configuration of the heatmap

representation of the data and the calculation for the data to be rendered using the quartic kernel function.

```

function quarticKernelWeight(distance, searchRadius) {
    var u = distance / searchRadius;
    var quarticWeight = (3 / Math.PI) * (1 - Math.pow(u, 2)) ** 2;
    return quarticWeight;
}

```

Figure 5: Quartic Kernel Function

Figure 5 presents the quartic kernel function, which will calculate the distance between the points and the radius provided and return the weight between the points provided based on the quartic kernel function. The resulting weight will represent the distribution between points and an overall density estimation within the search radius of the points.

3.3 Project Technology

Several technologies are used in this study. These technologies are used to accomplish the desired outcomes of the study.

3.3.1 ArcGIS

Data can be created and plotted on the map using the ArcGIS API. It enables programmers to map various markers, pins, polygons, and images to a specific position. The foundation of ArcGIS is a geodatabase, which is a method for storing any spatial datasets. [18]. This API will enable clients to plot their location based on where they live. When they apply as 4p's, they are needed to set their locations. Locations plotted by the different beneficiaries can be viewed, and through it, admins may create reports based on the plotted data.

3.3.2 Flutter

Flutter is a mobile UI framework created by Google. It is a free and open-source mobile framework. This framework lets developers build mobile applications capable of running on different platforms, such as Android, IOS, Web, and Linux. The Flutter framework uses Dart as its programming language based on JavaScript. This framework allows developers to create different platform applications based on one programming language and code base.

3.3.3 Hosted Database (Hostinger)

To access the database for this application, the researchers use Hostinger as the web host. Through Hostinger, researchers can store and retrieve data needed in the application. The researchers can create online databases that will be accessible anywhere and everywhere.

3.3.4 Local Database (SQLite)

The application uses SQLite as a way to store data locally. The SQLite database enables the

application to store data without online access and sync it when an Internet connection is available. The database table will be the same as the online database hosted by the hosting. Through SQLite, the application will save its database file locally, which will be stored in its directory; without an internet connection, it prevents the system from accessing online data, and the locally saved data can be used instead. This also enables users to save data without any internet connection, which will be a significant advantage in the system's portability.

3.3.5 Programming Language

Dart is the primary programming language used in the application, which is used to create the functionality and the overall application. The dart language is the primary language that will be operated using the Flutter framework. The researchers also use PHP as the scripting language for accessing the database in the shared web host (Hostinger).

3.4 Software Tools in Application Development

The following software is used in the development of the application. This software is used to create and edit codes and the design (UI and UX) of the application.

3.4.1 Android Studio

Android Studio is an IDE for creating Android applications. Android Studio IDE is used since the system is for mobile users, specifically Android users. The researchers can program the application using the Flutter framework through the Android studio. Android Studio is the main software used to accomplish and develop the application since the android studio is an IDE capable of developing applications, and it also supports the Flutter framework, which is the framework being used in the development of the project.

3.4.2 VS Code

VS Code (Visual Studio Code) is a source code editor where researchers can create and edit PHP codes for service-side programming. This software is used to edit code files written in PHP that are essential for accessing databases in the web host. The researchers have used vs. code to edit server-side programs that let the mobile application communicate with the hosted database. Through VS Code, researchers can create PHP files that can be used as an API to connect the application to the online database hosted in Hostinger.

4. METHODOLOGY

In this chapter, the researchers presented the methods used in developing the application. The SDLC used to create this software is the Agile Methodology, a model with a concept of both iterative and incremental process models. Researchers can focus on the applications' adaptability and the system's objectives with the model used. This chapter's subsections describe the concepts and how the application was implemented. The researchers will undergo different phases to complete the application. The development has been broken down into steps; researchers will cycle through planning, executing, and evaluating.

4.1 Requirement Analysis

The e4p map is an application that can collect 4p's beneficiary data. The data collected will be essential in identifying the different beneficiaries around Digos City. The city consists of 26 barangays, each with a population of 4p's members. The number of members may vary in different barangays.

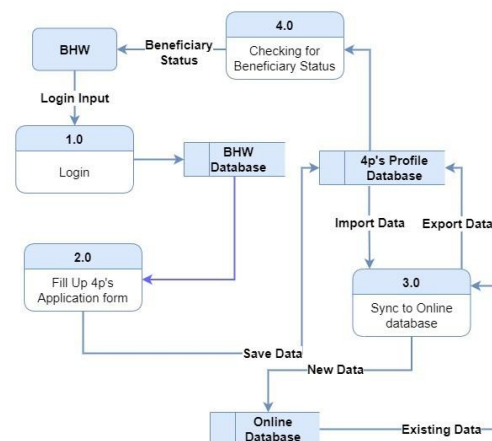


Figure 6: Data Flow Diagram

Figure 6 depicts the system's data flow diagram, showing the main entity (1) BHW that will interact with the system and its data. The BHW will log in to the system to interact with it, and their login details are stored in the database. The BHW can then collect new data or update existing 4p's beneficiary data, the data collected will then be synced to the online database where all data are consolidated, and those synced data will be available. The admin website can view them. Data that only exists in the database will also be imported to the local database if needed. With the data received by the admin, they can now verify and approve those applications; any changes on the status of the beneficiaries will be received by the designated barangay BHW.

4.2 Requirement Documentation

The program's main objective is to assist users in making their tasks more manageable and delivering the high-quality outcomes that the client demands. Functional requirements are the primary means customers convey their needs to the project team, whereas non-functional requirements list system quality attributes.

4.2.1 Functional Requirements

This section includes an explanation of the set requirements and the processes and functions of the system that will be added as modules.

4.2.1.1 Barangay Health Workers (BHW).

The Barangay Health Workers are the ones who will gather information from the 4p's beneficiaries. They will also be provided with graphical analytics that can give them meaningful insights about the current beneficiaries within their barangay.

4.2.1.1.1 Login Module. This module contains the Dashboard UI, where all the primary transactions and information of the application can be found. The number of data points collected per barangays, all barangays in Digos City, and the user's profile will be displayed.

4.2.1.1.2 Dashboard Module. This module contains the Dashboard UI, where all the main transactions and information of the application can be found. The number of data points collected per barangays, all barangays in Digos City, and the User's profile will be displayed.

4.2.1.1.3 Map Module. This module displays each family's information and geographically referenced data in each barangay. The map will show different points where the beneficiary is located. These points can also contain information about the selected point on the map.

4.2.1.1.4 4Ps Information Module. This module has four categories: add applicants, 4P's Local Data, and 4P's Online Data and Update Applicant Location for display. This will contain information about the 4ps beneficiaries. It will display information such as the name, location, and other information a beneficiary may have.

4.2.1.1.4.1 Add Applicant. This module allows the user to add new 4P applicants. This module may use either offline or online mode.

4.2.1.1.4.3 4Ps Local Data. This module displays the 4PS registration data saved on the local device. The data displayed may contain

the 4PS reference code of the beneficiary if the data is uploaded online.

4.2.1.1.4.4 4 P's Online Data. This module displays the applicant data, containing the status of 4P's beneficiaries, full names, contact, and their prospective barangays from the online database.

4.2.1.1.4.5 Update Applicant Location. This module is where the users search the 4PS beneficiaries' reference codes to update their location.

4.2.1.1.6 Sync Data Module. This module is used to sync the data online from offline transactions. Through this module, data saved locally will be synced and imported to the online database, which the website uses where admins can interact with the data provided.

4.2.1.1.7 Announcement Module. This module is where users can announce important updates about meetings, releasing funds, or the status of 4P's beneficiaries.

4.2.1.1.8 Support Module. This module allows users to ask for help, view FAQs and Contact Us. This module will contain any contact information that can be used for questions and technical queries. It may be through email or contact number.

4.2.1.1.9 Logout Module. This module allows the user to log out of their account. This lets users log their accounts out to the application. This will lock the application and will return to the login module.

4.2.1.2 4p's Beneficiaries. The 4P's beneficiary is one of the users of the system. They can view the current status of their application alongside their information. This will help the beneficiaries easily apply for and avail themselves of the government-provided program.

4.2.1.2.1 Dashboard. This module will display the current status of the beneficiaries and announcements posted by the CSWDO about the updates of the 4Ps.

4.2.1.2.2 Profile. In this module, 4p's beneficiaries can view their information, such as their household members, current location, qualifications, and other information.

4.2.2 Non - Functional Requirements

Non-functional requirements define criteria or activities as opposed to specific system

behaviors. This method is used to classify software quality characteristics; it describes how well or to what standard a function should be supplied to measure system quality, efficiency, and successful delivery.

4.2.3 Hardware and Software Requirements

Tables 2 and 3 state the application's minimum hardware and software requirements. When the minimum requirements are met, users will be able to experience the optimal performance of the application.

Hardware	Requirement
Processor	Dual Core, 1.5 GHZ or higher
RAM	4gb or higher
Storage/Memory	100mb or higher
Wifi	Yes
Cellular Network	4G or 5G capable

Table 2. Hardware Requirements

Software	Requirement
Operating System	Android
OS Version	Android 8 or higher

Table 3. Software Requirements

4.3 System Architecture

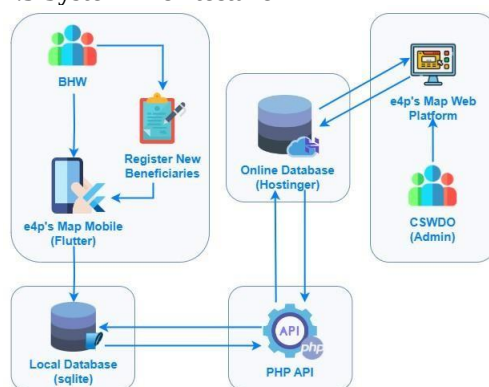


Figure 7: System Architecture

Figure 7 illustrates the system's architecture, which shows the visual representation of the system, how it works, and how it behaves. In The application, there is only one user, which is the BHW; they are the ones who can interact with the application. This means that the BHW

can either add a new 4p application or view existing 4p's data. The application is made through the use of the Flutter framework. All the collected data will be saved in the application's local database, making the data accessible through offline interaction. Data saved in the local database will only be accessible by the same device where it was saved. To make the data accessible to all users, BHW needs to sync it to the online database hosted through Hostinger. Syncing the data online means that they need an internet connection to do it. PHP API will be the bridge between the application and the online database to interact; the API will carry on the request to insert new data sets into the online database. All data in the online database will be viewed and accessible to the online use of the app and the online website of e4p's map, which the CSWDO manages.

4.4 Developmental Approach

The application for this project was developed using the Agile Methodology. The researchers chose this developmental approach because it is compatible with and in line with the project's aim and purpose and produces a high-quality output that the users expected [19]. This is a project management style that divides a project into phases. This method begins by defining the users and developing a vision statement for the issues, opportunities, and values that will be addressed. It demands constant user interaction as well as development at every stage. This section emphasizes regular client involvement, prompt evaluations, ongoing preparation and testing, and close collaboration to produce working software quickly.

4.4.1 Planning

The steps required to validate the identified problem are data gathering via surveys and end-user interviews. The proponent gives a survey to the beneficiaries of 4ps in the City of Digos through the use of the application and survey questionnaire. Beneficiaries of 4ps in the City of Digos, regardless of their employment position (BHW), are one of the app's target end users, known as BHW Users. During the planning phase, the obtained data from the BHW User will be examined, and the result will lead to the development of a prototype for the BHW's Graphical Interface. Furthermore, an interview with a representative from 4ps beneficiaries is conducted, as it is the foundation for the process and procedure that will be implemented in the app. These include the resources to be used, the timeline, and the expense of implementing the software and procedures, be taken into account, and most

importantly, the objectives to be fulfilled. The main aim is to ascertain the requirements, how their service transactions work, and the procedure must be automated. The researcher went on through a series of discussions and consultations in an iterative method that solicits user input frequently. As a result, the information and data have been acquired to create and complete the system.

4.4.2 Design

After the planning stage is complete, the designing stage comes next. The design process involves several steps: comprehending, researching, sketching, designing, prototyping, Testing, and refining. Beginning with the first stage and moving on to the last, iterative design starts with the initial step. According to the needs of the users and the system, the output is subsequently scrutinized and subjected to several iterations for further optimization. This method aims to improve the design with each iteration rather than create a prototype. In the creation of the application, this strategy is beneficial.

4.4.2.1 Use Case Diagram

Researchers used Use Case Diagram in the system design to demonstrate how a participant might interact with a system.

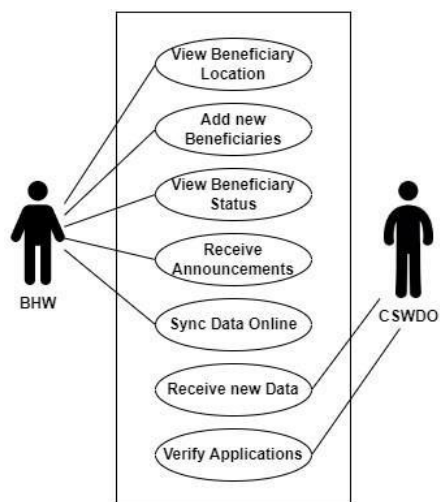


Figure 8: Use Case Diagram

Figure 8 shows the use case diagram of the system, where it shows two actors that can interact with the system. The actors and their possible interactions are the following:

BHW: Create/add new beneficiaries, view beneficiary status, view beneficiary location, receive notifications, and sync data to the online database.

CSWDO: Verify newly registered beneficiaries and receive new data from different barangays.

4.4.3 Coding

Developing the application through coding is the next step in completing the project. The respondent used Dart as the primary programming language for developing this project's mobile application. Researchers also use PHP as a scripting language to access databases in a shared web hosting (hosting) in the development of the app to implement the geotagging feature of the app using the ArcGIS API, with the use of Android Studio and VS Code as the development tools. In addition, SQLite will serve as a tool to create the offline database for mobile applications.

4.4.4 Testing

Before the actual implementation of the new application, Testing is performed in this section to identify flaws and other issues that users may encounter. The testing stage is critical for ensuring the application works properly and meets all requirements established during the analysis stage. Unit testing and user acceptance testing are some of the several test criteria that must be included. As part of the user acceptance testing, the researchers conducted alpha and beta tests with the customer to collect final input and confirm if the required result was effectively delivered. Test cases and results were created and distributed. In addition, a list of testing approaches is described below to provide a better and more precise understanding of each test.

4.4.4.1 Unit Testing

Unit testing's main objective is to separate the smallest piece of tested software in the application from the rest of the code and see if it performed as expected. The researchers finalized it immediately after the software was finished.

One of the units being tested in the application is the mapping functionality to ensure the accuracy of the points portrayed in the mapping module.

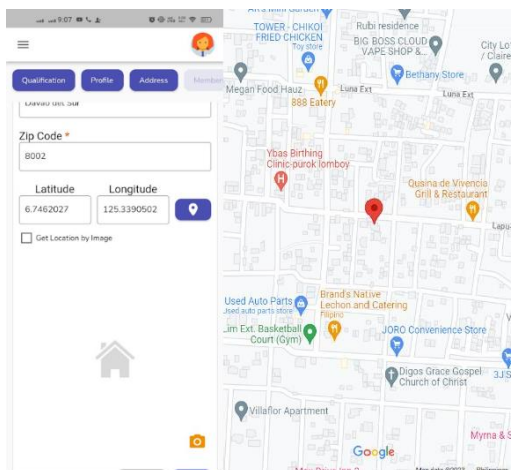


Figure 9: Location Obtained by the e4psMap App

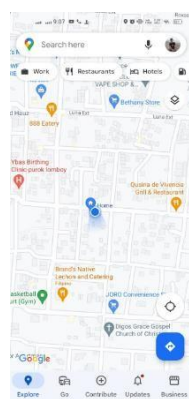


Figure 10: Current Location Base on Google Maps

Figure 9 and Figure 10 portray the different location information retrieved by the e4psMap application and from Google Maps within the exact location simultaneously. As the figure illustrates, the "Get Location" functionality is the same as Google Maps because the e4psMap application uses the same components as Google Maps. Like Google Maps, the developed application uses the same technology embedded in the smartphone, which is the GPS component of the smartphone. The app takes advantage of this component through the use of Geolocator, a plugin that lets the app utilize the GPS functionality of the application. With accuracy the same as Google Maps, the application will ensure that the location obtained during the survey is as accurate as Google Maps. The accuracy of tagging the location of the beneficiaries will help give precise visualization of the data in the mapping functionality.

4.4.4.2 User Acceptance Testing

In this section, the researchers conducted the final Testing after the development phase—the alpha and beta tests, which are types of user acceptance testing in which the researchers

allowed the client to perform and test the system with the agreed-upon requirements.

The data was collected and analyzed using a weighted arithmetic mean. A 5-point Likert scale was utilized to evaluate the system's attributes in testing its performance and features. It's a type of psychometric response scale in which people rate how much they agree with a statement on a scale of one to five.

Strongly Agree	=	5 points
Agree to Some Extent	=	4 points
Neither Disagree or Nor	=	3 points
Agree		
Disagree to Some Extent	=	2 points
Strongly Disagree	=	1 point

The calculation of the mean and the frequency of the score were utilized to understand the survey results. The following scale was used to visualize how users rated each application attribute.

Range of Weighted Mean (%)	Interpretation
4.51 - 5.00	Strongly Agree
3.50 - 4.50	Agree to Some Extent
2.51 - 3.50	Neither Disagree or Nor Agree
1.51 - 2.50	Disagree to Some Extent
1.50 and below	Strongly Disagree

Table 4. Range of Mean and Interpretation

The results shown below are the results from the survey conducted on the users of the system, which is the BHW. To identify the respondents, cluster sampling has been used, where respondents are divided into groups per barangay and choose one participant per cluster. The survey was participated by 26 different barangays in Digos City, which means that the survey also had 26 participants. To calculate the result, Microsoft Excel has been used to generate survey reports. Also, the 5-point Likert scale has been applied for the data gathering.

Frequency						
Area	5	4	3	2	1	Total
Ease of Use	15	10	1	0	0	26
Training	19	7	0	0	0	26
Technical Support	17	9	0	0	0	26
Consultation	13	12	1	0	0	26
Work Need Met	17	9	0	0	0	26
Application Capabilities	19	7	0	0	0	26
Commitment	21	5	0	0	0	26

Table 5: BHW Frequency Distribution

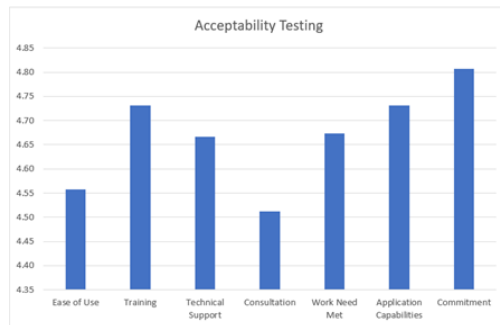


Figure 11: Result of the conduct of testing for users

Figure 11 presents the results from the calculation per category from the BHW's response to the survey. The categories are the following:

Ease of Use has an average score of 4.56, corresponding to Strongly Agree; this indicates that the system is easy to use and understand. This also means that users are confident using the app for their tasks.

Training has an average score of 4.73, corresponding to Strongly Agree; this indicates that the users will dedicate time and effort to train how to use the application. This also suggests that they will be willing to attend any further training for the application.

Technical Support has an average score of 4.67, equivalent to Strongly Agree; users can reach developers if they have trouble accessing or using the application. They can raise queries if they ever encounter difficulties while using the application.

Consultation has an average score of 4.51 which corresponds to Strongly Agree; this indicates that BHW can be appropriately consulted about using the application and can adopt the new system they will be using in completing their designated tasks.

Work Need Met has an average score of 4.67, corresponding to Strongly Agree; this means the application has met the functional requirements needed to complete its tasks. The

application can aid the BHW in their field working tasks.

Application Capabilities has an average score of 4.73 which corresponds to Strongly Agree; this indicates that the application can increase the efficiency and the number of work done by the BHW when it comes to gathering 4p's data.

Commitment has an average score of 4.81, equivalent to Strongly Agree; this means that if the application is introduced, they are willing to use it. This also shows that the users are committed to learning the application.

4.4.4.3 Alpha Testing

This type of Testing is performed to identify all potential issues or bugs before releasing a product to the general public or regular consumers[20]. It was created and implemented using test cases and test outcomes. Alpha testing is performed early in the software development process, near the end.

The alpha testing has been conducted by the research adviser of this capstone project through a series of test criteria. The test criteria for the alpha testing include the Testing of all the units and modules included in the app and also the overall functionality of the application. The test criteria were crafted first based on the application's objectives and functionality. The test questionnaire was given to the research adviser, who thoroughly checked all the modules and features in the test criteria. The test results of the alpha testing are attached in Appendix A. The research adviser ensures that all of the functionalities of the applications pass the standards being set.

4.4.4.4 Beta Testing

During this testing phase, applications will be used by real or targeted users; they are the ones who will evaluate the application's performance and overall usability [21]. The beta testers for the application are the BHW from the 26 Barangay in Digos City and the panels for this capstone project.

A questionnaire based on the ISO 25010 software quality model, an international standard for software evaluation, was used for software evaluation. The end users will be the ones to judge if it meets the needed requirements. In the beta testing, respondents will closely monitor the application's performance and solicit user input. This helps identify bugs or issues that need to be addressed and allows researchers to make necessary application improvements based on user feedback. Beta testing also provides

respondents with valuable insights into how the application is being used and how it can be improved to meet users' needs better. The beta testing result is also attached in Appendix A. This Testing will ensure that the application will be delivered with utmost quality and meet the needs of the end users.

5. RESULTS AND DISCUSSION

Researchers have reviewed the survey results obtained throughout the system's development in this chapter. The discussion starts from the objective to the testing plan, which covers the system's functional and non-functional facets.

5.1 Implementation Plan

In implementing the Android Smartphone application for barangay health workers (BHW) in each of the 26 barangays in Digos City, the following activities were conducted to guide the end - users:

Strategy	Activities	Person's Involved	Duration
Information Distribution	Application Presentation	Proponent, End users	30 mins.
Installation	Application Installation	End Users	30 mins.
Training	Hands - on Demonstration	End Users	30 mins.

Table 6. Implementation Plan

5.2 Implementation Result

After the implementation, the BHWs from the 26 barangay can use the application. The BHWs can perform transactions such as saving, updating, and viewing 4ps information from the application. To go back to the main objectives of the research, which aim to create an application capable of capturing data from the 4ps beneficiaries within Digos City. The objectives of the systems are being completed through the development of the application. The following are the discussions on how the study proceeded: (1) To create a mobile application system that collects the needed 4Ps data using the Flutter framework. (2) To integrate offline mode for data gathering and will be synced to the online database using the internet. (3) To geotag the beneficiary's address and generate a heatmap visualization using ArcGIS

5.2.1 To create a mobile application system that collects the 4Ps data using the Flutter framework. After the implementation, it has been shown that the objectives have been met; the BHW can collect data from the 4p's beneficiaries and can store it in the dedicated databases.

Figure 12: Application Form

Figure 12 shows the add applicant module where the BHWs can record 4ps beneficiaries' data. This lets the BHW save the data needed for the system. Information that needs to be input is the qualification information of the beneficiaries, such as their monthly income, number of members within the household, year of residency, type of residency, and other information that corresponds to the qualification of the beneficiaries. After all the qualifications, the BHW can now input the personal information of the household head, such as their name, birthday, contact information, and educational attainment. After that is the address part of the form, where the BHW needs to supply information corresponding to where the beneficiaries live, including the latitude and longitude, which will be used in the mapping function of the application.

Figure 13: Saved Data

Figure 13 portrays the data that are being saved using the application form shown in Figure 12. The BHW can view all the 4ps beneficiary data inputted in the application form. All the online

data will be displayed in this table, and the BHWs can view all the information corresponding to a specific beneficiary.

5.2.2 To integrate offline mode for data gathering and will be synced to the online database using the internet. The implementation results have proved that the integration of the offline mode is useful in the data-gathering tasks of the BHW. It shows that the users can use it on their fieldwork wherever they are.

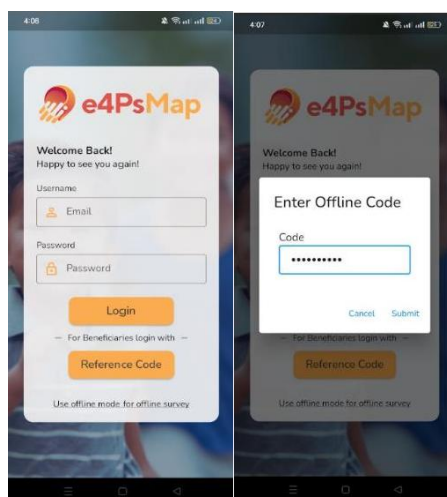


Figure 14: Offline Login

To access the offline mode of the application, figure 14 shows the login form of the e4psMap application where they can enter their offline login credentials to access the offline module of the app. The offline module will also include the application form shown in Figure 12; this form will save the data to an offline database in the application. The data will only be accessible to that mobile phone and will be available online if the BHW synced it with the online database when the internet is available.

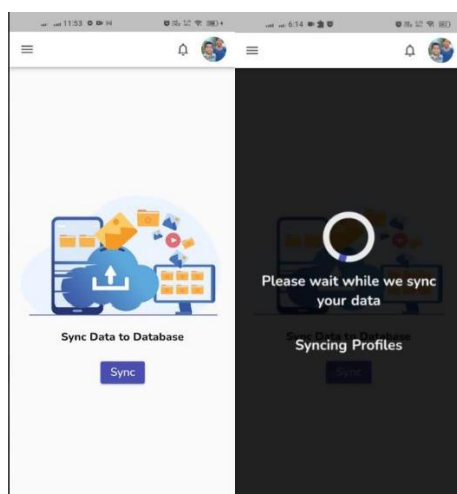


Figure 15: Offline Data to Online Sync

Figure 16 shows the sync module of the e4psMap application. During this process, all of the data collected and saved offline will be synced to the online database, which means that all the data will be uploaded to the online database, which makes the data available online and can be accessed by the web system.

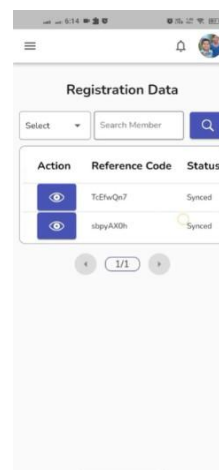


Figure 17: Table of Synced Offline Data

The table where the offline data resides will be updated when all the are synced online, as shown in Figure 17. After it is successfully synced, the data will be updated; the status will turn to sync, and a reference code will be inserted. The reference code is the code to be used to access that specific data online. If the data has not been synced, it will still show in the status as unsynced, and the particular data will not have a reference code.

5.2.3 To geotag the beneficiary's address and generate an ArcGIS heatmap visualization. The mapping module of the system has been tested to be helpful in the fieldwork task of the BHW. The map functionality can help BHW to locate beneficiaries in their designated barangay and see the density of beneficiaries in a particular area.

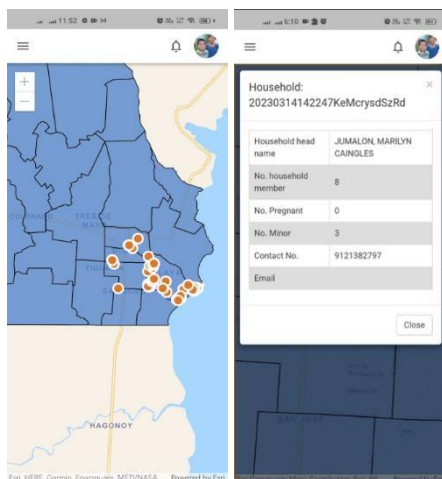


Figure 17: Point Map

Figure 17 shows the mapping module of the application. In this module, all beneficiaries in a specified barangay will be plotted on the map. The points displayed on the map will depend on where the BHW is assigned. The facts are also pressable and contain information about that specific point. Information may include the name of the household leader, number of family members, number of pregnant, number of minors, and contact information.



Figure 18: Heatmap Module

Included in the mapping module of the application is the heatmap module. The heatmap module will convert all the points, which are the 4ps beneficiaries, into a heatmap. The heatmap will display the density of beneficiaries in a particular area; the darker the color, the denser the site and the faded the color is, the less dense the area. The density is computed through the quartic density function, which calculates the distance between points within a certain radius.

6. CONCLUSION AND RECOMMENDATION

6.1 Conclusion

Based on the results gathered from the research, the researchers have concluded that the application is functional and has met the objectives stated. The study also concluded the following:

- 6.1.1 Using the application, the BHW can save, retrieve and update 4ps data. It is easy for them to gather data during their fieldwork since there will be no burden on carrying out survey forms.
- 6.1.2 Through the application, the BHW can save and access data that are saved locally without using the internet. The feature has been helpful, especially in areas without an internet connection, and they can sync it when the internet is available.
- 6.1.3 The BHW can locate and plot beneficiaries' locations using the application. The feature helps know where the beneficiaries are located, and it can produce information based on the density of the beneficiaries in a particular area.

6.2 Recommendation

The researchers have listed recommendations for future researchers to improve or add to the system and will serve as their guide in their research.

- 6.2.1 The researchers have suggested integrating an offline map version. Since mapping is a vital part of the application. Even though the app is accessed offline, BHW can still access the map using offline data saved locally.
- 6.2.2 Also, since the application can import offline data online, researchers recommended integrating functionality that can import the data online to offline so that BHW can still retrieve the recently ported in even without an internet connection.
- 6.2.3 Finally, the app must also be available on IOS platforms for IOS users. Even though the Flutter framework can build Android and IOS applications, the researchers still need the technology to package it to IOS since it requires Apple devices to be compiled.

7. REFERENCES

- [1] madhu nair, "Is Society Too Dependent On Technology? 10 Shocking Facts," *University of the People*, May 26, 2020. <https://www.uopeople.edu/blog/society-too-dependent-on-technology/> (accessed Sep. 15, 2022).
- [2] "20190417-RA-11310-RRD.pdf." Accessed: Sep. 15, 2022. [Online]. Available: <https://www.officialgazette.gov.ph/downloads/2019/04apr/20190417-RA-11310-RRD.pdf>
- [3] X. Gregorio, "COA: 90% of 4Ps beneficiaries remain poor," *Philstar.com*. <https://www.philstar.com/business/2022/10/13/2216405/coa-90-4ps-beneficiaries-remain-poor> (accessed Nov. 10, 2022).
- [4] "4Ps-First-Quarter-2022-Accomplishment-Report.pdf." Accessed: Sep. 22, 2022. [Online]. Available: <https://pantawid.dswd.gov.ph/wp-content/uploads/2022/05/4Ps-First-Quarter-2022-Accomplishment-Report.pdf>
- [5] C. J. Diokno-Sicat, C. E. Adaro, and R. B. Maddawin, "The Community-Based Monitoring System (CBMS) as a Local Planning Tool: Results from the PIDS-DILG Baseline Study on Policy and Governance Gaps for the Local Government Support Fund Assistance to Municipalities (LGSF-AM) Program," p. 34.
- [6] R. Sultani, A. Soliman, and K. Al-Hagla, "The Use of Geographic Information System (GIS) Based Spatial Decision Support System (SDSS) in Developing the Urban Planning Process," *APJ Archit. Plan. J.*, vol. 1, pp. 97–115, Jan. 2009.
- [7] "EM_RC54_Tech_Disc_3_en.pdf." Accessed: Oct. 26, 2022. [Online]. Available: https://apps.who.int/iris/bitstream/handle/10665/122558/EM_RC54_Tech_Disc_3_en.pdf;jsessionid=E5209DC946FED3FE6BA8999A2DD99B69?sequence=1
- [8] H. K. Liu, M. J. Hung, L. H. Tse, and D. Saggau, "Strengthening urban community governance through geographical information systems and participation: An evaluation of my Google Map and service coordination," *Aust. J. Soc. Issues*, vol. 55, no. 2, pp. 182–200, 2020, doi: 10.1002/ajs4.98.
- [9] D. Kurniadi, A. Mulyani, Y. Septiana, and G. G. Akbar, "Geographic information system for mapping public service location," *J. Phys. Conf. Ser.*, vol. 1402, no. 2, p. 022073, Dec. 2019, doi: 10.1088/1742-6596/1402/2/022073.
- [10] I. Mannan *et al.*, "Using spatial analysis and GIS to improve planning and resource allocation in a rural district of Bangladesh," *BMJ Glob. Health*, vol. 4, Jul. 2019, doi: 10.1136/bmjgh-2018-000832.
- [11] "What is Barangay Health Worker? | Department of Health website." <https://doh.gov.ph/faqs/What-is-Barangay-Health-Worker> (accessed Nov. 25, 2022).
- [12] C. J. Abenir *et al.*, "Pantawid Pamilyang Pilipino Program 4Ps: A Profiling System," in *2021 1st International Conference in Information and Computing Research (iCORE)*, Dec. 2021, pp. 102–106. doi: 10.1109/iCORE54267.2021.00037.
- [13] P. Read, G. Higgs, and G. Taylor, "The potential and barriers to the use of geographical information systems for marketing applications in higher educational institutions," *Mark. Intell. Plan.*, vol. 23, no. 1, pp. 30–42, Jan. 2005, doi: 10.1108/02634500510577456.
- [14] "Barangay Profiling System With Analytics | PDF | Analytics | System," *Scribd*. <https://www.scribd.com/document/519575750/Barangay-Profiling-System-with-Analytics> (accessed Oct. 26, 2022).
- [15] J. H. Tiongson, "Rural Health Unit Decision Support System with Mapping," *Int. J. Recent Technol. Eng.*, vol. 9, p. 611, Feb. 2021, doi: 10.35940/ijrte.C4681.099320.
- [16] "Development of an Information-Based Dashboard: Automation of Barangay Information Profiling System (BIPS) for Decision Support towards e-Governance | 2020 The 4th International Conference on E-Society, E-Education and E-Technology." <https://dl.acm.org/doi/10.1145/3421682.3421691> (accessed Oct. 26, 2022).
- [17] T. Hengl, "Finding the right pixel size," *Comput. Geosci.*, vol. 32, no. 9, pp. 1283–1298, Nov. 2006, doi: 10.1016/j.cageo.2005.11.008.
- [18] "About ArcGIS | Mapping & Analytics Software and Services." <https://www.esri.com/en-us/arcgis/about-arcgis/overview> (accessed Sep. 15, 2022).
- [20] S. Alsaqqa, S. Sawalha, and H. Abdel-Nabi, "Agile Software Development: Methodologies and Trends," *Int. J. Interact. Mob. Technol. IJIM*, vol. 14, no. 11, p. 246, Jul. 2020, doi: 10.3991/ijim.v14i11.13269.

- [20] “Alpha Test.”
<https://www.productplan.com/glossary/alpha-test/> (accessed Jul. 08, 2023).
- [21] “Beta Test.”
<https://www.productplan.com/glossary/beta-test/> (accessed Jul. 08, 2023).

APPENDICES

Appendix A

Test Case



COLLEGE OF COMPUTING EDUCATION

[] Main [x] Branch Digos

ALPHA TESTING CRITERIA

(Final System / Adviser)

Thesis / Capstone Title:

eQpsMap: A Mobile Application for 9w beneficiaries for the City of
Digos with Geo-tagging Technology

Group Members:

Ralph Ian O. Noyway, Rhona Mae Magayo, Jonalyn E. Calvo

Product Quality:	Weight (100%)	Score
a. Reliability, proper functioning as specified and expected.	10%	10
b. Robustness, acceptable response to unusual inputs, loads and conditions.	10%	9
Functional Testing: execute each use case, use case flow, or function, using valid and invalid data with an objective to verify that:		
a. The expected results occur when valid data is used.	10%	10
b. The appropriate error / warning message are displayed when invalid data is used.	10%	10
Functionality: - Suitability - Accuracy - Interoperability - Compliance - Security	10%	10
Reliability: - Maturity - Fault-tolerance - Recoverability	10%	10
Usability: - Understandability - Learnability - Operability	10%	10
Efficiency: - Time Behaviour - Resource Behaviour	10%	9
Maintainability: A set of attributes that bear on the effort needed to make specified modified modifications.	10%	10
Portability: A set of attributes that bear on the ability of software to be transferred from one environment to another.	10%	8

TOTAL 100% 96

Tester:

JOHN RALPH O. NOYWAY
Signature over Printed Name


COLLEGE OF COMPUTING EDUCATION

 [] Main [✓] Branch Digos
BETA TESTING CRITERIA

(Final System / End-user)

Thesis / Capstone Title:

e4psMap: A mobile application for 4Ps beneficiaries for the City of Digos with Geo-tagging Technology

Group Members:

Ralph Ian D. Noybay, Rhona Mae Magrayo, Jonalyn E. Calvo

Product Quality:	Weight (100%)	Score
a. Reliability, proper functioning as specified and expected.	5%	5
b. Robustness, acceptable response to unusual inputs, loads and conditions.	5%	5
c. Efficiency of use by the frequent users.	5%	4
d. Easy to use even for the frequent users.	5%	5
Functional Testing: execute each use case, use case flow, or function, using valid and invalid data with an objective to verify that:		
a. The expected results occur when valid data is used.	10%	9
b. The appropriate error / warning message are displayed when invalid data is used.	10%	9
Functionality: - Suitability - Accuracy - Interoperability - Compliance - Security	10%	10
Reliability: - Maturity - Fault-tolerance - Recoverability	10%	10
Usability: - Understandability - Learnability - Operability	10%	10
Efficiency: - Time Behaviour - Resource Behaviour	10%	10
Maintainability: A set of attributes that bear on the effort needed to make specified modified modifications.	10%	8
Portability: A set of attributes that bear on the ability of software to be transferred from one environment to another.	10%	10

TOTAL

100%

95

Tester:

Signature over Printed Name


COLLEGE OF COMPUTING EDUCATION

 [] Main [x] Branch Davao
BETA TESTING CRITERIA
 (Final System / End-user)

Thesis / Capstone Title:

e4psMap: A Mobile Application for 4Ps beneficiaries for the City of Davao with Geo-tagging Technology

Group Members:

Ralph Jan O. Naymay, Rhonda Mae Magsayo, Jonalyn E. Colvo

Product Quality:	Weight (100%)	Score
a. Reliability, proper functioning as specified and expected.	5%	5
b. Robustness, acceptable response to unusual inputs, loads and conditions.	5%	5
c. Efficiency of use by the frequent users.	5%	5
d. Easy to use even for the frequent users.	5%	5
Functional Testing: execute each use case, use case flow, or function, using valid and invalid data with an objective to verify that:		
a. The expected results occur when valid data is used.	10%	8
b. The appropriate error / warning message are displayed when invalid data is used.	10%	10
Functionality: - Suitability - Accuracy - Interoperability - Compliance - Security	10%	8
Reliability: - Maturity - Fault-tolerance - Recoverability	10%	10
Usability: - Understandability - Learnability - Operability	10%	8
Efficiency: - Time Behaviour - Resource Behaviour	10%	10
Maintainability: A set of attributes that bear on the effort needed to make specified modified modifications.	10%	10
Portability: A set of attributes that bear on the ability of software to be transferred from one environment to another.	10%	10

 TOTAL 100% 94

Tester:

EDUARD PULUBRA, MCW
 Signature over Printed Name

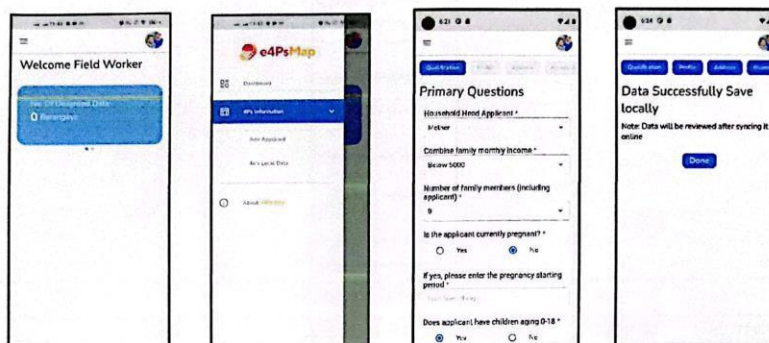

COLLEGE OF COMPUTING EDUCATION
☐ Main ☒ Branch Digos
BETA TESTING CRITERIA
(Final System / End-user)
Thesis / Capstone Title:
e4prMap: A Mobile Application for 4pr beneficiaries for the City of Digos with Geo-tagging Technology
Group Members:
Ralph Jan O. Noyway, Rhona Mae Nogscayo, Jonathan E. Calvo

Product Quality:	Weight (100%)	Score
a. Reliability, proper functioning as specified and expected.	5%	✓
b. Robustness, acceptable response to unusual inputs, loads and conditions.	5%	✓
c. Efficiency of use by the frequent users.	5%	✓
d. Easy to use even for the frequent users.	5%	✓
Functional Testing: execute each use case, use case flow, or function, using valid and invalid data with an objective to verify that:		
a. The expected results occur when valid data is used.	10%	10
b. The appropriate error / warning message are displayed when invalid data is used.	10%	9
Functionality: - Suitability - Accuracy - Interoperability - Compliance - Security	10%	9
Reliability: - Maturity - Fault-tolerance - Recoverability	10%	9
Usability: - Understandability - Learnability - Operability	10%	10
Efficiency: - Time Behaviour - Resource Behaviour	10%	9
Maintainability: A set of attributes that bear on the effort needed to make specified modified modifications.	10%	9
Portability: A set of attributes that bear on the ability of software to be transferred from one environment to another.	10%	10

TOTAL 100%
82
Tester:
CECIL DAVE T. DARARCE
 Signature over Printed Name



General Information			
Test Stage	☒ Unit ☐ Functionality ☐ Integration ☐ System ☐ Interface ☐ Performance ☐ Regression ☐ Acceptance ☐ Pilot		
Test Date	May 04, 2023	System Date, if applicable	
Tester	CYRIL DAVE DASARCO	Test Case Number	U_01
Test Case Description	Testing the Log-in Module of the Application		
Results	☒ Pass ☐ Fail	Incident Number, if applicable	IN_01
Introduction			
Requirement(s) to be Tested	The system/application must be installed to an android smartphone to access the login module		
Roles and Responsibilities	The following are the member of a team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.		
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Enter a valid Username and Password		
Stop Procedures	When the Username and Password is already accepted the testing procedure is terminated.		
Environmental Needs			
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users		

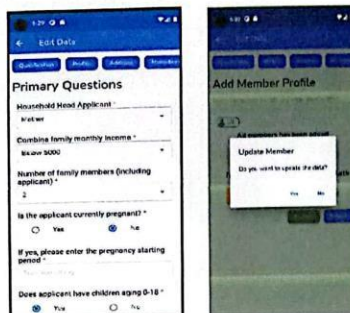


General Information						
Test Stage	☒ Unit		☐ Functionality	☐ Integration	☐ System	☐ Interface
	☐ Performance		☐ Regression	☐ Acceptance	☐ Pilot	
Test Date	MAY 04, 2023		System Date, if applicable			
Tester	CYRIL DAVE T. DASARAO		Test Case Number		U_03	
Test Case Description	Testing the Offline Mode Form Module of the Application					
Results	☒ Pass		☐ Fail	Incident Number, if applicable		IN_03
Introduction						
Requirement(s) to be Tested	The user must be able to submit an application without internet and will be saved to the local database of the app					
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.					
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Click the Go offline Mode 4. Enter the Given Code 5. Select the 4ps Information 6. Click the Field Work 7. Fill up the fields required 8. Click submit after all fields are filled up					
Stop Procedures	The testing process was stopped once the users had successfully added the 4PS beneficiaries' information without errors.					
Environmental Needs						
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users					

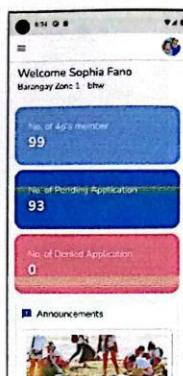
General Information				
Test Stage	☒ Unit ☐ Functionality ☐ Integration ☐ System ☐ Interface ☐ Performance ☐ Regression ☐ Acceptance ☐ Pilot			
Test Date	MAY 04, 2023	System Date, if applicable		
Tester	CYRIL PAUL DAKARUD	Test Case Number		U_08
Test Case Description	Testing the online add applicant module of the application			
Results	☒ Pass	☐ Fail	Incident Number, if applicable	IN_08
Introduction				
Requirement(s) to be Tested	The user must be able to submit 4ps information online			
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.			
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Log in valid email and password 4. Click log in 5. Click hamburger menu 6. Select 4ps Information 7. Select Add applicant 8. Fill up required fields 9. Click Submit			
Stop Procedures	The testing process will be stopped until the users are able to add new applicants online.			
Environmental Needs				
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users			



General Information			
Test Stage	☒ Unit ☐ Functionality ☐ Integration ☐ System ☐ Interface ☐ Performance ☐ Regression ☐ Acceptance ☐ Pilot		
Test Date	MAY 04, 2019	System Date, if applicable	
Tester	CYRIL DAUG DIBARGAO	Test Case Number	U_04
Test Case Description	Testing the offline mode display data module of the application		
Results	☒ Pass ☐ Fail	Incident Number, if applicable	IN_04
Introduction			
Requirement(s) to be Tested	The user must be able to view previously saved data using the offline mode form in the device.		
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.		
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Click the Go offline Mode 4. Enter the Given Code 5. Select the 4ps Information 6. Click the Local Data 7. Click eye icon to show more information		
Stop Procedures	The testing process was stopped once the user can view the data they want to view or being selected.		
Environmental Needs			
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users		



General Information				
Test Stage	☒ Unit	☐ Functionality	☐ Integration	☐ System ☐ Interface
	☐ Performance	☐ Regression	☐ Acceptance	☐ Pilot
Test Date	MAY 04 . 2023	System Date, if applicable		
Tester	CYRIL DANE DAKARBO	Test Case Number		U_05
Test Case Description	Testing the edit module of the application			
Results	☒ Pass	☐ Fail	Incident Number, if applicable	IN_05
Introduction				
Requirement(s) to be Tested	The user must be able to edit locally saved data before it is synced online.			
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.			
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Click the Go offline Mode 4. Enter the Given Code 5. Select the 4ps Information 6. Click the Local Data 7. Click eye icon to show more information 8. Click edit icon 9. Edit data that is needed to be updated 10. Click submit 11. Select yes			
Stop Procedures	The testing process will be stopped until the users have successfully edited or updated the selected data.			
Environmental Needs				
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users			



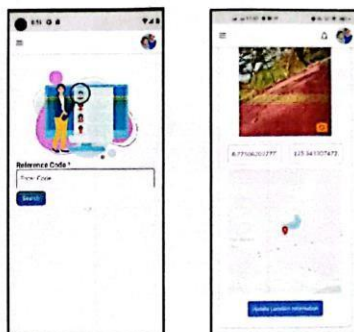
General Information						
Test Stage	☒ Unit		☐ Functionality	☐ Integration	☐ System	☐ Interface
	☐ Performance		☐ Regression	☐ Acceptance	☐ Pilot	
Test Date	MAY 04, 2023		System Date, if applicable			
Tester	CYVIL DAVE DASARLO		Test Case Number		U_06	
Test Case Description	Testing the dashboard of the application					
Results	☒ Pass		☐ Fail	Incident Number, if applicable		IN_06
Introduction						
Requirement(s) to be Tested	The user must be able to view different number indicators and announcements.					
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.					
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Log in valid email and password 4. Click log in					
Stop Procedures	The testing process will be stopped until the users have successfully logged in and be able to see the dashboard.					
Environmental Needs						
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users					



General Information			
Test Stage	☒ Unit ☐ Functionality ☐ Integration ☐ System ☐ Interface ☐ Performance ☐ Regression ☐ Acceptance ☐ Pilot		
Test Date	MAY 04, 2023	System Date, if applicable	
Tester	CYRIL DAVID DASARAO	Test Case Number	U_07
Test Case Description	Testing the map of the application		
Results	☒ Pass ☐ Fail	Incident Number, if applicable	IN_07
Introduction			
Requirement(s) to be Tested	The user must be able to view the map with different points.		
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.		
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Log in valid email and password 4. Click log in 5. Click hamburger menu 6. Select Map		
Stop Procedures	The testing process will be stopped until the users are able to view the map with different points being displayed.		
Environmental Needs			
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users		



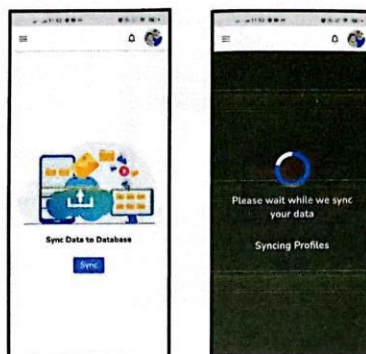
General Information			
Test Stage	☒ Unit ☐ Functionality ☐ Integration ☐ System ☐ Interface ☐ Performance ☐ Regression ☐ Acceptance ☐ Pilot		
Test Date	MAY 04, 2019	System Date, if applicable	
Tester	LOUIS DAVE DALARAU	Test Case Number	U_09
Test Case Description	Testing the online data module		
Results	☒ Pass ☐ Fail	Incident Number, if applicable	IN_09
Introduction			
Requirement(s) to be Tested	The user must be able to view 4ps information online.		
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.		
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Log in valid email and password 4. Click log in 5. Click hamburger menu 6. Select 4ps Information 7. Select 4ps Online data		
Stop Procedures	The testing process will be stopped until the users are able to view different data that is saved online.		
Environmental Needs			
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users		



General Information						
Test Stage	☒ Unit		☐ Functionality	☐ Integration	☐ System	☐ Interface
	☐ Performance		☐ Regression	☐ Acceptance	☐ Pilot	
Test Date	May 04, 2023		System Date, if applicable			
Tester	Cyvil DAVE DALANGU		Test Case Number		U_10	
Test Case Description	Testing the update applicant location module of the application					
Results	☒ Pass		☐ Fail	Incident Number, if applicable		IN_10
Introduction						
Requirement(s) to be Tested	The user must be able to update beneficiary location					
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.					
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Log in valid email and password 4. Click log in 5. Click hamburger menu 6. Select 4ps Information 7. Select Update Applicant Location 8. Enter a valid reference code 9. Take a picture or select image from gallery 10. Click Update information 11. Select yes					
Stop Procedures	The testing process will be stopped until the users successfully update the location data of the beneficiary.					
Environmental Needs						
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users					



General Information						
Test Stage	☒ Unit		☐ Functionality	☐ Integration	☐ System	☐ Interface
	☐ Performance		☐ Regression	☐ Acceptance	☐ Pilot	
Test Date	May 04, 2019		System Date, if applicable			
Tester	Cyvil DAVE DASARAO		Test Case Number		U_11	
Test Case Description	Testing the announcement module of the application					
Results	☒ Pass		☐ Fail	Incident Number, if applicable		IN_11
Introduction						
Requirement(s) to be Tested	The user must be able to update beneficiary location					
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.					
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Log in valid email and password 4. Click log in 5. Click hamburger menu 6. Select Announcements					
Stop Procedures	The testing process will be stopped until the users can view all the announcements being posted.					
Environmental Needs						
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users					



General Information				
Test Stage	☒ Unit ☐ Functionality ☐ Integration ☐ System ☐ Interface			
	☐ Performance ☐ Regression ☐ Acceptance ☐ Pilot			
Test Date	May 04, 2023	System Date, if applicable		
Tester	CYRIL DAVE DASARLHO	Test Case Number		U_12
Test Case Description	Testing the Online Sync Data Module of the software			
Results	☒ Pass	☐ Fail	Incident Number, if applicable	IN_12
Introduction				
Requirement(s) to be Tested	The local data must be successfully uploaded to the online database			
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.			
Set up Procedures	The following are the procedure before the test will run; 1. Turn on the Mobile Phone 2. Open the installed system (e4psMap) 3. Log in valid email and password 4. Click log in 5. Click hamburger menu 6. Select Sync Data 7. Click Sync			
Stop Procedures	The testing process was stopped once the user had successfully synced all the data that was processed in offline mode.			
Environmental Needs				
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users			



General Information				
Test Stage	☒ Unit	☐ Functionality	☐ Integration	☐ System
	☐ Performance	☐ Regression	☐ Acceptance	☐ Pilot
Test Date	May 04, 2023	System Date, if applicable		
Tester	IGNIL DAME DASARUO	Test Case Number		U_02
Test Case Description	Testing the Log-out Module of the Application			
Results	☒ Pass	☐ Fail	Incident Number, if applicable	IN_02
Introduction				
Requirement(s) to be Tested	The user must be logged out their accounts successfully in the application			
Roles and Responsibilities	The following are the member of team that is included in the testing; 1. System Developers will take note of the error to be listed in a sprint for further deliberation in improving or solving the problems. 2. Tester will be the one to take the sequence of steps and to input the necessary data to be tested. 3. User/Expert will be the one to validate the result if the test will be marked pass or fail.			
Set up Procedures	The following are the procedure before the test will run; 1. Open the e4ps Map application. 2. Login existing user information. 3. Press profile in the top navigation 4. Select Log out 5. Press yes in dialogue box			
Stop Procedures	When the user have successfully log out their accounts in the application			
Environmental Needs				
Hardware	The following are the things to be considered in terms of hardware; 1. Android Smartphone Users			

E4ps Map Mobile System Testing				
Test Case ID: TC_OM_01		Test Designed by: Ralph Ian Noynay Rhona Mae Magsayo Jonalyn E. Calvo		
Test Priority (Low/Med/High): High		Test Designed Date: April 30, 2023		
Description: Testing the overall functionality of the application				
Pre-conditions: e4p's app must be installed to the device and users are able to open the app.				
Step	Test Description	Yes	No	Note
1	Is the app downloadable online in a public website or through the playstore?	/		
2	Is the application installable via android smartphones?	/		
Log in				
1	Are BHW's able to login with incorrect username and password or via incorrect offline code?	/		
2	Are BHW's able to login with the correct username and password or correct offline code?	/		
3	Can users be able to login with their reference code?	/		
4	Can the application notify the user that they entered the incorrect email, password, reference code and offline code?	/		
Offline Mode				
1	Are BHW's able to see the correct number or synced and unsynced data?	/		
2	Can the BHW be able to create new applications without the use of the internet?	/		
3	Can the BHW be able to view different applications that are recorded offline?	/		
4	Can the BHW be able to delete offline data?	/		
5	Can the BHW be able to edit offline data when it's unsynced?	/		

Online Mode				
1	Does the application display accurate number indicators for pending, approved, and denied beneficiaries?	/		
2	Can BHW be able to view the different announcements being posted?	/		
3	Is the BHW be able to view the following in the map:	/		
	a. General Outline of Digos City	/		
	b. Points representing Beneficiaries	/		
	c. Points are pressable and able to display corresponding beneficiary information	/		
	e. Does a recently added applicant with location data be plotted in the map?	/		
4	Is the BHW able to create an application online?	/		
5	Can the BHW update the location of the applicants?	/		
6	Can the application extract location information (latitude and longitude) from an image with the following meta data?	/		
7	Can the BHW change their offline code?	/		
8	Does all the unsynced data be synced when the BHW decides to sync all the local data the app has?	/		
10	Is the app able to return reference code after the data is done syncing?	/		
11	Can BHW change their offline code to access offline module?	/		
User Dashboard				
1	Can beneficiaries view the current status of their application?	/		
2	Can the beneficiaries view announcements posted?	/		
3	Is all information presented accurate or	/		

	corresponds to the currently logged in beneficiary?			
--	---	--	--	--


DAVID T. DASARGAO, MIT
Tester

05/04/2023
Test Date

**SURVEY ON ACCEPTABILITY ON E-4PsMap: A data analytics-driven Pantawid
Pamilyang Pilipino Program (4Ps) maternal and neonatal outcomes monitoring,
visualization, and dashboard system for the City of Digos**

Barangay: TRES DE MARZO Name (Optional): _____

Isip mogamit sa e4Psmap na system, buot namong mahibaw-an kung unsa ang inyohang panan-aw o opinyon aron mas mapanindot pa namo siya, andam sa inyohang paggamit sa umaabot na mga adlaw. Giawhag kamo na basahon ug sutaon ang kada pahayag kung nakita, nasinati, o nabantayan mo ba kini sa imohang paggamit karon sa e4PsMap. Gamita ang iskala sa ubos sa paghatag ug pagtamod.

Point Rating	5	4	3	2	1
Description	Hingpit sa Pag-uyon	Uyon	Dili pa masuta kung uyon o dili	Dili uyon	Hingpit sa pagdumili

Pahinumdam: Kung aduna moy dugang na komento o sugyot aron mas mapanindut pa ang e4PsMap system, palihug sa pagsuwat sa gihatag na blangko.

Part 1: Ang Kasayon sa Paggamit	5	4	3	2	1	Komento
1. Kung gamiton ang e4PsMap, nisalig ko na dali ug hapsay ra ang akoang paggamit ani.	✓					
2. Kumpyansa ko na mas dali ang pagtudlo nako sa akoang nahibaw-an kung masweto nako sa paggamit sa e4PsMap	✓					
3. Mamahimong mas pamub-on sa e4PsMap ang oras na magahin sa pagbuhat sa trabaho kumpara sa kasamtangang mano-mano na proseso.	✓					
4. Nakita nako na dili ing-ana kakomplikado gamiton ang e4PsMap	✓					

Part 2: Pagbansay (Training)	5	4	3	2	1	Comments
1. Andam ko na mobalik ug <i>training</i> kung aduna may regular na pagbansay o pagpasabot sa paggamit sa e4PsMap.	✓					
2. Motagana ko ug oras o panahon kung aduna may nagapadayon na pagbansay o training sa paggamit sa e4PsMap.	✓					

3. Motagana ko ug oras o panahon kung aduna may pormal na pagbansay o training sa paggamit sa e4PsMap.	✓					
--	---	--	--	--	--	--

Part 3: Teknikal na Suporta	5	4	3	2	1	Comments
1. Sa akong panlantaw, kinahanglan gyud ug tabang o suporta gikan sa technical staff para mas epektibo ang paggamit sa e4PsMap.	✓					
2. Kung maglisod man tuod sa paggamit sa e4PsMap, magkinahanglan na adunay hinanali na masangpit na technical support staff.	✓					
3. Kung maglisod man tuod sa paggamit sa e4PsMap, kinahanglan abtik o alisto ang technical support staff sa paghatag ug tabang.	✓					

Part 4: Konsultasyon	5	4	3	2	1	Comments
1. Nakita nako na konsultahon ko ug husto ang pagkonsiderar sa akoang panginahanglan isip mogamit sa e4PsMap ayha pa kini ipatuman sa CSWDO.	✓					
2. Nakita nako na pangutan-on ko kung ang e4PsMap na system nakuha ba kung unsa akoang gilauman para sa usa ka system.	✓					
3. Ang unsa mang kabalaka nako kabahin sa e4PsMap system hugot na paminawon ayha pa kini ipatuman.	✓					

Part 5: Pag-abot sa Kinahanglan na Trabaho	5	4	3	2	1	Comments
1. Gilauman nako na mas daghan ang mahuman nako na trabahoon sa susamang panahon sa akoang paggamit sa e4PsMap.	✓					
2. Gilauma nako na mas mapanindot pa ang kalidad sa akoang trabaho pinaagi sa e4PsMap.	✓					

Part 6: Mga Kapabilidad sa Sistema	5	4	3	2	1	Comments
1. Nakita nako ang gamit sa e4PsMap sa paghuman sa mga gitakna na mga	✓					

tulumanon.						
2. Nakita nako na mas madugangan pa sa e4PsMap Website ang akoang kapasidad sa pagbuhat sa akoang mga tulumanon.	✓					
3. Misalig ako na ang e4PsMap na system dili mahagsa ug makalangan sa akoa trabaho.	✓					
4. Kaya sa e4PsMap na system ang pagproseso sa akoang mga trabahoon	✓					
5. Kaya sa e4PsMap na mas mahuman nako akoang tulumanon sa gitaganang oras	✓					

Part 7: Pasalig	5	4	3	2	1	Comments
1. Gamiton nako ang e4PsMap na system kung ipaila ug ipagamit kini sa CSWDO.	✓					
2. Mopasalig ko na maningkamot ko sa pagtuon sa e4PsMap na system kung ipaila ug ipagamit kini sa CSWDO.	✓					

Mga Sugyot:

NEED NG MATERIAL KAY DILI DANAN BITU
NAGY ABLE NA CP PARA MAGAMIT.

APPENDIX B

Source Code

The source code provided below are only sample codes and not the complete code of the system.

Login Interface

```

class Login extends StatefulWidget {
  const Login({Key? key}) : super(key:
key);

  @override
  State<Login> createState() =>
_LoginState();
}

class _LoginState extends State<Login> {
  bool isChecked = false;

  bool showcode = true;

  Color? yellow = Color.fromRGBO(251,
171, 80, 1);

  @override
  void initState() {
    countoffpas();

    Checkinternet();

    checkpermstorage();

    setter();

    super.initState();
  }

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(image:
DecorationImage(image:
AssetImage('assets/kids.png'), fit:
BoxFit.cover)),

      child: Scaffold(

        backgroundColor:
Color(0x070D30).withOpacity(0.4),

        body: Stack(

          children: [

            Center(

              child: SingleChildScrollView(

                child: Container(

                  alignment:
Alignment.topCenter,

                  decoration:
BoxDecoration(color:
Colors.grey[100]?.withOpacity(0.9),
borderRadius:
BorderRadius.all(Radius.circular(15))),

                  margin: const
EdgeInsets.only(left: 25, right: 25),

                  padding: const
EdgeInsets.symmetric(horizontal: 20,
vertical: 20),

                  child: Column(

                    mainAxisAlignment:
MainAxisAlignment.start,

                    crossAxisAlignment:
CrossAxisAlignment.start,

                    children: [

```

```

      Center(child:
Image.asset('assets/logolong.png', height:
85,)),

```

```

      SizedBox(height: 20,),

```

```

      Text("Welcome Back!", style:
TextStyle(fontFamily: 'Nunito', fontWeight:
FontWeight.w700, fontSize: 17),),

```

```

      Text("Happy to see you
again!", style: TextStyle(fontFamily:
'Nunito', fontWeight: FontWeight.w500,
fontSize: 15)),

```

```

      SizedBox(height: 20,),

```

```

      Form(

```

```

        child: Column(

```

```

          crossAxisAlignment:
CrossAxisAlignment.start,

```

```

          children: [

```

```

            Text("Username", style:
TextStyle(color: Colors.grey[800],
fontFamily: 'nunito'),),

```

```

            SizedBox(height: 5,),

```

```

            TextField( controller:
_email_controller,decoration:
InputDecoration(

```

```

              border:
OutlineInputBorder(),

```

```

              hintText: 'Email',

```

```

            contentPadding:EdgeInsets.only(top: 10,
bottom: 10, left: 20),

```

```

            prefixIcon:
Icon(Icons.person_outline, color:
Color.fromRGBO(251, 171, 80, 1),),

```

```

          ),),

```

```

      SizedBox(height: 20,),

```

```

      Text("Password", style:
TextStyle(color: Colors.grey[800],
fontFamily: 'nunito'),), ),

```

```

      SizedBox(height: 5,),

```

```

      TextField(controller:
_passw_controller,obscureText:
showpass,decoration: InputDecoration(

```

```

        border:
OutlineInputBorder(),

```

```

        hintText: 'Password',

```

```

      contentPadding:EdgeInsets.only(top: 10,
bottom: 10, left: 20),

```

```

      prefixIcon:
Icon(Icons.lock_outline, color:
Color.fromRGBO(251, 171, 80, 1),),

```

```

      suffixIcon:
GestureDetector(

```

```

        onTap: (){
          setState() {

```

```

            showpass =
!showpass;

```

```

          });

```

```

        }, child:
Icon(showpass ? Icons.visibility :
Icons.visibility_off),

```

```

      )

```

```

    ),),

```

```

  ],

```

```

),

```

```

),

```

```

      SizedBox(height: 20,),

```

```

      Center(child:
ElevatedButton(

```



```

        title: const
Text("Something went wrong"),

        content: Text("An
error has occurred while logging in to the
application"),

        actions: [

TextButton(onPressed: () =>
Navigator.pop(context), child: Text("OK"))

        ],

    ));

}

},

        child: Text('Login', style:
TextStyle(color: Colors.grey[800],
fontSize: 18)),

        style:
ElevatedButton.styleFrom(

        primary: yellow,

        padding:
EdgeInsets.symmetric(horizontal: 60,
vertical: 10),

        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))

        ),),

        SizedBox(height: 10,),

        Container(

        padding:
EdgeInsets.symmetric(horizontal: 30),

        child: Row(

        mainAxisAlignment:
MainAxisAlignment.center,

        crossAxisAlignment:
CrossAxisAlignment.center,

```

```

        children: [

        Expanded(child:
Divider(thickness: 2, height: 2, color:
Colors.grey, endIndent: 10,)),

        Text("For Beneficiaries
login with"),

        Expanded(child:
Divider(thickness: 2, height: 2, color:
Colors.grey, indent: 10,)),

        ],

        ),

        ),

        SizedBox(height: 10,),

        Center(

        child:
ElevatedButton(onPressed: (){

        showDialog
<String>(context: context,

        builder: (BuildContext
context) => AlertDialog(

        scrollable: true,

        title: const
Text("Enter Reference Code", style:
TextStyle(fontSize: 25)),

        content:
Padding(padding: EdgeInsets.all(8),

        child: Form(

        child: Column(

        crossAxisAlignment:
CrossAxisAlignment.start,

        children: [

        Text('Code',
style: TextStyle(fontSize: 18)),

```

```

TextFormField(controller: refcode,
decoration: InputDecoration(border:
OutlineInputBorder(),

contentPadding:EdgeInsets.only(top: 10,
bottom: 10, left: 20),),),

),

),),

actions: [

TextButton(onPressed: () =>
Navigator.pop(context), child:
Text("Cancel")),

TextButton(onPressed: ()async{
}, child: Text('Reference Code', style:
TextStyle(color: Colors.grey[800],
fontSize: 18),),style:
ElevatedButton.styleFrom(

primary: yellow,

padding:
EdgeInsets.symmetric(horizontal: 20,
vertical: 10),

shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))

),),

),

SizedBox(height: 30,),

Center(

child: GestureDetector(

child: Text('Use offline
mode for offline survey',

textAlign: TextAlign.end,

style:
TextStyle(decoration:
TextDecoration.underline),),

onTap: (){

if(haveoffcode >0){

showDialog
<String>(context: context,

builder:
(BuildContext context) => AlertDialog(

scrollable: true,

title: const
Text("Enter Offline Code", style:
TextStyle(fontSize: 25)),

content:
StatefulBuilder(

builder:(BuildContext context, StateSetter
setState){

return Padding(padding:
EdgeInsets.all(8),

child: Form(

child: Column(

crossAxisAlignment:
CrossAxisAlignment.start,

children: [

Text('Code', style: TextStyle(fontSize:
18),),

TextFormField(obscureText: showcode
,controller: _offline_code,

decoration: InputDecoration(border:
OutlineInputBorder(),

suffixIcon: IconButton(onPressed: (){

```

```

setState(){
  showcode = !showcode;

  icon: Icon(showcode ? Icons.visibility :
    Icons.visibility_off),

  contentPadding:EdgeInsets.only(top: 10,
    bottom: 10, left: 20),),),

  ),

  ),);

  }

  ),

  actions: [

    TextButton(onPressed: () =>
      Navigator.pop(context), child:
      Text("Cancel")),

    Navigator.pushReplacement(context,
      MaterialPageRoute(builder: (context)=>
        Homepage()));

    }else{

      fluttoast_func('Incorrect Code');

    }

    }, child:

    Text("Submit"))

  ],

  ));

}else{

  fluttoast_func('Login
  to online first to setup device offline code');

  }

  },),

  ),

  ],

  ),

  ),

  ),

  ),

  Visibility(visible: isloading,child:
    Container(padding:
      EdgeInsets.all(20),decoration:
      BoxDecoration(shape:
        BoxShape.rectangle, color:
        Colors.black38),child:
        Center(child:isloading?CircularProgressIn
          dicator(strokeWidth: 10, valueColor:
            AlwaysStoppedAnimation<Color>(
              Color.fromRGBO(75, 79, 177, 1)
            ),):Text("Loading")),)),

  ]

  ),

  ),

  );

  }

  }

  Dashboard Interface

  class Homepage_online extends
    StatefulWidget {

    const Homepage_online({Key? key}) :
    super(key: key);

```

```

@override

State<Homepage_online> createState()
=> _Homepage_onlineState();

}

class _Homepage_onlineState extends
State<Homepage_online> {

  @override

  void initState() {

    count_appl();

    count_pending();

    count_denied();

    loadannouncement();

    // countProf();

    setState(() {

      barangay = globals.user_brgy;

      position = globals.user_perm;

    });

    super.initState();

  }

  @override

  Widget build(BuildContext context) {

    return Scaffold(

      appBar: top_header(context),

      drawer: Navigationmenu(),

      body: WillPopScope(

        onWillPop: () async{

          return await scope_toout(context);

        },

        child: SingleChildScrollView(

```

```

      child: Container(

        child: Column(crossAxisAlignment:
CrossAxisAlignment.start,

        children: [

          Padding(

            padding: const
EdgeInsets.only(top: 10, left: 20),

            child: Text('Welcome ' +
globals.user_name,style:
TextStyle(fontFamily: 'nunito', fontSize:
25,)),

          ),

          Padding(

            padding: const
EdgeInsets.only(left: 20),

            child: Row(children: [

              // Text(numberoflog , style:
TextStyle(fontWeight: FontWeight.w800,
color: Colors.blue, fontSize: 20),),

              Text('Barangay $barangay -
$position', style: TextStyle(fontSize: 18),)

            ]),

          ),

          SizedBox(height: 30,),

          Container(

            margin:
EdgeInsets.fromLTRB(15, 0, 15,0),

            padding:
EdgeInsets.symmetric(horizontal: 30,
vertical: 30),

            decoration:
BoxDecoration(color:
Color.fromRGBO(121, 120, 233, 1),
borderRadius:
BorderRadius.all(Radius.circular(20))),

```

```

        child: Column(
          crossAxisAlignment:
            CrossAxisAlignment.start,
          children: [
            Text('No. of 4p's member',
              style: TextStyle(color: Colors.white,
                fontSize: 20),),
            Row(
              children: [
                Visibility(visible: waiting,
                  child: Text("Loading Data...", style:
                    TextStyle(color: Colors.white, fontWeight:
                      FontWeight.w900, fontSize: 30),)),
                Visibility(visible: !waiting,
                  child: Text(num_of_applicants.toString(),
                    style: TextStyle(color: Colors.white,
                      fontWeight: FontWeight.w900, fontSize:
                        30),)),
                // Text(' ' + dateupdated,
                  style: TextStyle(color: Colors.white,
                    fontSize: 20),)
              ],
            )
          ],
        ),
        SizedBox(height: 10,),
        Container(
          margin:
            EdgeInsets.fromLTRB(15, 0, 15,0),
          padding:
            EdgeInsets.symmetric(horizontal: 30,
              vertical: 30),
          decoration:
            BoxDecoration(color: Color.fromRGBO(71,

```

```

71, 161, 1), borderRadius:
            BorderRadius.all(Radius.circular(20))),
        child: Column(
          crossAxisAlignment:
            CrossAxisAlignment.start,
          children: [
            Text('No. of Pending
              Application', style: TextStyle(color:
                Colors.white, fontSize: 20),),
            Row(
              children: [
                Text(numberofpedning,
                  style: TextStyle(color: Colors.white,
                    fontWeight: FontWeight.w900, fontSize:
                      30),),
                // Text(' ' + dateupdated,
                  style: TextStyle(color: Colors.white,
                    fontSize: 20),)
              ],
            )
          ],
        ),
        SizedBox(height: 10,),
        Container(
          margin:
            EdgeInsets.fromLTRB(15, 0, 15,0),
          padding:
            EdgeInsets.symmetric(horizontal: 30,
              vertical: 30),
          decoration:
            BoxDecoration(color:
              Color.fromRGBO(233, 120, 137, 1.0),
              borderRadius:
                BorderRadius.all(Radius.circular(20))),

```



```

        ),
        ),
        ),
    );
}

Widget widget_barangays(String brgy,
String data){

    return Column(

        children: [

            Row(

                children: [

                    Expanded(child: Text(brgy, style:
TextStyle(fontSize: 18, fontWeight:
FontWeight.w400)),),

                    Expanded(child: Text(data, style:
TextStyle(fontSize: 18, fontWeight:
FontWeight.w400)),),

                ],

            ),

            SizedBox(height: 10,),

        ],

    );
}

class Dash_Pages extends StatefulWidget
{

    const Dash_Pages({Key? key}) :
super(key: key);

    @override

    State<Dash_Pages> createState() =>
_Dash_PagesState();
}

```

```

class _Dash_PagesState extends
State<Dash_Pages> {

    @override

    Widget build(BuildContext context) {

        return Column(

            children: [

                Container(

                    width: 500,

                    height: 140,

                    child: PageView(

                        controller: _pageController,

                        onPageChanged: (index){

                            setState() {

                                pagechange = index;

                                isshown();

                            });

                        },

                        children: <Widget> [

                            // Text('data1'),

                            // Text('data2'),

                            // Text('data3')

                        ],

                        Container(

                            margin:

                                EdgeInsets.fromLTRB(15, 0, 15,0),

                            padding:

                                EdgeInsets.symmetric(horizontal: 30,
vertical: 30),

                            decoration:

                                BoxDecoration(color:
Color.fromRGBO(125, 160, 250, 1),
borderRadius:
BorderRadius.all(Radius.circular(20))),

```

```

        child: Column(
          crossAxisAlignment:
            CrossAxisAlignment.start,
          children: [
            Text('Digos City\'s Barangay',
              style: TextStyle(color: Colors.white,
                fontSize: 20)),
            Row(
              children: [
                Text(barangay_quant,
                  style: TextStyle(color: Colors.white,
                    fontWeight: FontWeight.w900, fontSize:
                    30)),
                Text(' Barangays', style:
                  TextStyle(color: Colors.white, fontSize:
                    20)),
              ],
            ),
          ],
        ),
        Container(
          margin:
            EdgeInsets.fromLTRB(15, 0, 15, 0),
          padding:
            EdgeInsets.symmetric(horizontal: 30,
              vertical: 30),
          decoration:
            BoxDecoration(color: Color.fromRGBO(71,
              71, 161, 1), borderRadius:
              BorderRadius.all(Radius.circular(20))),
          child: Column(
            crossAxisAlignment:
              CrossAxisAlignment.start,
            children: [

```

```

              Text('Number of Data
                Collected', style: TextStyle(color:
                  Colors.white, fontSize: 20)),
              Row(
                children: [
                  Text(databasecount.toString(), style:
                    TextStyle(color: Colors.white, fontWeight:
                      FontWeight.w900, fontSize: 30)),
                  // Text(' ' + dateupdated,
                    style: TextStyle(color: Colors.white,
                      fontSize: 20)),
                ],
              )
            ],
          ),
          ),
        SizedBox(height: 5,),
        Container(
          child: Center(
            child: Row(
              crossAxisAlignment:
                CrossAxisAlignment.center,
              mainAxisAlignment:
                MainAxisAlignment.center,
              children: [
                Container(
                  decoration: BoxDecoration(

```

```

        color: _color[0],

        shape: BoxShape.circle

    ),

    height: 8,

    width: 8,

),

SizedBox(width: 5,),

Container(

    decoration: BoxDecoration(

        color: _color[1],

        shape: BoxShape.circle

    ),

    height: 8,

    width: 8,

),

],

),

),

),

],

);

}

}

```

Application Form

```

// StreamController <bool>
streamController =
StreamController<bool>();

class add_applicant_online extends
StatefulWidget {

```

```

    const add_applicant_online({Key? key}) :
super(key: key);

    @override

    State<add_applicant_online>
createState() =>
add_applicant_onlineState();

}

class add_applicant_onlineState extends
State<add_applicant_online> {

    @override

    void initState() {

        _contact_city.text = 'Digos City';

        _contact_province.text = 'Davao del
Sur';

        super.initState();

    }

    void dispose(){

        _scrollController.dispose();

        super.dispose();

    }

    void _scrollToTop() {

        _scrollController.animateTo(0,

            duration: const Duration(milliseconds:
150), curve: Curves.linear);

    }

    @override

    Widget build(BuildContext context) {

        return Scaffold(

            key: _scaffoldkey,

            resizeToAvoidBottomInset: true,

            appBar: top_header(context),

```

```

drawer: Navigationmenu(),
body: WillPopScope(
  onWillPop: ()async{
    final result = await showDialog
<bool> (
      context: context,
      builder: (context){
        return AlertDialog(
          title: Text('Exiting the form?'),
          content: Text('You are about
to EXIT the from, work will not be save'),
          actions:[
            TextButton(onPressed: () =>
Navigator.of(context).pop(false), child:
Text('Cancel')),
            TextButton(onPressed: (){
              if(!globals.online_access){
                Navigator.of(context).pop(true);

                Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) =>
Homepage()));
              }else{
                Navigator.of(context).pop(true);

                Navigator.pushReplacement(context,
MaterialPageRoute(builder: (context) =>
Homepage_online()));
              }
            } , child: Text('Close'))
          ],
        );
      },
    );
    return Future.value(result);
  },
  child: Stack(
    children: [
      Column(
        children: [
          Flexible(
            flex: 1,
            child: Container(
              margin:
EdgeInsets.symmetric( vertical: 10),
              child: SingleChildScrollView(
                scrollDirection:
Axis.horizontal,
                child: Row(
                  children: [
                    SizedBox(width: 10,),
                    // ----- qualification

                    ElevatedButton(onPressed: (){
                      DBhelper_qualification
path = new DBhelper_qualification();

                      getDatabasesPath().then((value) =>
print(value));

                      _pagecontroller.jumpToPage(0);
                    },

```

```

        child:
Text('Qualification'),

        style:
ElevatedButton.styleFrom(primary:
Color.fromRGBO(75, 79, 177, 1),

        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10))),

    ),

    SizedBox(width: 10,),

    //-----profile

ElevatedButton(onPressed:
enable_profile? (){

_pagecontroller.jumpToPage(1);

        }:null,

        child: Text('Profile'),

        style:
ElevatedButton.styleFrom(primary:
Color.fromRGBO(75, 79, 177, 1),

        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)), onSurface:
Color.fromRGBO(75, 79, 177, 1)),

    ),

    SizedBox(width: 10,),

    //-----contact

ElevatedButton(onPressed: enable_cont?
){

_pagecontroller.jumpToPage(2);

        }:null,

        child: Text('Address'),

```

```

        style:
ElevatedButton.styleFrom(primary:
Color.fromRGBO(75, 79, 177, 1),

        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)), onSurface:
Color.fromRGBO(75, 79, 177, 1)),

    ),

    SizedBox(width: 10,),

    // ----- children

    Visibility(

        visible: true,

        child:
ElevatedButton(onPressed: enable_child?
){

_pagecontroller.jumpToPage(3);

        }:null,

        child: Text('Members'),

        style:
ElevatedButton.styleFrom(primary:
Color.fromRGBO(75, 79, 177, 1),

        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),onSurface:
Color.fromRGBO(75, 79, 177, 1)),

    ),

    ),

    ],

    ),

    ),

    ),

    ),

```

```

// ----- for
forms -----

Flexible(...)

],

)

],

),

);

}

//End image picker -----
-----

}

Sync Module

class Syncdata extends StatefulWidget {

  @override

  State<Syncdata> createState() =>
  _SyncdataState();

class _SyncdataState extends
State<Syncdata> {

  @override

  void initState() {

    countProf();

    super.initState();

  }

Widget build(BuildContext context) {

  return Scaffold(

    appBar: top_header(context),

    drawer: Navigationmenu(),

```

```

body: WillPopScope(

  onWillPop: ()async{

    scope_dashboard(context);

    return false;

  },

  child: Stack(

    children: [

      Center(

        child: SingleChildScrollView(

          child: Column(

            mainAxisAlignment:
MainAxisAlignment.center,

            children: [

              Image.asset('assets/online_sync.png'),

              Text('Sync Data to Database',
style: TextStyle(fontSize: 20, fontWeight:
FontWeight.w800),),

              SizedBox(height: 20,),

              ElevatedButton(onPressed: notosync ? ()
async {}:null, child: Text('Sync', style:
TextStyle(fontSize: 20,),), style:
ElevatedButton.styleFrom(primary:
Color.fromRGBO(75, 79, 177, 1)),),

                ], child: Text('Text'))

            ],

          ),

        ),

      ),

      Visibility(visible: isloading, child:
Container(padding:
EdgeInsets.all(20), decoration:

```

```

BoxDecoration(shape:
BoxShape.rectangle, color:
Colors.black87),child: Column(

    mainAxisAlignment:
MainAxisAlignment.center,

    crossAxisAlignment:
CrossAxisAlignment.center,

    children: [

        Center(child: SizedBox(height:
70, width: 70,child:
isLoading?CircularProgressIndicator(

            backgroundColor:
Color.fromRGBO(237, 237, 238, 1.0),
strokeWidth: 10, valueColor:
AlwaysStoppedAnimation<Color>(

                Color.fromRGBO(75, 79, 177,
1)

            ),

        ):Text("Loading")),),

        SizedBox(height: 20,),

        text('Please wait while we sync
your data'),

        SizedBox(height: 40,),

        text(syncingnow),

    ],

    )),

]

),

),

);

}

Widget text(String data){

```

```

return Text(data, style:
TextStyle(fontSize: 25, fontWeight:
FontWeight.w600,

    color: Color.fromRGBO(255, 255,
255, 1.0)),textAlign: TextAlign.center,);

}

}

```

Mapping Module

```

import 'dart:convert';

import 'dart:typed_data'

import
'package:e4psmap/config/APIs.dart';

import
'package:e4psmap/funtions/textwidget.dart
';

import 'package:flutter/material.dart';

import
'package:flutter_inappwebview/flutter_inap
pwebview.dart';

import '.././config/variables.dart' as
globals;

import '.././funtions/Header.dart';

import '.././funtions/sidemenu.dart';

import '.././funtions/willscope_router.dart';

import 'dashboard.dart';

import 'dashboard_online.dart';

class Mappage extends StatefulWidget {

    const Mappage({Key? key}) : super(key:
key);

    @override

    State<Mappage> createState() =>
_MappageState();

}

```

```

class _MappageState extends
State<Mappage> {

  bool flutter_map = true;

  bool web_map = true;

  String user_brgy = globals.user_brgy;

  bool isloadingmap = false;

  String buttontxt = "Switch to Heatmap";

  @override

  Widget build(BuildContext context) {

    return Scaffold(

      floatingActionButton: ElevatedButton(

        style:
ElevatedButton.styleFrom(primary:
Color.fromRGBO(75, 79, 177, 1) ),

        onPressed: (){

          setState() {

            web_map = !web_map;

            isloadingmap = true;

          });

          if(web_map){

            setState() {

              buttontxt = "Switch to Heatmap";

            });

          }else if (!web_map){

            setState() {

              buttontxt = "Switch to Points
Map";

            });

          }

        },


```

```

        child: Text(buttontxt),

      ),

      appBar: top_header(context),

      drawer: Navigationmenu(),

      body: WillPopScope(

        onWillPop: ()async{

          scope_dashboard(context);

          return false;

        },

        child: Container(

          child: Center(

            child: Stack(

              children:[ Column(

                children: [

                  Visibility(visible:
isloadingmap,child: Container(

                    margin: EdgeInsets.all(40),

                    child: Center(child: Column(

                      children: [

                        CircularProgressIndicator(strokeWidth: 10,
valueColor:
AlwaysStoppedAnimation<Color>(

                          Color.fromRGBO(75,
79, 177, 1)

                        ),),

                      SizedBox(height: 20,),

                      label('Loading Map', 20)

                    ],

                  ),),

                ),),


```

```

Visibility(visible: web_map, child:
Flexible(child: InAppWebView(

initialUrlRequest: URLRequest(

url: Uri.parse(api_config.e4ps_map),

method: 'POST',body:
Uint8List.fromList(utf8.encode("barangay=
$user_brgy")),headers: { 'Content-Type':
'application/x-www-form-urlencoded' } ),

onWebViewCreated: (controller) {},

onLoadStart: (controller, url){

setState() { isloadingmap = true; });

},

onLoadStop: (controller, url){

setState() {isloadingmap = false;});

}, ), ),

Visibility(visible: !web_map,child: Flexible(

child: InAppWebView(

initialUrlRequest: URLRequest(

url: Uri.parse(api_config.heatmap_api),

method: 'POST',body:
Uint8List.fromList(utf8.encode("barangay=
$user_brgy")),

headers: {'Content-Type': 'application/x-
www-form-urlencoded'}),

onWebViewCreated: (controller) {},

onLoadStart: (controller,
url

```

```

setState() {

isloadingmap = true;

});

},

onLoadStop: (controller,

setState() {

isloadingmap = false;

});

},

),

),

],

)

),

),

),

),

);

}

}

```

APPENDIX C
Certificate of Permission



CERTIFICATE
OF RECOGNITION

is Given to

Ralph Ian O. Noynay

As the recipient for the GODDESS (Governance through Data Science and Decision Support System) project funded by DOST (Department of Science and Technology).

Given this 22nd day of August, 2022 at University of Mindanao Digos College.


CONRADO B. PANERION JR., MAT
Research Coordinator


EDUARD L. PULVERA, MSIS
Dean of College


DR. JOHN VIANNE MURCIA
Project Leader



CERTIFICATE
OF RECOGNITION

is Given to

Rhona Mae Magsayo

As the recipient for the GODDESS (Governance through Data Science and Decision Support System) project funded by DOST (Department of Science and Technology).

Given this 22nd day of August, 2022 at University of Mindanao Digos College.


CONRADO B. PANERION JR., MAT
Research Coordinator


EDUARD L. PULVERA, MSIS
Dean of College


DR. JOHN VIANNE MURCIA
Project Leader



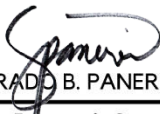
CERTIFICATE OF RECOGNITION

is Given to

Jonalyn E. Calvo

As the recipient for the GODDESS (Governance through Data Science and Decision Support System) project funded by DOST (Department of Science and Technology).

Given this 22nd day of August, 2022 at University of Mindanao Digos College.


CONRADO B. PANERION JR., MAT
Research Coordinator


EDUARD L. PULVERA, MSIS
Dean of College


DR. JOHN VIANNE MURCIA
Project Leader

APPENDIX D

Routing Form

 The University of Mindanao	RESEARCH AND PUBLICATION CENTER ☐ Main ☒ Branch <u>Digos</u> ROUTING FORM				
Title: <u>oqprMap: A Mobile Application for 4ps beneficiaries for the City of Digos with Geo-tagging Technology</u>					
Proponents: <u>RALPH JAN O. NOYNAY</u> Program <u>DTP/BSIT</u> <u>RHONA MAB MAGSAYO</u> Course Code <u>876</u> <u>JONALYN E. CALVO</u> Semester/SY <u>1st semester / 2022-2023</u>					
Name	Date Received	Signature	Date Released	Signature	Remarks
Adviser (5 days)	June 5, 2023		June 12, 2023		
John Raven V. Manulat, MIT	July 26, 2023		July 26, 2023		only for handbook.
Statistician (3 days)					
Panel 1 (RC/ARC) (5 days)	June 18, 2023		June 23, 2023		Follow as w format of Chapter 9 and 5
Joane May B. Delima, MIT					
Panel 2 (2 days)	June 26, 2023		June 28, 2023		
Shone Ryan A. Diaz, MIT					
Panel 3 (2 days)	July 1, 2023		July 6, 2023		Pls see with
Edward L. Pulvera, MSIS					
Plagiarism Check (3 days)	July 7, 2023		July 7, 2023		
Joane May B. Delima, MIT					
Grammarly Check (1 day)	July 24, 2023		July 24, 2023		
John Raven V. Manulat, MIT					
Dean (2 days)	July 27, 2023		July 27, 2023		
EDUARDO L. PULVERA, MSIS					

APPENDIX E

Assignment of Research Personnel Form

	RESEARCH AND PUBLICATION CENTER ☐ Main ☒ Branch <u>Digos</u> ASSIGNMENT OF RESEARCH PERSONNEL
---	--

Course Code: 876

Program: DTP / BSIT

This is to acknowledge acceptance of assignment as Research Personnel for the thesis entitled:

e4psMap: A MOBILE APPLICATION FOR 4PS BENEFICIARIES FOR THE CITY
OF DIGOS WITH GEO TAGGING TECHNOLOGY

	Name	Signature	Date
Adviser	<u>JOHN RAVEN V. MANULAT, MIT</u>		_____
Statistician/Data Analyst	_____		_____
Editor	<u>JOHN RAVEN V. MANULAT, MIT</u>		_____
Panel Members	<u>EDUARD L. PULVERA, MSIS</u>		_____
	<u>JOANÉ MAY B. DEUMA, MIT</u>		_____
	<u>SHANE A. DIAZ, MIT</u>		_____
Endorsed by:	<u>CONRADO B. PANERIO JR., MAT</u> Research Coordinator/Asst. Research Coordinator		_____
Approved by:	<u>EDUARD L. PULVERA, MSIS</u> Dean of College		_____

APPENDIX F

Grammarly Report



Report: E4PSMAP: A MOBILE APPLICATION FOR 4PS BENEFICIARIES FOR THE CITY OF DIGO...

E4PSMAP: A MOBILE APPLICATION FOR 4PS BENEFICIARIES FOR THE CITY OF DIGOS WITH GEO TAGGING TECHNOLOGY

by Joane Delima

General metrics

58,836	9,174	512	36 min 41 sec	1 hr 10 min
characters	words	sentences	reading time	speaking time

Writing Issues

 No issues found

Plagiarism

This text hasn't been checked for plagiarism

APPENDIX G

Plagiarism Result



Digital Receipt

This receipt acknowledges that Turnitin received your paper. Below you will find the receipt information regarding your submission.

The first page of your submissions is displayed below.

Submission author: **Ralph Ian Noynay, Jona Calvo Rhona Magsayo**
 Assignment title: **INFORMATION TECHNOLOGY**
 Submission title: **e4PsMap: A Mobile Application for 4Ps Beneficiaries for the ...**
 File name: **e4PsMap_2023_Turnitin.docx**
 File size: **2.81M**
 Page count: **18**
 Word count: **9,188**
 Character count: **48,196**
 Submission date: **07-Jul-2023 10:10AM (UTC+0800)**
 Submission ID: **2127497054**



e4PsMap: A Mobile Application for 4Ps Beneficiaries for the City of Digos with Geo Tagging Technology

ORIGINALITY REPORT

12%

SIMILARITY INDEX

9%

INTERNET SOURCES

4%

PUBLICATIONS

6%

STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Mindanao Student Paper	2%
2	www.ijrte.org Internet Source	1%
3	www.researchgate.net Internet Source	1%
4	www.sci-int.com Internet Source	1%
5	www.bau.edu.lb Internet Source	1%
6	Submitted to Quezon City University Student Paper	1%
7	www.emro.who.int Internet Source	1%
8	www.academia.edu Internet Source	1%
9	mcsprogram.org Internet Source	1%

10	www.econstor.eu Internet Source	<1 %
11	Submitted to Tarlac State University Student Paper	<1 %
12	Submitted to Angeles University Foundation Student Paper	<1 %
13	doc.arcgis.com Internet Source	<1 %
14	Submitted to Southampton Solent University Student Paper	<1 %
15	Submitted to Cavite State University Student Paper	<1 %
16	pdfcoffee.com Internet Source	<1 %
17	Submitted to Lyceum of the Philippines University Student Paper	<1 %
18	serp-p.pids.gov.ph Internet Source	<1 %
19	I Gede Susrama Mas Diyasa, Dwi Arman Prasetya, Mohammad Idhom, Anggraini Puspita Sari, Anuar Mohamed Kassim. "Implementation of Haversine Algorithm and Geolocation for travel recommendations on Smart Applications for Backpackers in Bali",	<1 %

Curriculum Vitae

RALPH IAN O. NOYNAY

1238, Lapu-Lapu Extension, San Miguel
Digos City, Davao del Sur
+639290699248
ralphnoynay03@gmail.com



PERSONAL INFORMATION

Birthday: September 3, 2001	Age: 21
Nationality: Filipino	Religion: Roman Catholic
Civil Status: Single	Gender: Male

EDUCATIONAL ATTAINMENT

College	Bachelor of Science in Information Technology UM Digos College, Digos City 2019-2023
Senior High School	Polytechnic College of Davao del Sur Digos City, Davao del Sur 2017-2019
Secondary	Sawata National High School Sawata, San Isidro, Davao del Norte 2013-2017
Elementary	Mamangan Elementary School Mamangan, San Isidro, Davao del Norte 2007-2013

TRAININGS/ SEMINARS/ACCOMPLISHMENTS

- PSITS Programming Contest
- 7th TOPCIT Philippines
- Getting Grounded on Analytics Certification
- Huawei ICT Competition 2022-2023
- Upscaled Webinar Series
- Mindanao Conference for IT Students (MCITE)
- Seminar Workshop for Capstone Project Writing
- Cyber Smart Online Caravan

SKILLS

- Computer Programming
- Mobile Development
- Web development
- MS Office (Word, Powerpoint, Excel)
- Basic Computer Troubleshooting

CURRICULUM VITAE

JONALYN E. CALVO

Purok 2 Barangay Matti
Digos City, Davao del Sur
+63909075897478
Jonalyncalvo041@gmail.com



PERSONAL INFORMATION

Birthday: May 05, 1997	Age: 26
Nationality: Filipino	Religion: Roman Catholic
Civil Status: Single	Gender: Female

EDUCATIONAL ATTAINMENT

College	Bachelor Of Science In Information Technology UM Digos College, Digos City (2019-2023)
Secondary	Matti National High School Matti, Digos City Davao del Sur (2010-2013)
Elementary	Remedios N. Saplala Elementary School Matti, Digos City Davao del Sur (2006-2010)

TRAININGS/ SEMINARS/ACCOMPLISHMENTS

- Certified Data Analysis in Getting Grounded (University of Mindanao)
- Development Academy of the Philippines SP101 Certificate
- IT Congress at Mapua Malayan Colleges Mindanao
- Technopreneurship Pitching Seminar (University of Mindanao)
- StackLeague x SympH Tech Session
- Seminar Workshop on Practical Guide to Capstone Project Writing in Computing (University of Mindanao)
- WordPress Webinar 2021 (University of Mindanao)

SKILLS

- Microsoft office, Excel and Powerpoint
- Editing Pictures and Videos
- Photography
- Graphics Designing

CURRICULUM VITAE

RHONA MAE MAGSAYO

Purok 4 Malabang Hagonoy
Davao del Sur, 8006
+639271330037
rhonamaemagsayo@gmail.com



PERSONAL INFORMATION

Birthday: September 15, 2000
Nationality: Filipino
Civil Status: Single

Age: 22
Religion: Roman Catholic
Gender: Female

EDUCATION ATTAINMENT

College	Bachelor Of Science In Information Technology UM Digos College, Digos City 2019-2024
Senior High School	Mary Mediatrix Of All Graces Academy, Inc. Corner Mabini Extension – Sta. Ana Avenue, San Miguel, Digos City 2016-2018
Secondary	Davao del Sur School of Fisheries Bagumbayan, Malalag Davao del Sur 2015-2017
Elementary	Leoncio S. Peliño Elementary School Malabang Hagonoy Davao del Sur 2012-2013

TRAININGS/ SEMINARS/ACCOMPLISHMENTS

- Webinar Session: Prototyping 2021
- Food Processing NCII (Davao City)
- Technopreneurship Pitching 101 Seminar
- WordPress Webinar 2021 (University of Mindanao)
- Getting Grounded on Analytics (University of Mindanao)
- Electronic Products Assembly and Servicing NCII (University of Mindanao)
- Computer System Servicing NCII (Mary Mediatrix of All Graces Academy, Inc)

- Information Technology Research Congress (Mapua Malayan Colleges Mindanao)
- Seminar Workshop on Practical Guide to Capstone Project Writing in Computing (University of Mindanao)

SKILLS

- Video Editing
- Word Processing
- Presentation Software
- Computer Networking